Framework for Evaluating Explainability Methods in Artificial Intelligence

User Manual
XAIMetrics version 1.0
October 15, 2026

Maria Teresa Alba Rueda
Amelia Zafra Gómez

This user guide is for the XAIMetrics library (version 1.0). Copyright ©2026 Maria Teresa Alba Rueda and Amelia Zafra Gómez.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 3.0 or any later version published by the Free Software Foundation; with no Invariant Sections, with no Front-Cover Texts, and with no Back-Cover Texts.

Contents

Acronyms 1

1 Introduction 3

2 Getting and running the library 5

3 XAIMetrics library 7

3.1 XAIMetrics evaluation taxonomy . . . 8

3.1.1 Complexity . . . 8

3.1.2 Faithfulness . . . 9

3.1.3 Fidelity . . . 9

3.1.3.1 Completeness . . . 10

3.1.3.2 Soundness . . . 10

3.1.4 Robustness . . . 10

3.1.5 Sensitivity . . . 11

3.2 XAIMetrics library architecture . . . 11

3.3 Configuration and file formats . . . 15

3.3.1 Configuration file . . . 15

3.3.2 Automatic context discovery . . . 16

3.3.3 Explicit context definition . . . 17

3.3.4 Input data files . . . 18

3.3.5 Model files . . . 18

3.3.6 Attribution files . . . 19

3.3.7 Evaluation report files . . . 19

4 Running Available Metrics in the Library 21

4.1 Faithfulness . . . 21

4.1.1 Consistency . . . 21

4.1.1.1 Direct evaluation using the Consistency class . . . 22

4.1.1.2 Automated evaluation using run_evaluation() . . . 23

4.1.2 Faithfulness Estimate . . . 23

4.1.2.1 Direct evaluation using the FaithfulnessEstimate class . . . 24

4.1.2.2 Automated evaluation using run_evaluation() . . . 25

4.1.3 Faithfulness . . . 25

4.1.3.1 Direct evaluation using the Faithfulness class . . . 27

4.1.3.2 Automated evaluation using run_evaluation() . . . 27

4.1.4 Monotonicity Correlation . . . 28

4.1.4.1 Direct evaluation using the MonotonicityCorrelation class . . . 29

4.1.4.2 Automated evaluation using run_evaluation() . . . 30

4.1.5 Monotonicity Metric . . . 30

4.1.5.1 Direct evaluation using the MonotonicityMetric class . . . 32

4.1.5.2 Automated evaluation using run_evaluation() . . . 32

4.1.6 Monotonicity . . . 33

4.1.6.1 Direct evaluation using the Monotonicity class . . . 34

4.1.6.2 Automated evaluation using run_evaluation() . . . 34

4.1.7 Sensitivity-N . . . 35

4.1.7.1 Direct evaluation using the SensitivityN class . . . 36

4.1.7.2 Automated evaluation using run_evaluation() . . . 37

4.1.8 Sufficiency . . . 38

4.1.8.1 Direct evaluation using the Sufficiency class . . . 39

4.1.8.2 Automated evaluation using run_evaluation() . . . 39

4.2 Complexity . . . 40

4.2.1 Complexity . . . 40

4.2.1.1 Direct evaluation using the Complexity class . . . 41

4.2.1.2 Automated evaluation using run_evaluation() . . . 41

4.2.2 Sparseness . . . 42

4.2.2.1 Direct evaluation using the Sparseness class . . . 43

4.2.2.2 Automated evaluation using run_evaluation() . . . 43

4.3 Sensitivity . . . 44
4.3.1 Average Sensitivity . . . 44
4.3.1.1 LIME explanation function . . . 46
4.3.1.2 Direct evaluation using the AvgSensitivity class . . . 46
4.3.1.3 Automated evaluation using run_evaluation() . . . 47
4.3.2 Model Randomization . . . 48
4.3.2.1 Direct evaluation using the ModelRandomization class . . . 50
4.3.2.2 Automated evaluation using run_evaluation() . . . 51
4.3.3 Random Logit . . . 51
4.3.3.1 Direct evaluation using the RandomLogit class . . . 55
4.3.3.2 Automated evaluation using run_evaluation() . . . 55
4.4 Robustness . . . 56
4.4.1 Average Stability . . . 56
4.4.1.1 Direct evaluation using the AverageStability class . . . 59
4.4.1.2 Automated evaluation using run_evaluation() . . . 60
4.4.2 Local Lipschitz Estimate . . . 61
4.4.2.1 Direct evaluation using the LocalLipschitzEstimate class . . . 62
4.4.2.2 Automated evaluation using run_evaluation() . . . 63
4.4.3 Max Sensitivity . . . 64
4.4.3.1 Direct evaluation using the MaxSensitivity class . . . 66
4.4.3.2 Automated evaluation using run_evaluation() . . . 66
4.4.4 MeGe . . . 67
4.4.4.1 Model training function . . . 71
4.4.4.2 Direct evaluation using the MeGe class . . . 71
4.4.4.3 Automated evaluation using run_evaluation() . . . 72
4.4.5 Relative Input Stability . . . 73
4.4.5.1 Direct evaluation using the RelativeInputStability class . . . 74
4.4.5.2 Automated evaluation using run_evaluation() . . . 75
4.4.6 Relative Output Stability . . . 76
4.4.6.1 Direct evaluation using the RelativeOutputStability class . . . 78
4.4.6.2 Automated evaluation using run_evaluation() . . . 78
4.5 Fidelity . . . 79
4.5.1 Completeness . . . 79

4.5.1.1 Average Drop . . . 79
4.5.1.1.1 Direct evaluation using the AverageDrop class . . . 82
4.5.1.1.2 Automated evaluation using run_evaluation() . . . 83
4.5.1.2 Average Gain . . . 84
4.5.1.2.1 Direct evaluation using the AverageGain class . . . 87
4.5.1.2.2 Automated evaluation using run_evaluation() . . . 87
4.5.1.3 Average Increase . . . 88
4.5.1.3.1 Direct evaluation using the AverageIncrease class . . . 91
4.5.1.3.2 Automated evaluation using run_evaluation() . . . 92
4.5.1.4 Completeness . . . 92
4.5.1.4.1 Direct evaluation using the Completeness class . . . 94
4.5.1.4.2 Automated evaluation using run_evaluation() . . . 94
4.5.1.5 Deletion . . . 95
4.5.1.5.1 Direct evaluation using the Deletion class . . . 98
4.5.1.5.2 Automated evaluation using run_evaluation() . . . 99
4.5.1.6 Insertion . . . 99
4.5.1.6.1 Direct evaluation using the Insertion class . . . 103
4.5.1.6.2 Automated evaluation using run_evaluation() . . . 104
4.5.1.7 MuFidelity . . . 104
4.5.1.7.1 Direct evaluation using the MuFidelity class . . . 109
4.5.1.7.2 Automated evaluation using run_evaluation() . . . 109
4.5.2 Soundness . . . 110
4.5.2.1 Non-Sensitivity . . . 110
4.5.2.1.1 Direct evaluation using the NonSensitivity class . . . 112
4.5.2.1.2 Automated evaluation using run_evaluation() . . . 112

5 Running Available XAI Methods in the Library 115

5.1 Local Interpretable Model-agnostic Explanations (LIME) . . . 116

5.1.1 Direct explanation using the LIMEExplainer class . . . 119

5.1.2 Automated explanation using run_explanation() . . . 119

5.2 SHapley Additive exPlanations (SHAP) . . . 120

5.2.1 Direct explanation using the SHAPExplainer class . . . 124

5.2.2 Automated explanation using run_explanation() . . . 124

5.3 BreakDown . . . 125

5.3.1 Direct explanation using the BreakDownExplainer class . . . 129

5.3.2 Automated explanation using run_explanation() . . . 129

5.4 Model Agnostic Supervised Local Explanations (MAPLE) . . . 130

5.4.1 Direct explanation using the MAPLEExplainer class . . . 134

5.4.2 Automated explanation using run_explanation() . . . 135

5.5 Running and comparing several explanation methods . . . 136

6 Extending the XAIMetrics Library 139

6.1 Adding a New Metric . . . 139

6.1.1 Metric Implementation Process . . . 140

6.1.2 Metric Interface Requirements . . . 141

6.1.3 Example Metric Implementation . . . 142

6.1.4 Metric Configuration and Execution . . . 142

6.1.5 Testing a New Metric . . . 143

6.2 Adding a New Explainer . . . 143

6.2.1 Explainer Implementation Process . . . 144

6.2.2 Explainer Interface Requirements . . . 145

6.2.3 Example Explainer Implementation . . . 145

6.2.4 Explainer Configuration and Execution . . . 146

6.2.5 Testing a New Explainer . . . 146

6.3 Documentation and Contribution Checklist . . . 147

7 Reporting bugs 149

Bibliography 151

Acronyms

AI: Artificial Intelligence

API: Application Programming Interface

CPU: Central Processing Unit

CUDA: Compute Unified Device Architecture

Git LFS: Git Large File Storage

LIME: Local Interpretable Model-agnostic Explanations

MAPLE: Model Agnostic Supervised Local Explanations

NaN: Not a Number

RMSE: Root Mean Squared Error

SHAP: SHapley Additive exPlanations

SSIM: Structural Similarity Index

XAI: Explainable Artificial Intelligence

CHAPTER

1

Introduction

The rapid advancement of Artificial Intelligence (AI) techniques has transformed numerous sectors, including healthcare, finance, cybersecurity, and logistics. However, this progress has also highlighted an important limitation: many of these models lack of transparency, particularly those regarded as black-box systems [1]. In many real-world contexts, accurate predictions are not enough; it is also essential to understand how and why a model makes certain decisions, especially when those decisions must be interpretable, auditable, or justifiable [2].

In this context, Explainable Artificial Intelligence (XAI) [3] aims to provide understandable explanations of model behaviour. Among the different approaches proposed in the literature, local explanation methods such as SHapley Additive exPlanations (SHAP) [4], Local Interpretable Model-agnostic Explanations (LIME) [5], *Integrated Gradients* [6], and *BreakDown* [7] have gained considerable relevance because they make it possible to analyse the contribution of input variables to specific decisions, supporting tasks such as model validation, bias detection, and increased trust in AI systems.

Despite the significant development of XAI techniques, evaluating the quality of explanations remains an open and complex problem [2]. While predictive models can be compared using widely accepted performance metrics, the evaluation of explanations still lacks a common set of measures for objectively assessing explanation quality. Numerous metrics have therefore been proposed to evaluate different explanation properties, including fidelity, robustness, sensitivity, interpretability, and complexity [8, 9]. Nevertheless, these metrics are typically implemented in isolation and distributed across different tools and *ad hoc* experimental settings, making it difficult to configure, compare, reproduce, and interpret evaluation results consistently [10, 11]. Consequently, unified frameworks are required to organize metrics, standardize evaluation procedures, and support reproducible comparative studies of XAI methods.

To address this limitation, `XAIMetrics` provides a unified framework for the evaluation of explanations generated by local attribution-based XAI methods. The library combines a taxonomy of explainability metrics, a modular software architecture and an experimental workflow that supports the generation of explanations, the execution of evaluation metrics and the creation of comparative reports. The current version of the framework implements 17 evaluation metrics organised according to the proposed taxonomy and integrates four local, model-agnostic explanation methods: `LIME`, `SHAP`, `BreakDown` and `MAPLE`. Experiments can be defined through `YAML` configuration files, allowing users to execute complete evaluation studies without modifying the library source code.

The main objective of `XAIMetrics` is to facilitate systematic and reproducible experimental studies, enabling researchers and practitioners to compare explanations generated by different XAI methods across multiple datasets, models and evaluation metrics under homogeneous conditions.

This manual describes the taxonomy adopted by the framework (Section **??**), its architecture (Section **??**), the implemented evaluation metrics (Section **??**), the supported XAI methods (Section **??**), and the procedures required to execute experiments and analyse the generated results. Finally, Section **??** is devoted to the extensibility mechanisms of the framework, allowing users to incorporate new metrics, explanation methods and experimental workflows while preserving the existing evaluation pipeline.

CHAPTER

2

Getting and running the library

This chapter describes the steps required to download, install, and use the **XAIMetrics** library.

XAIMetrics is distributed as a Python package through its GitHub repository, available at `https://github.com/kdis-lab/XAIMetrics`. The current version of the package requires Python `3.13` or later.

Table 2.1 lists the direct dependencies declared by XAIMetrics and their minimum required versions.

Dependency	Minimum version
aix360	`>=0.3.0`
dalex	`>=1.7.2`
lime	`>=0.2.0.1`
numpy	`>=2.4.6`
pandas	`>=3.0.3`
pyod	`>=3.6.0`
PyYAML	`>=6.0.3`
quantus	`>=0.6.0`
shap	`>=0.52.0`
torch	`>=2.12.0`
pytest	`>=9.1.1`
Sphinx	`>=9.1.0`
sphinx-rtd-theme	`>=3.1.0`

Table 2.1: Minimum versions of the direct dependencies declared by XAIMetrics

The versions shown in Table 2.1 are lower bounds rather than exact versions. When the package is installed, `pip` automatically resolves these requirements and installs any additional transitive dependencies. In particular, packages such as `scikit-learn`, `joblib`, `matplotlib`, and `cloudpickle` may be installed as dependencies of the libraries listed above.

Some of the datasets, attribution files, and serialized models included in the repository are managed using Git Large File Storage (Git LFS). Therefore, both Git and Git LFS must be installed before cloning the repository. Git LFS only needs to be initialized once for each user account:

```
$ git lfs install
```

The library is distributed via GitHub, available at https://github.com/kdis-lab/XAIMetrics. The repository contains the following main folders:

- `docs`. Contains the documentation resources, including the user manual, the Application Programming Interface (API) documentation in PDF format, and the Sphinx project used to generate the HTML API reference.
- `examples`. Contains a basic usage notebook, an example YAML configuration file, sample datasets, pretrained models, attribution files, specific usage examples, and generated reports in CSV and JSON formats.
- `experiments`. Contains the datasets, trained models, attribution files, generated reports, and the Python script used to conduct the preliminary experimental study provided with the library.
- `tests`. Contains the automated tests used to verify the correct behaviour of the library and its components.
- `xai_metrics`. Contains the Python source code of the library, including the metric implementations, explainers, configuration management, experiment runner, and reporting functionality.

The library can be installed directly from the repository in editable mode:

```
$ git clone https://github.com/kdis-lab/XAIMetrics.git
$ cd XAIMetrics
$ pip install -e .
```

CHAPTER

3

XAIMetrics library

`XAIMetrics` is a Python library that provides a unified framework for generating and evaluating explanations produced by XAI methods. The library enables different explanation techniques to be applied and assessed across multiple models and datasets under consistent experimental conditions, facilitating systematic and reproducible comparisons between XAI methods.

`XAIMetrics` has been designed as a modular, extensible, and reproducible environment that integrates explanation generation, metric evaluation, experiment configuration, automated execution, and report generation. Experiments can be defined through declarative `YAML` configuration files, allowing users to specify datasets, models, explanation methods, evaluation metrics, and their corresponding parameters without modifying the source code of the library.

The main features of the library are as follows:

- Provides implementations of 17 metrics for evaluating different properties of feature-attribution explanations.
- Includes implementations of four local explanation methods: LIME, SHAP, BreakDown, and Model Agnostic Supervised Local Explanations (MAPLE).
- Supports both the generation of feature attributions and the evaluation of previously generated attribution files.
- Automatically discovers and evaluates the available combinations of datasets, predictive models, and XAI methods.
- Allows multiple explanation methods and evaluation metrics to be executed from the same experimental configuration.
- Uses declarative `YAML` files to define and reuse complete experiments, including the input paths, execution device, explanation methods, metrics, and their parameters.
- Supports both automatically constructed evaluation contexts and contexts provided directly at runtime.
- Uses automatic registration and discovery mechanisms for metrics and explanation methods, facilitating the integration of new components.

- Allows runtime dependencies to be supplied externally, including custom model-loading functions and explanation functions required by particular metrics.
- Records metrics or explanation methods that cannot be applied to a particular context without interrupting the remaining experiment.
- Generates an aggregated comparative report containing the dataset, model, metric, metric parameters, and results obtained for each XAI method.
- Generates observation-level reports for metrics whose outputs can be associated with the individual observations evaluated.
- Exports the aggregated results in `CSV` and `JSON` formats and the observation-level results in `CSV` format, facilitating their filtering, analysis, and reuse in other environments.
- Saves generated attribution matrices as `CSV` files that can subsequently be used by the evaluation workflow.

This chapter describes the main concepts and components of the `XAIMetrics` library. First, the evaluation taxonomy adopted by the library is presented, together with the explainability properties and metrics currently supported. Then, the internal architecture of the library and the responsibilities of its main modules are described. Finally, the configuration mechanism and the forms of the input and output files used throughout the explanation and evaluation workflows are presented.

3.1 XAIMetrics evaluation taxonomy

The evaluation metrics implemented in `XAIMetrics` are organized according to the explainability property that they assess. This organization provides a common structure for metrics that may rely on different computational procedures but evaluate related characteristics of feature-attribution explanations.

The taxonomy adopted by the library distinguishes five main evaluation dimensions: *complexity*, *faithfulness*, *fidelity*, *robustness* and *sensitivity*. Fidelity is additionally divided into *completeness* and *soundness*, since these two dimensions address complementary aspects of the relationship between an explanation and the behaviour of the predictive model.

These properties address different aspects of explanation quality and should not be interpreted independently as indicators of the overall quality of an explanation method. An explanation may, for example, be concise but fail to represent the behaviour of the predictive model accurately, or it may be faithful to the model while being unstable under small input perturbations. Consequently, the combined analysis of different properties provides a more complete characterization of an explanation method.

3.1.1 Complexity

Complexity evaluates the extent to which an explanation is concise and concentrates its relevance on a limited number of input features or explanatory elements [12]. For feature-attribution explanations, complexity is related to how the attribution values are distributed across the input dimensions. An explanation in which most of the attribution magnitude is concentrated on a small subset of features is generally considered less complex, whereas an explanation that distributes relevance across many features is considered more complex.

Complexity is particularly relevant from the perspective of interpretability, since explanations involving a limited number of relevant features may be easier for a user to inspect and understand, especially when the original input is high-dimensional. Different metrics can characterize this property by analysing, for example, the entropy, dispersion, or sparsity of the attribution distribution.

However, low complexity should not be interpreted as evidence that an explanation is necessarily more accurate or more faithful to the model. A highly sparse explanation may omit features that contribute meaningfully to the prediction, while a more distributed explanation may correctly represent a model whose decision depends on several interacting variables. Complexity should therefore be analysed together with properties that evaluate the relationship between the explanation and the predictive model.

3.1.2 **Faithfulness**

Faithfulness evaluates the correspondence between the importance assigned to the input features and their actual influence on the behaviour of the predictive model. For feature-attribution explanations, a faithful explanation should assign greater relevance to features whose modification has a greater effect on the model output.

Faithfulness metrics analyse this relationship using different strategies. Some evaluate the change in the model output when relevant features are removed, perturbed, or introduced progressively. Others analyse whether the ranking or magnitude of the attribution values is consistent with the changes observed in the prediction, whether feature contributions behave monotonically, or whether similar explanation patterns correspond to similar predictive behaviour.

These metrics are therefore useful for determining whether the generated attribution values reflect the decision-making behaviour of the model rather than merely providing a plausible interpretation. Nevertheless, their results may depend strongly on the perturbation strategy, the baseline used to replace features, the feature ordering, and the dependencies between input variables. Perturbing features independently can generate observations outside the data distribution or break relationships between correlated variables. These aspects must therefore be considered when interpreting faithfulness results.

3.1.3 **Fidelity**

Fidelity evaluates the extent to which the explanation accurately represents the information involved in the behaviour of the predictive model. While faithfulness focuses primarily on the correspondence between feature importance and changes in model behaviour, fidelity considers whether the explanation as a whole provides an adequate representation of the information used by the model.

Fidelity is particularly important because an explanation may appear understandable or plausible without actually representing the predictive process of the model. An explanation with high fidelity should capture the relevant information on which the prediction depends without introducing information what is not supported by the model.

In the taxonomy adopted by `XAIMetrics`, fidelity is divided into two complementary dimensions.

3.1.3.1 **Completeness**

Completeness evaluates whether an explanation contains sufficient information to account for the relevant behaviour of the predictive model. For feature-attribution explanations, this means determining whether the features and attribution values provided by the explanation capture the information required to characterize or reproduce the model prediction.

An incomplete explanation may correctly identify some influential features while omitting other variables that also contribute substantially to the prediction. Completeness metrics are therefore useful for identifying explanations that provide only a partial representation of the model decision.

Completeness does not, however, guarantee that all the information included in the explanation is correct. An explanation may contain sufficient relevant information to account for a prediction while simultaneously assigning importance to additional features that are not genuinely supported by the model.

3.1.3.2 **Soundness**

Soundness evaluates whether the information included in an explanation is actually supported by the behaviour of the predictive model. A sound explanation should not assign relevance to features, concepts, or relationships that do not influence the model prediction.

Soundness and completeness address different types of explanation error. Completeness is primarily concerned with relevant information that may have been omitted, whereas soundness is concerned with unsupported information that has been included. Consequently, an explanation may be sound but incomplete if every feature reported as relevant genuinely affects the prediction but other important features are missing. Conversely, an explanation may be complete but unsound if it contains enough information to account for the prediction together with additional features that do not actually influence the model.

Soundness can be evaluated by comparing attribution values with changes in model output produced by feature perturbations or through other strategies that independently assess whether the relevance assigned by the explanation is justified by the predictive model.

The results of fidelity metrics may depend on the baseline or perturbation strategy, the model-output representation, and the specific definition of sufficient or supported information used by each metric. Completeness and soundness should therefore be interpreted jointly whenever possible.

3.1.4 **Robustness**

Robustness evaluates the stability of explanations under small changes in the input or evaluation conditions when the underlying model behaviour remains approximately unchanged. For feature-attribution explanations, robustness analyses whether observations that are close to each other and receive similar predictions also produce similar attribution patterns.

A robust explanation method should not produce disproportionately different explanations as a consequence of minor or semantically irrelevant changes in the input. Robustness metrics can quantify this property by comparing the distance between explanations generated for neighbouring observations, by relating explanation variation to input variation, or by evaluating the maximum or average change observed within a local neighbourhood.

The interpretation of robustness depends on how the neighbourhood and perturbations are defined. Perturbations that are too large may alter the semantic meaning of an observation or change its model prediction, in which case a different explanation may be expected. Conversely, excessively small perturbation may fail to reveal meaningful instability. Feature scaling, dependencies between variables, distance functions, and the number of sampled neighbours can also affect the resulting robustness.

3.1.5 **Sensitivity**

Sensitivity evaluates how an explanation responds when elements on which it depends are modified. Depending on the metric, these modifications can affect the input observation, the target output being explained, or the parameters of the predictive model.

Sensitivity therefore considers whether changes in experimental conditions produce an appropriate change in the resulting explanation. When small and irrelevant input perturbations are considered, limited explanation variation is generally desirable. In contrast, when relevant elements such as the target class or learned model parameters are deliberately modified, an explanation should reflect these changes rather than remaining insensitive to them.

This property is closely related to robustness, since both involve comparing explanations generated under modified conditions. The distinction adopted by this library is based on the primary purpose of the evaluation. Robustness focuses on whether explanations remain stable under changes for which similar explanations are expected, whereas sensitivity evaluates how explanations react to modifications of the elements involved in the explanation -generation process.

Sensitivity results depend on the type and magnitude of the modification introduced during the evaluation. Input perturbations that are too large may change the semantic meaning of an observation, while very small perturbations may not provide enough information to identify unstable behaviour. Metrics based on target or model modifications must likewise be interpreted according to the particular intervention being performed and the quantity returned by the metric.

The boundaries between these evaluation dimensions are not necessarily strict. Some metrics can be related to more than one property because similar experimental procedures may be used to investigate different aspects of explanation quality. For example, both robustness and sensitivity can involve perturbing the input and comparing the resulting attribution values. The classification adopted in `XAIMetrics` therefore reflects the primary evaluation purpose assigned to each metric in the current implementation.

Table 3.1 summarizes the metrics currently implemented in the library and their corresponding evaluation category. The preferred result of each metric is also indicated. For numerical metrics, *higher* indicates that larger values are preferred, whereas *lower* indicates that smaller values are preferred. For Boolean metrics, the preferred Boolean result, `True` or `False`, is indicated directly.

The taxonomy is directly reflected in the internal structure of the library. Metric implementations are distributed among the *complexity*, *faithfulness*, *fidelity*, *robustness*, and *sensitivity* packages. The *fidelity* package is further divided into the *completeness* and *soundness* subpackages. This correspondence between the conceptual organization and the software architecture facilitates metric selection and the integration of new evaluation measures.

3.2 **XAIMetrics library architecture**

The library has been developed as an open-source Python project. It follows a modular architecture that separates the responsibilities associated with configuration management, explanation generation, metric evaluation, experiment execution, and result reporting. This organization allows new metrics, explanation methods, and supporting components to be incorporated without modifying the main execution workflow.

The library is organized into six main modules: `base`, `config`, `explainers`, `metrics`, `runner`, and `reporting`. The main functionality of each module is described below:

Category	Evaluation objective	Metric	Preferred result
Complexity	Conciseness and concentration of the information contained in the explanation.	`Complexity`	lower
		`Sparseness`	higher
Faithfulness	Correspondence between feature importance and the behaviour of the predictive model.	`Consistency`	lower
		`FaithfulnessEstimate`	higher
		`MonotonicityCorrelation`	higher
		`Monotonicity`	`True`
		`SensitivityN`	higher
		`Sufficiency`	higher
		`Faithfulness`	higher
		`MonotonicityMetric`	`True`
Fidelity – Completeness	Degree to which the explanation contains sufficient information to account for the model prediction.	`Completeness`	`True`
		`MuFidelity`	higher
		`Deletion`	lower
		`Insertion`	higher
		`AverageDrop`	lower
		`AverageIncrease`	higher
		`AverageGain`	higher
Fidelity – Soundness	Degree to which features identified as irrelevant by the explanation are also irrelevant to the model output.	`NonSensitivity`	lower
Robustness	Stability of the explanation under input perturbations or changes between models with equivalent predictive behaviour.	`LocalLipschitzEstimate`	lower
		`MaxSensitivity`	lower
		`RelativeInputStability`	lower
		`RelativeOutputStability`	lower
		`AverageStability`	lower
		`MeGe`	higher
Sensitivity	Response of the explanation to changes in the input, target class, or learned model parameters.	`AvgSensitivity`	lower
		`RandomLogit`	lower
		`ModelRandomization`	lower

Table 3.1: Evaluation taxonomy and metrics implemented in `XAIMetrics`.

- *base*. Contains the common abstractions, data structures, and registration mechanisms used throughout the library. It defines the `BaseMetric` and `BaseExplainer` classes, from which all metrics and explanation methods inherit, respectively. The module also defines two context classes. The `MetricContext` class groups the information required to evaluate a metric, including the predictive model, test data, labels, selected observations, attribution values, and execution device. The `ExplainerContext` class contains the information required to generate explanations, including the background data, the predictive model, the observations to explain, their associated targets, and the execution device. In addition, this module provides separate registries for metrics and explainers, together with the `register_metric` and `register_explainer` decorators. These mechanisms allow implementations to be discovered and instantiated automatically from their names. The `MetricSkipped` and `ExplainerSkipped` exceptions are also defined in this module and are used to indicate that a component cannot be applied to a particular experimental context without stopping the complete execution.

- *config*. Manages the loading and interpretation of the experimental configuration through the `ConfigController` class. A configuration can be supplied as a Python mapping or as the path to a `YAML` file. This module supports two ways of defining an experiment. A single context can be specified explicitly by providing direct paths to the model, data, labels, and attribution files. Alternatively, the library can discover multiple experimental contexts automatically from the configured dataset, model, and attribution directories. The module is also

responsible for loading the required data and models and for constructing the `MetricContext` and `ExplainerContext` objects used by the execution workflows. A custom model-loading function can be provided when the default loading mechanism is not suitable for a particular model format.

- *explainers*. Contains the implementations of the explanation methods integrated into the library. The current version provides LIME, SHAP, BreakDown, and MAPLE. Each explanation method is implemented as a class that inherits from `BaseExplainer` and implements the `explain()` method. This method receives a predictive model and a batch of observations and returns the corresponding feature-attribution matrix. The module also provides an automatic discovery mechanism that imports and registers the available explanation methods. Consequently, explainers can be selected and configured by name in the `YAML` file. New explanation techniques can be incorporated by implementing the common interface and registering the resulting class.

- *metrics*. Contains the implementations of the 17 evaluation metrics included in the library. Each metric is implemented as a class that inherits from `BaseMetric`, receives a `MetricContext`, and defines its computation through the `run()` method. Metric implementations are independent of the main execution workflow and are discovered automatically through the metric registry. This design allows users to select metrics by name in the experimental configuration and facilitates the addition of new evaluation measures. The metrics are organized into submodules according to the main explainability property they evaluate:

  - *metrics.complexity*. Contains metrics that evaluate the complexity and sparsity of an explanation.
  - *metrics.faithfulness*. Contains metrics that evaluate the correspondence between feature attributions and the behaviour of the predictive model.
  - *metrics.fidelity*. Contains metrics related to the fidelity of an explanation. This module is further divided into *completeness* and *soundness* submodules.
  - *metrics.fidelity.completeness*. Contains metrics that evaluate whether an explanation captures the information required to represent the model prediction sufficiently.
  - *metrics.fidelity.soundness*. Contains metrics that evaluate whether the information identified as relevant by an explanation is supported by the behaviour of the model.
  - *metrics.robustness*. Contains metrics that evaluate the stability of explanations under small changes in the input or model output.
  - *metrics.sensitivity*. Contains metrics that evaluate how strongly explanations change when the corresponding input observations are perturbed.

- *runner*. Coordinates the main execution workflows of the library. It provides the `run_explanation()` and `run_evaluation()` functions. The `run_explanation()` function manages the explanation-generation workflow. It discovers the registered explainers, loads the experimental configuration, constructs the required `ExplainerContext` objects, selects the configured explanation methods, and generates their attribution matrices. The generated attributions can be returned directly and optionally exported as `CSV` files. The `run_evaluation()` function manages the metric-evaluation workflow. It discovers the registered metrics, constructs or receives the required `MetricContext` objects, selects the configured metrics, injects the available runtime dependencies, and executes each metric for every valid combination of dataset, model, and XAI method. Both workflows record components that cannot be executed in the current context instead of interrupting the complete experiment. The evaluation workflow collects the results and forwards them to the reporting module to construct the aggregated and observation-level reports.

- *reporting*. Organizes, aggregates, and exports the outputs produced by the explanation and evaluation workflows. For the explanation workflow, this module converts the generated

attribution matrices into data frames and saves them as `CSV` files. The files are organized according to the corresponding dataset, model, and XAI method. For the evaluation workflow, the module generates two types of reports. First, it builds an aggregated comparative report in which each row identifies a dataset, model, metric, and metric-parameter configuration, while the results of the different XAI methods are represented in separate columns. When a metric returns multiple numeric values, these values are aggregated to produce the corresponding summary result. Second, the module builds observation-level reports for metric outputs that can be aligned with the evaluated observations. These reports preserve the individual metric value obtained for each observation and organize the results into columns corresponding to the evaluated XAI methods. The aggregated results from all datasets and models are combined into a single report and exported in both `CSV` and `JSON` formats. Observation-level reports are exported as separate `CSV` files for each metric, dataset, and model.

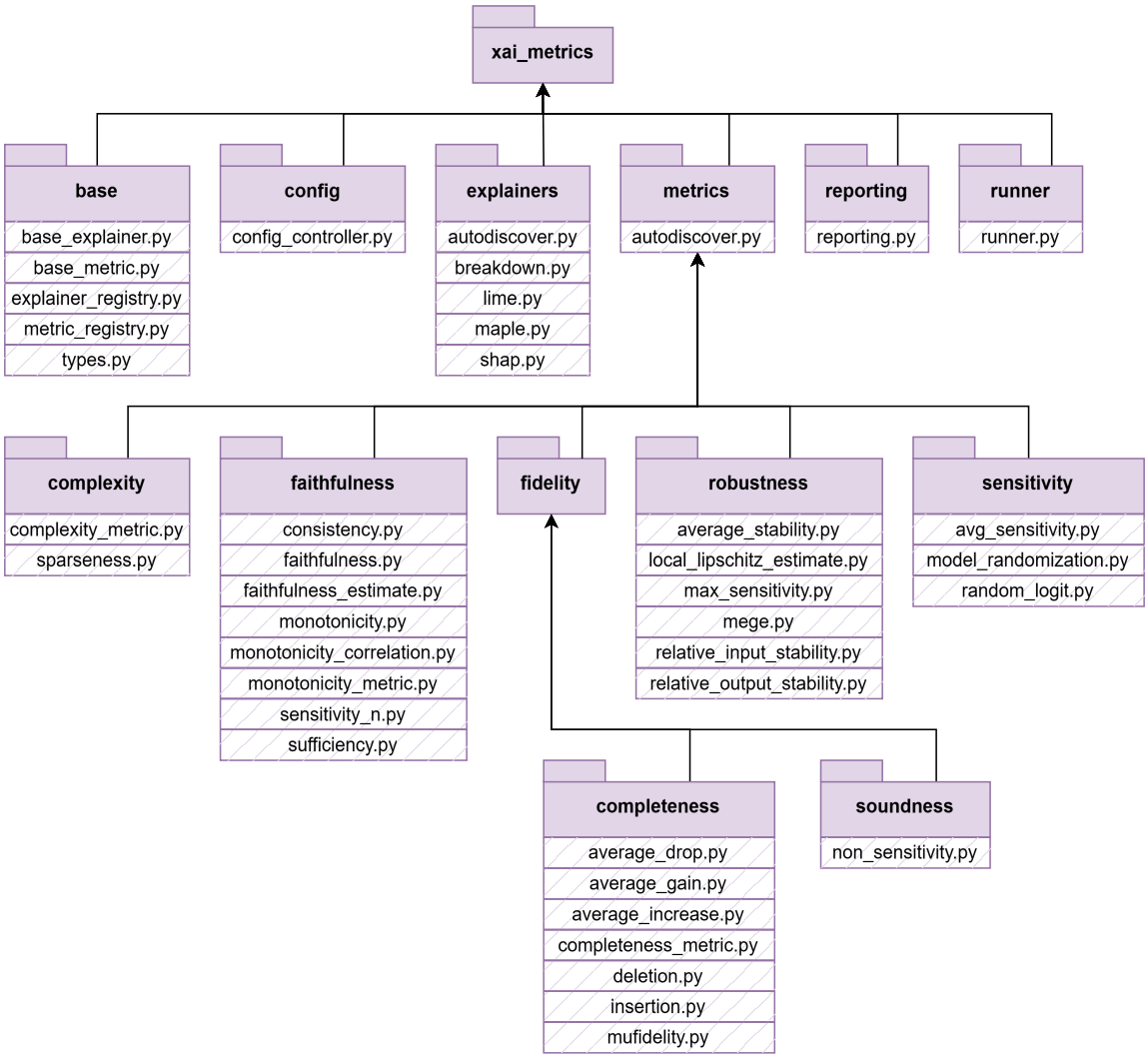


Figure 3.1: Architecture of `XAIMetrics` library

Figure 3.1 illustrates the overall architecture of the `XAIMetrics` library and the relationships between the components involved in experiment configuration, explanation generation, metric evaluation, and report generation. The configuration module constructs the contexts required by the execution workflows, while the runner coordinates the registered explainers and metrics. Finally, the reporting module organizes and exports the generated attributions and evaluation results.

3.3 Configuration and file formats

`XAIMetrics` uses a set of configuration, input, and output files to define and execute explanation-generation and evaluation experiments. The experimental configuration can be provided either as a Python mapping or through a declarative `YAML` file. The remaining information, including input data, target values, generated attributions, and evaluation reports, is represented using standard file formats to facilitate intolerability with external tools.

The configuration file is organized into three main components:

- `context:` defines the data, predictive models, attribution files, and execution device used in the experiment. It can contain dataset, model, and attribution directories for automatic context discovery, or explicit file paths for defining a single experimental context. The execution device can also be specified in this section.
- `metrics:` defines the evaluation metrics to execute and their corresponding parameters. Each metric is identified by its registered name and can include a metric-specific parameter mapping. The metrics currently available in the library are summarized in Table 3.1.
- `explainers:` defines the explanation methods used to generate feature attributions and their corresponding parameters. The explanation methods currently supported by the library are LIME, SHAP, BreakDown, and MAPLE.

3.3.1 Configuration file

A complete experiment can be defined through a `YAML` configuration file. A simplified configuration has the following structure:

```yaml
context:
  device: "cpu"
  datasets_dir: "experiments/data"
  models_dir: "experiments/models"
  attributions_dir: "experiments/attributions"

metrics:
  - name: Complexity
    params:
      normalise: true

  - name: Completeness
    params:
      abs: false
      normalise: true

explainers:
  - name: LIME
    params:
      mode: classification
      random_state: 42

  - name: SHAP
    params:
      mode: classification
      algorithm: auto
```

Each entry in the `metrics` and `explainers` sections contains a `name` field and an optional `params` mapping. The `name` identifies a component registered in the library, whereas `params` contains the parameters passed to that particular implementation. Consequently, different configurations of the same metric or explanation method can be defined without modifying its source code.

The `metrics` section can contain any of the evaluation metrics listed in Table 3.1. The `explainers` section currently supports the four explanation methods integrated into the library: LIME, SHAP, BreakDown and MAPLE.

The `context` section can be defined in two different ways. The first option is to provide root directories and allow the library to discover the available datasets, models, and attribution files automatically. The second option is to define a single experimental context by providing the corresponding file paths explicitly.

3.3.2 Automatic context discovery

For experiments involving several datasets, models, or explanation methods, the `context` section can specify directory roots:

```yaml
context:
  device: "cpu"
  datasets_dir: "experiments/data"
  models_dir: "experiments/models"
  attributions_dir: "experiments/attributions"
```

The `datasets_dir` field identifies the root directory containing the input datasets, `models_dir` identifies the root directory containing the trained predictive models, and `attributions_dir` identifies the root directory containing attribution files. The optional `device` field determines the device used during executions, for example Central Processing Unit (CPU) or Compute Unified Device Architecture (CUDA) device.

When automatic context discovery is used, datasets, models, and attribution files can be organized following the directory structure illustrated in Figure 3.4.

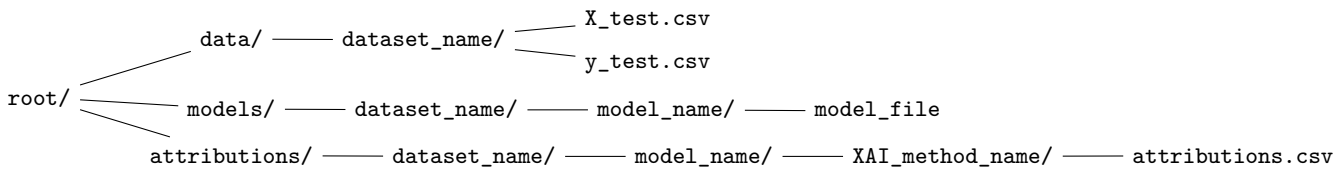


Figure 3.2: Directory structure used by `XAIMetrics`.

The directory names shown in the figure are illustrative. The actual root paths are specified independently through the `datasets_dir`, `models_dir`, and `attributions_dir` configuration fields.

For each dataset, the library searches the corresponding model and attribution directories and automatically constructs the valid combinations of dataset, predictive model, and XAI method.

When discovering evaluation data, files whose names begin with `X_batch` and `y_batch` are considered first. If they are not available, files beginning with `X_test` and `y_test` are used instead.

The explanation-generation workflow additionally requires background data. In this case, the library first searches for files whose names begin with `X_background` and if none are available, files beginning with `X_train` are used. The observations to explain are obtained from `X_batch` file when available and from `X_test` files otherwise. Corresponding target files can be provided using the prefixes `y_background`, `y_train`, `y_batch`, and `y_test`.

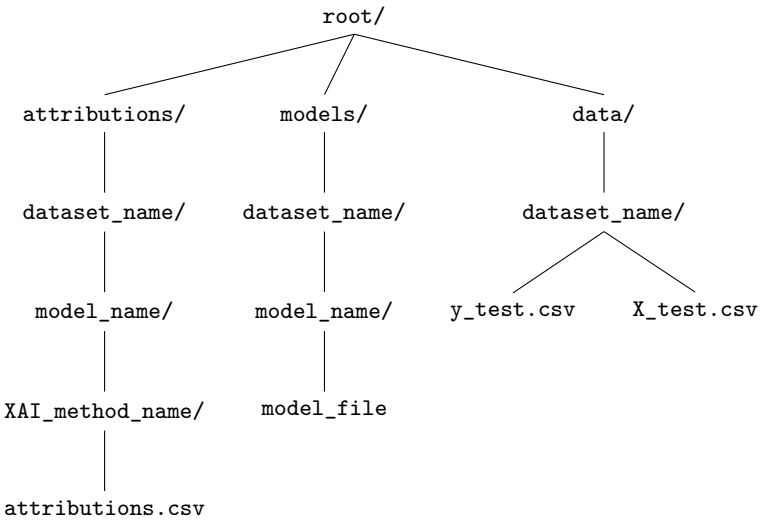


Figure 3.3: Directory structure used by `XAIMetrics`.

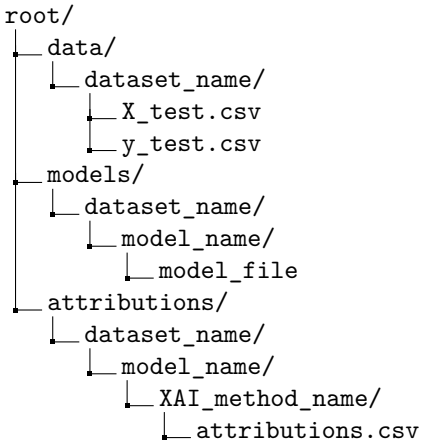


Figure 3.4: Directory structure used by `XAIMetrics`.

3.3.3 Explicit context definition

For experiments involving a single combination of dataset, model, and explanation method, the required resources can alternatively be specified directly in the configuration file.

For metric evaluation, the context can be defined as follows:

```yaml
context:
  dataset_name: "dataset"
  model_name: "model"
  xai_method_name: "SHAP"
  device: "cpu"

  model_path: "path/to/model.pt"
  X_test_path: "path/to/X_test.csv"
  y_test_path: "path/to/y_test.csv"
  attributions_path: "path/to/attributions.csv"
```

The `dataset_name`, `model_name`, and `xai_method_name` fields provide the metadata used to identify the experiment in the generated reports. The remaining fields identify the model, input data, target values, and feature-attribution file used to construct the corresponding `MetricContext`.

For explanation generation, the context requires a model, background data, and the observations to explain. Background and batch target files can also be provided when required by the selected explanation method. The main fields used for this workflow are `model_path`, `X_background_path`, `y_background_path`, `X_batch_path`, and `y_batch_path`.

3.3.4 Input data files

Input observations and target values are represented as `CSV` files. In all cases, the first column of the file is interpreted as the observation index.

An input file contains one observation per row and one input feature per column:

```
index,feature_1,feature_2,...,feature_d
0,value,value,...,value
1,value,value,...,value
...
```

Target values are stored in a separate `CSV` file:

```
index,target
0,value
1,value
...
```

The indexes of the input and target files must contain the same observation identifiers. The library aligns the target values according to the order of the input-data index before constructing the execution context.

The current implementation expects a single target column. If a target file contains more than one column, the first one is selected and the remaining columns are ignored. Numeric class labels represented using integer-valued floating-point values, such as `0.0` and `1.0`, are converted to integer labels automatically.

3.3.5 Model files

Predictive models are loaded using the model loader configured for the experiment. The default loader uses Python `pickle` for files with the `.pkl` and `.pickle` extensions, `joblib` for files with the `.joblib` and `.jl` extensions, and PyTorch loading facilities for other model files.

During automatic discovery, the library searches for model files with the `.pkl`, `.pickle`, `.joblib`, `.pt` and `.pth` extensions.

A custom model-loading function can be supplied to the execution workflow when the default loading mechanism is not appropriate for the model format used in an experiment.

3.3.6 **Attribution files**

Feature attributions used during metric evaluation are also represented as `CSV` files. Each row corresponds to an explained observation and each column contains the attribution assigned to one input feature:

```
index,feature_1,feature_2,...,feature_d
0,attribution,attribution,...,attribution
1,attribution,attribution,...,attribution
...
```

The first column is interpreted as the observation index. Every observation contained in an attribution file must correspond to an observation available in the associated input data. However, the attribution file does not need to contain every observation from the complete input dataset. Its index determines the subset of observations that will be evaluated.

When explanations are generated directly by `XAIMetrics`, the library automatically constructs the attribution `CSV` using the indexes of the explained observations and the feature names of the corresponding input data. Generated attribution files are stored using the following structure:

```
attributions/
  dataset_name/
    model_name/
      XAI_method_name/
        dataset_name_model_name_xai_method_attributions.csv
```

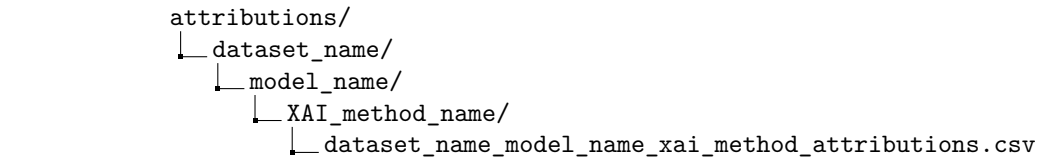


Figure 3.5: Directory structure used by `XAIMetrics` to generate explanations.

This format allows the output of the explanation-generation workflow to be subsequently reused as input to the metric-evaluation workflow.

3.3.7 **Evaluation report files**

The evaluation workflow generates two types of result files:

- Aggregated comparative reports
- Observation-level reports

The aggregated report contains the fields `dataset_name`, `model_name`, `metric` and `metric_params`, followed by one result column for each evaluated XAI method. Therefore, the values obtained by different explanation methods for the same dataset, model, metric and parameter configuration can be compared directly.

The complete aggregated report is exported in both `CSV` and `JSON` formats. Generated filenames include a timestamp identifying the corresponding execution:

```
xai_metrics_report_YYYYMMDD_HHMMSS.csv
xai_metrics_report_YYYYMMDD_HHMMSS.json
```

The `JSON` file stores the serialized report records under the `report` field.

For metrics whose outputs can be associated with individual observations, the library also generates observation-level `CSV` reports. These files contain the dataset name, model name, observation identifier, metric parameters and one result column for each evaluated XAI method. Observation-level reports are organized in metric-specific directories.

Together, these formats establish a consistent workflow in which input data and trained models are used to generate feature attributions, the attribution files are consequently evaluated through the configured metrics, and the resulting values are exported in structured formats for later analysis.

CHAPTER

4

Running Available Metrics in the Library

In this chapter, the individual metrics implemented in the **XAIMetrics** library are presented. For each metric, a brief description of its purpose and the explainability property it evaluates is provided, followed by two illustrative examples. The first example shows how to execute the metric individually using its corresponding Python class, whereas the second demonstrates how to include it in an automated evaluation experiment configured through a `YAML` file and executed by the `runner` module

The metrics described in this chapter follow the taxonomy introduced in Section 3.1. These include Faithfulness in Section 4.1, Sensitivity in Section 4.3, Robustness in Section 4.4, Complexity in Section 4.2, and Fidelity 4.5. The latter is further divided into Completeness in Section 4.5.1 and Soundness in Section 4.5.2.

4.1 Faithfulness

The faithfulness property and its interpretation are described in Section 3.1. This section presents the faithfulness metrics implemented in `XAIMetrics`, including their computation, configurable parameters, implementation details, and execution examples.

4.1.1 Consistency

The Consistency metric evaluates whether observations with similar explanations are assigned to the same predicted class [13]. To perform this comparison, the continuous attribution vectors are first transformed into discrete representations. For each observation, the metric computes the proportion of the remaining observations in its group that receive the same predicted class.

A high consistency score indicates that observations with equivalent attribution patterns frequently receive the same model prediction. Conversely, a low score indicates that similar explanations may be associated with different predicted classes. Therefore, the metric assesses whether the explanations consistently represent the decision-making behaviour of the model.

The parameters used by this metric can be summarized as follows:

- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before discretising and comparing the explanations. (*Default:* `True`)

The implementation uses the default discretisation and normalisation functions provided by `Quantus`. In particular, the default discretisation function is `top_n_sign`, which converts each continuous attribution vector into a discrete representation according to the most relevant features and the signs of their attributions.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The metric returns a Consistency score for each evaluated observation. When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Consistency` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.1.1), or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.1.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration also specifies that the original signs of the attribution values must be preserved by setting `abs` to `false`, while attribution normalisation remains enabled.

Examples of these usage modes are available in the library under: `examples/specific_examples/metrics/Consistency_examples.py`.

4.1.1.1 Direct evaluation using the Consistency class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Consistency` class is instantiated directly using the desired parameters. Calling the `run()` method returns one Consistency score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Consistency

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Consistency(
    context=context,
    params={
        "abs": False,
        "normalise": True
    }
)
scores = metric.run()
```

4.1.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Consistency"` metric, although this argument would not be necessary if `Consistency` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Consistency"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Consistency
    params:
      abs: false
      normalise: true
```

4.1.2 Faithfulness Estimate

The Faithfulness Estimate metric evaluates whether the importance assigned to the input features by an explanation is consistent with their actual influence on the model output [11]. To perform this evaluation, features are perturbed in groups according to their attribution importance. For each perturbation step, the metric compares the sum of the attribution values of the perturbed features with the corresponding decrease in the target model output.

The agreement between these two sequences is measured using the Pearson correlation coefficient. A high positive score indicates that features with larger attribution values also produce larger changes in the model output when perturbed. Conversely, a low or negative score indicates weak or inverse agreement between the feature importance assigned by the explanation and the behaviour of the model. Therefore, the metric assesses whether the attribution values accurately reflect the influence of the features on the prediction.

The parameters used by this metric can be summarized as follows:

- `features_in_step (int)`: number of features perturbed together at each evaluation step. (*Default:* `1`)

- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the attribution values before computing the metric. (*Default:* `True`)
- `perturb_baseline (str)`: baseline used to replace the selected features during perturbation. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default normalisation and perturbation functions provided by `Quantus`. The similarity between the attribution sums and the corresponding decreases in the model output is calculated using an internal safe implementation of the Pearson correlation coefficient. When either of the compared vectors has zero variance, this function returns 0.0 instead of an undefined Not a Number (`NaN`) value.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The metric returns one Faithfulness Estimate score for each evaluated observation. Higher values indicate stronger agreement between the importance assigned to the features and the changes produced in the model output after perturbation. When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used, and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `FaithfulnessEstimate` class in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.2.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration perturbs one feature at each step, preserves the original signs of the attribution values, enables attribution normalisation, and uses the mean value as the perturbation baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/FaithfulnessEstimate_examples.py`

4.1.2.1 Direct evaluation using the FaithfulnessEstimate class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `FaithfulnessEstimate` class is instantiated directly using the desired parameters. Calling the `run()` method returns one faithfulness estimate score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import FaithfulnessEstimate

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = FaithfulnessEstimate(
    context=context,
```

```
    params={
        "features_in_step": 1,
        "abs": False,
        "normalise": True,
        "perturb_baseline": "mean"
    }
)
scores = metric.run()
```

4.1.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"FaithfulnessEstimate"` metric, although this argument would not be necessary if `FaithfulnessEstimate` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["FaithfulnessEstimate"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: FaithfulnessEstimate
    params:
      features_in_step: 1
      abs: false
      normalise: true
      perturb_baseline: mean
```

4.1.3 Faithfulness

The Faithfulness metric evaluates whether the attribution values assigned to the input features reflect their actual influence on the model prediction [11]. For each evaluated observation, the metric first determines the class predicted by the model. It then replaces each feature individually

with its corresponding baseline value and records the probability assigned to the originally predicted class after each replacement.

The score is computed as the negative Pearson correlation between the attribution values and the predicted-class probabilities obtained after replacing the individual features. Features with larger attribution values are expected to produce larger decreases in the predicted-class probability when replaced by their baseline values. Therefore, a high positive score indicates strong agreement between the importance assigned by the explanation and the influence of the features on the model prediction. Conversely, a low or negative score indicates weak or inverse agreement.

The parameters used by this metric can be summarized as follows:

- `base_values (array-like, optional)`: explicit baseline values used to replace the input features. The provided array must contain one baseline value for each feature. When specified, these values take priority over `base_strategy`. (*Default:* `None`)
- `base_strategy (str)`: strategy used to calculate a common baseline vector from the complete test dataset when `base_values` is not provided. The supported strategies are:
  - `"mean"`: uses the feature-wise mean of the test dataset;
  - `"median"`: uses the feature-wise median of the test dataset;
  - `"zero"`: uses a vector containing one zero for each feature.

  (*Default:* `"mean"`)

Before computing the metric, a common baseline vector is obtained either from the values explicitly provided through `base_values` or from the complete test dataset using the selected `base_strategy`. The observations specified in the `MetricContext` are then selected and evaluated independently together with their corresponding attribution vectors.

The metric returns one Faithfulness score for each evaluated observation. Higher positive values indicate stronger agreement between the attribution values and the decrease in predicted-class probability caused by replacing individual features. If the attribution vector or the probabilities obtained from the perturbed inputs have zero variance, the Pearson correlation is undefined, and the underlying `AIX360` implementation may return `NaN`.

If an unsupported value is provided through `base_strategy`, the metric raises a `ValueError`. Moreover, the implementation relies on the `faithfulness_metric` function provided by `AIX360`, which requires a classification model exposing a `predict_proba` method. If the evaluated model does not implement this method, the metric raises an `AttributeError`.

The metric is implemented through the `Faithfulness` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.3.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.3.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The baseline vector is calculated using the feature-wise mean of the complete test dataset. This experiment assumes that the loaded model provides the `predict_proba` method required by the metric.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Faithfulness_examples.py`

4.1.3.1 **Direct evaluation using the Faithfulness class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Faithfulness` class is instantiated directly using the desired parameters. Calling the `run()` method returns one faithfulness score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Faithfulness

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Faithfulness(
    context=context,
    params={
        "base_strategy": "mean"
    }
)
scores = metric.run()
```

4.1.3.2 **Automated evaluation using run_evaluation()**

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Faithfulness"` metric, although this argument would not be necessary if `Faithfulness` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Faithfulness"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Faithfulness
```

```
  params:
    base_strategy: mean
```

4.1.4 Monotonicity Correlation

The Monotonicity Correlation metric evaluates whether feature attribution values are monotonically related to the uncertainty produced in the model output when the corresponding features are perturbed [14]. For each evaluated observation, the features are ordered according to their attribution values and divided into groups. Each group is perturbed repeatedly, and the uncertainty associated with that group is estimated from the changes produced in the target model output.

The metric compares the sum of the attribution values in each feature group with the corresponding uncertainty estimate using the Spearman rank correlation coefficient. A high positive score indicates that groups containing features with greater attribution values tend to produce greater uncertainty in the model output when perturbed. Conversely, a low or negative score indicates a weak or inverse monotonic relationship between the importance assigned by the explanation and the effect of perturbing the corresponding features. Therefore, the metric assesses whether the ordering induced by the attribution values agrees with the sensitivity of the model output to feature perturbations.

The parameters used by this metric can be summarized as follows:

- `eps (float)`: threshold used when calculating the inverse prediction factor employed to scale the changes in the model output. This parameter avoids numerical instability when the magnitude of the original output is close to zero. (*Default:* `1e-5`)
- `nr_samples (int)`: number of perturbed samples generated for each feature group to estimate the uncertainty associated with its perturbation. (*Default:* `100`)
- `features_in_step (int)`: number of features included in each group and perturbed together at every evaluation step. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before ordering the features and computing the metric. (*Default:* `True`)
- `perturb_baseline (str)`: baseline used when perturbing the selected features. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"uniform"`)

The implementation uses the default normalisation and perturbation functions provided by `Quantus`. The relationship between the attribution sums and the uncertainty estimates is calculated using an internal safe implementation of the Spearman correlation coefficient. When either of the compared vectors has zero variance, this function returns 0.0 instead of an undefined `NaN` value.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their target labels and attribution values. For each observation, the feature groups are perturbed `nr_samples` times. The uncertainty associated with each group is estimated from the mean squared change in the target model output, scaled relative to the magnitude of the original output. The metric then returns one Monotonicity Correlation score for each evaluated observation.

Higher values indicate a stronger positive monotonic relationship between feature importance and the uncertainty caused by perturbing the corresponding feature groups. When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `MonotonicityCorrelation` class in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.4.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.4.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration perturbs one feature at each step, generates 100 perturbation samples for each feature group, uses the absolute and normalised attribution values, and replaces the perturbed features with their mean values.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MonotonicityCorrelation_examples.py`

4.1.4.1 **Direct evaluation using the MonotonicityCorrelation class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `MonotonicityCorrelation` class is instantiated directly using the desired parameters. Calling the `run()` method returns one monotonicity correlation score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import MonotonicityCorrelation

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MonotonicityCorrelation(
    context=context,
    params={
        "eps": 1e-5,
        "nr_samples": 100,
        "features_in_step": 1,
        "abs": True,
        "normalise": True,
        "perturb_baseline": "mean"
    }
)
scores = metric.run()
```

4.1.4.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MonotonicityCorrelation"` metric, although this argument would not be necessary if `MonotonicityCorrelation` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MonotonicityCorrelation"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: MonotonicityCorrelation
    params:
      eps: 1e-5
      nr_samples: 100
      features_in_step: 1
      abs: true
      normalise: true
      perturb_baseline: mean
```

4.1.5 Monotonicity Metric

The Monotonicity Metric evaluates whether the probability assigned to the originally predicted class increases monotonically as the input features are progressively restored from a baseline vector [15]. For each evaluated observation, the metric first obtains the class predicted by the model using the complete original input. It then creates an initial input containing the baseline value of each feature and progressively restores the original feature values in increasing order of their attribution values.

After restoring each feature, the metric records the probability assigned to the originally predicted class. The resulting sequence is expected to be monotonically non-decreasing, meaning that restoring progressively more relevant information should not reduce the probability of the original prediction. The metric returns `True` when this condition is satisfied and `False` otherwise.

The parameters used by this metric can be summarized as follows:

- `base_values (array-like, optional)`: explicit baseline values used as the initial feature values. The provided array must contain one baseline value for each input feature. When specified, these values take priority over `base_strategy`. (*Default:* `None`)

- `base_strategy (str)`: strategy used to calculate a common baseline vector from the complete test dataset when `base_values` is not provided. The supported strategies are:

  - `"mean"`: uses the feature-wise mean of the test dataset;
  - `"median"`: uses the feature-wise median of the test dataset;
  - `"zero"`: uses a vector containing one zero for each feature.

  (*Default:* `"mean"`)

Before computing the metric, a common baseline vector is obtained either from the values explicitly provided through `base_values` or from the complete test dataset using the selected `base_strategy`. The observations specified in the `MetricContext` are then selected and evaluated independently together with their corresponding attribution vectors.

For each observation, the features are ordered by increasing attribution value. Starting from the baseline vector, their original values are restored cumulatively following this order. After each restoration, the probability assigned to the class predicted for the original complete input is obtained.

The metric returns one Boolean result for each evaluated observation. A value of `True` indicates that the predicted-class probability never decreases as the features are progressively restored, whereas `False` indicates that the monotonicity condition is violated. When the Boolean results are averaged, `True` and `False` are interpreted as (1) and (0), respectively. Therefore, the resulting mean represents the proportion of evaluated observations that satisfy the monotonicity criterion.

If an unsupported value is provided through `base_strategy`, the metric raises a `ValueError`. Moreover, the implementation relies on the `monotonicity_metric` function provided by `AIX360`, which requires a classification model exposing a `predict_proba` method. If the evaluated model does not implement this method, the metric raises an `AttributeError`.

The metric is implemented through the `MonotonicityMetric` class in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.5.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.5.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The initial baseline vector is calculated using the feature-wise mean of the complete test dataset. The experiment assumes that the loaded model provides the `predict_proba` method required by the metric.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MonotonicityMetric_examples.py`

4.1.5.1 Direct evaluation using the MonotonicityMetric class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `MonotonicityMetric` class is instantiated directly using the desired parameters. Calling the `run()` method returns one monotonicity metric score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import MonotonicityMetric

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MonotonicityMetric(
    context=context,
    params={
        "base_strategy": "mean"
    }
)
scores = metric.run()
```

4.1.5.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MonotonicityMetric"` metric, although this argument would not be necessary if `MonotonicityMetric` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MonotonicityMetric"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: MonotonicityMetric
```

```
  params:
    base_strategy: mean
```

4.1.6 Monotonicity

The Monotonicity metric evaluates whether the target model output evolves monotonically as feature groups are progressively introduced according to their attribution values [15]. For each evaluated observation, the features are ordered by increasing attribution value. The evaluation starts from an input defined by a baseline, and the original feature values are progressively introduced in groups following this ordering.

After introducing each feature group, the metric evaluates the model output associated with the target label. The resulting sequence is expected to be monotonically non-decreasing, meaning that adding progressively more relevant information should not reduce the target model output. The metric returns `True` when this condition is satisfied throughout the complete sequence and `False` when the output decreases in at least one step.

The parameters used by this metric can be summarized as follows:

- `features_in_step (int)`: number of features introduced together at each evaluation step. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute values of the attributions before ordering the features and computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before ordering the features and computing the metric. (*Default:* `True`)
- `perturb_baseline (str)`: baseline used to initialise the input before the original feature values are progressively introduced. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default normalisation and perturbation functions provided by `Quantus`. For each observation, the input is initially replaced by the selected baseline. The original feature values are then introduced in groups of size `features_in_step`, following the increasing order of their attribution values.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their target labels and attribution values. After each feature group is introduced, the target model output is evaluated and compared with the output obtained in the previous step.

The metric returns one Boolean value for each evaluated observation. A value of `True` indicates that the target model output never decreases as the feature groups are progressively introduced, whereas `False` indicates that the monotonicity condition is violated at least once. When the Boolean results are averaged, `True` and `False` are interpreted as (1) and (0), respectively. Therefore, the resulting mean represents the proportion of observations that satisfy the monotonicity criterion.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Monotonicity` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.6.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.6.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration introduces one feature at each step, uses absolute and normalised attribution values, and initialises the perturbed input using the `"black"` baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Monotonicity_examples.py`

4.1.6.1 Direct evaluation using the Monotonicity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Monotonicity` class is instantiated directly using the desired parameters. Calling the `run()` method returns one monotonicity metric score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Monotonicity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Monotonicity(
    context=context,
    params={
        "features_in_step": 1,
        "abs": True,
        "normalise": True,
        "perturb_baseline": "black"
    }
)
scores = metric.run()
```

4.1.6.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Monotonicity"` metric, although this argument would not be necessary if `Monotonicity` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Monotonicity"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Monotonicity
    params:
      features_in_step: 1
      abs: true
      normalise: true
      perturb_baseline: black
```

4.1.7 **Sensitivity-N**

The Sensitivity-N metric evaluates whether the attribution values assigned to groups of input features agree with the changes produced in the target model output when those features are perturbed [15]. For each evaluated observation, the features are ordered by decreasing attribution value and progressively perturbed in groups. At each step, the metric records both the attribution values associated with the processed feature group and the corresponding variation in the target model output.

The agreement between these quantities is measured using the Pearson correlation coefficient. A high positive correlation indicates that feature groups with greater attribution values tend to produce larger changes in the target output when perturbed. Conversely, a low or negative correlation indicates weak or inverse agreement between the importance assigned by the explanation and the actual influence of the corresponding features on the model prediction. Therefore, the metric assesses whether the attribution values accurately represent the sensitivity of the model output to feature perturbations.

The parameters used by this metric can be summarized as follows:

- `n_max_percentage (float)`: maximum proportion of input features considered during the evaluation. For example, a value of `0.8` limits the experiment to perturbation steps covering up to 80% of the available features. (*Default:* `0.8`)
- `features_in_step (int)`: number of features perturbed together at each evaluation step. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute values of the attributions before ordering the features and computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the attribution values before computing the metric. (*Default:* `True`)

- `perturb_baseline (str)`: baseline used to replace the selected features during perturbation. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default Pearson correlation, normalisation function, and baseline-replacement function provided by `Quantus`. For each observation, the features are ordered by decreasing attribution value and processed in groups of size `features_in_step`. Only the perturbation steps covered by `n_max_percentage` are included in the evaluation.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their target labels and attribution values. After each feature group is perturbed, the variation in the target model output is compared with the attribution values associated with that group.

The metric returns the Sensitivity-N result produced by `Quantus`. Higher values indicate stronger agreement between the attribution values and the changes caused in the target model output by perturbing the corresponding features. Since result aggregation is enabled by default in the underlying `Quantus` implementation, the returned values may represent an aggregation of the correlations calculated across the evaluated perturbation steps rather than one independent score for every input observation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `SensitivityN` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.7.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.7.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. In this example, all input features are considered, one feature is perturbed at each step, the original signs of the attribution values are preserved, normalisation is enabled, and the perturbed features are replaced using the `"uniform"` baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/SensitivityN_examples.py`

4.1.7.1 Direct evaluation using the SensitivityN class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `SensitivityN` class is instantiated directly using the desired parameters. Calling the `run()` method returns one sensitivity-n score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import SensitivityN

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = SensitivityN(
    context=context,
    params={
        "n_max_percentage": 1.0,
        "features_in_step": 1,
        "abs": False,
        "normalise": True,
        "perturb_baseline": "uniform"
    }
)
scores = metric.run()
```

4.1.7.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"SensitivityN"` metric, although this argument would not be necessary if `SensitivityN` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["SensitivityN"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: SensitivityN
    params:
      n_max_percentage: 1.0
      features_in_step: 1
      abs: false
      normalise: true
      perturb_baseline: uniform
```

4.1.8 Sufficiency

The Sufficiency metric evaluates whether observations with similar explanations receive the same model prediction [13]. To perform this evaluation, the attribution vectors of all observations included in the evaluation batch are flattened and compared using a pairwise distance function. The resulting distance matrix is normalised, and two explanations are considered similar when the distance between them is less than or equal to a predefined threshold.

For each evaluated observation, the metric identifies the remaining observations whose explanations satisfy this similarity condition and calculates the proportion that receive the same predicted class. The observation itself is excluded from the comparison. A high score indicates that observations with similar explanations frequently exhibit the same model behaviour. Conversely, a low score indicates that similar attribution patterns may be associated with different predicted classes. If no other explanation lies within the specified distance threshold, the metric returns a score of `0.0` for that observation.

The parameters used by this metric can be summarized as follows:

- `threshold (float)`: maximum normalised distance between two attribution vectors for their explanations to be considered similar. Lower values impose a stricter similarity criterion, whereas higher values allow more distant explanations to be included in the comparison. (*Default:* `0.6`)
- `distance_func (str)`: distance function used to compare the attribution vectors. The value is passed to `Quantus` and is generally a valid distance name supported by SciPy, such as `"seuclidean"`. (*Default:* `"seuclidean"`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the distances between explanations. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before comparing the explanations. (*Default:* `True`)

The implementation uses the default attribution normalisation function provided by `Quantus`. For the complete set of evaluated observations, the attribution vectors are flattened and their pairwise distances are calculated using `distance_func`. After normalising the distance matrix, the `threshold` parameter is used to determine which explanations are sufficiently similar.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The model predictions are then compared among observations whose explanations satisfy the similarity criterion.

The metric returns one Sufficiency score for each evaluated observation. Scores range from `0.0` to `1.0`. Higher values indicate that observations with similar explanations more frequently receive the same predicted class. Since each explanation is compared only with the other observations included in the same evaluation batch, the resulting scores depend on the observations selected in the `MetricContext`. Changing the evaluated batch may therefore change the neighbourhood of each explanation and its resulting score.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Sufficiency` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.8.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.8.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. In this example, all input features are considered, one feature is perturbed at each step, the original signs of the attribution values are preserved, normalisation is enabled, and the perturbed features are replaced using the `"uniform"` baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Sufficiency_examples.py`

4.1.8.1 Direct evaluation using the Sufficiency class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Sufficiency` class is instantiated directly using the desired parameters. Calling the `run()` method returns one sufficiency score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Sufficiency

config_path = "examples/specific_examples/config.yaml"
metric = Sufficiency(
    context=context,
    params={
        "threshold": 0.6,
        "distance_func": "seuclidean",
        "abs": True,
        "normalise": True
    }
)
```

4.1.8.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Sufficiency"` metric, although this argument would not be necessary if `Sufficiency` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Sufficiency"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Sufficiency
    params:
      threshold: 0.6
      distance_func: seuclidean
      abs: true
      normalise: true
```

4.2 Complexity

The complexity property and its interpretation are described in Section 3.1. This section presents the complexity metrics implemented in `XAIMetrics`, together with their computation, configurable parameters, implementation details, and execution examples.

4.2.1 Complexity

The Complexity metric evaluates how widely the attribution importance is distributed across the input features [10]. For each evaluated observation, the absolute attribution values are converted into relative contributions by dividing the importance of each feature by the total attribution distribution.

An explanation whose importance is concentrated on a small number of features produces a low entropy value and is therefore considered less complex. Conversely, when the attribution importance is distributed more uniformly across many features, the entropy increases and the explanation is considered more complex. Thus, lower scores indicate more concise explanations, whereas higher scores indicate that a larger number of features contribute substantially to the explanation.

The parameters used by this metric can be summarized as follows:

- `normalise (bool)`: whether to normalise the attribution values before computing their relative contribution and entropy. When this parameter is enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)

The absolute-value operation is always applied to the attribution values before computing the metric. This behaviour is defined by the underlying `Quantus` implementation and is fixed in the wrapper since the `abs` parameter is not exposed through the metric configuration. In addition, if all the attribution values supplied to the wrapper are negative, they are explicitly converted to their absolute values before being passed to `Quantus`.

Before computing the metric, the model is set to training mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The metric returns one Complexity score for each evaluated observation. Lower values indicate that the attribution importance is concentrated on fewer features, while higher values indicate a more widely distributed attribution pattern. The mean of the returned scores can be used as an aggregated measure of explanation complexity for the evaluated observations.

The metric is implemented through the `Complexity` class, which can be found in the module `xai_metrics.metrics.complexity`. It can be executed directly using this class (Section 4.2.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.2.1.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. Attribution normalisation is enabled before calculating the entropy-based complexity score.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Complexity_examples.py`

4.2.1.1 Direct evaluation using the Complexity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Complexity` class is instantiated directly using the desired parameters. Calling the `run()` method returns one complexity score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.complexity import Complexity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Complexity(
    context=context,
    params={
        "normalise": True
    }
)
scores = metric.run()
```

4.2.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Complexity"` metric, although this argument would not be necessary if `Complexity` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Complexity"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Complexity
    params:
      normalise: true
```

4.2.2 Sparseness

The Sparseness metric evaluates how concentrated the attribution magnitude is across the input features [16]. For each evaluated observation, the absolute attribution values are sorted, and the Gini index of the resulting attribution distribution is calculated.

An explanation that assigns most of its attribution magnitude to a small subset of features produces a high Sparseness score and is therefore considered more concise. Conversely, when the attribution magnitude is distributed relatively uniformly across many features, the score decreases. Thus, higher values indicate that fewer features account for most of the explanation, whereas values close to (0) indicate a more uniform attribution distribution.

The parameters used by this metric can be summarized as follows:

- `normalise (bool)`: whether to normalise the attribution values before computing the Gini index. When this parameter is enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)

The absolute-value operation is always applied to the attribution values before computing the metric. This behaviour is defined by the underlying `Quantus` implementation and is fixed in the wrapper since the `abs` parameter is not exposed through the metric configuration. In addition, if all the attribution values supplied to the wrapper are negative, they are explicitly converted to their absolute values before being passed to `Quantus`.

The metric returns one Sparseness score for each evaluated observation. Higher values indicate that the attribution magnitude is concentrated on fewer features, while lower values indicate a more

uniform distribution across the input features. The mean of the returned scores can be used as an aggregated measure of explanation sparseness for the evaluated observations.

The metric is implemented through the `Sparseness` class, which can be found in the module `xai_metrics.metrics.complexity`. It can be executed directly using this class (Section 4.2.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.2.2.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. Attribution normalisation is enabled before calculating the Gini-based Sparseness score.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Sparseness_examples.py`

4.2.2.1 Direct evaluation using the Sparseness class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Sparseness` class is instantiated directly using the desired parameters. Calling the `run()` method returns one sparseness score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.complexity import Sparseness

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Sparseness(
    context=context,
    params={
        "normalise": True
    }
)
scores = metric.run()
```

4.2.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Sparseness"` metric, although this argument would not be necessary if `Sparseness` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Sparseness"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Sparseness
    params:
      normalise: true
```

4.3 Sensitivity

The sensitivity property and its interpretation are described in Section 3.1. This section presents the sensitivity metrics implemented in `XAIMetrics`, together with their computation, configurable parameters, implementation details, and execution examples.

4.3.1 Average Sensitivity

The Average Sensitivity metric evaluates how much an explanation changes, on average, when small random perturbations are applied to the corresponding input [17]. For each evaluated observation, several perturbed versions of the original input are generated, and their explanations are recomputed using an explanation function supplied by the user.

For each perturbation, the metric compares the new explanation with the original attribution vector. The sensitivity is calculated as the norm of the difference between both explanations divided by the norm of the original explanation. The final score is obtained by averaging these relative changes across all the generated perturbations.

A low Average Sensitivity score indicates that the explanation changes only slightly under small input perturbations and is therefore considered more stable. Conversely, a high score indicates that minor changes in the input produce substantially different explanations, suggesting greater sensitivity and lower robustness.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. A larger number of samples provides a more extensive estimate of sensitivity but increases the computational cost because the explanation must be recomputed for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before comparing their explanations. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `False`)

- `lower_bound (float)`: lower bound of the uniform noise used to generate the input perturbations. (*Default:* `0.2`)
- `upper_bound (float, optional)`: upper bound of the uniform noise used to generate the input perturbations. When no value is specified, the default behaviour defined by `Quantus` is used. (*Default:* `None`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument will contain the model being evaluated, `inputs` will contain the observations whose explanations must be generated, and `targets` optionally will identify the target outputs. The function must return a `NumPy array` containing the generated attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default comparison, norm, normalisation, and perturbation functions provided by `Quantus`. By default, explanations are compared using their element-wise difference, while the numerator and denominator of the relative change are calculated using the Frobenius norm. The perturbed inputs are generated by adding uniform noise within the configured bounds.

Before computing the metric, the model is set to training mode, following the current implementation of the wrapper. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The execution device stored in the context is also passed to `Quantus`.

The metric returns one Average Sensitivity score for each evaluated observation. Lower values indicate that the generated explanations are more stable under small random input perturbations, whereas higher values indicate greater variation between the original and perturbed explanations.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `AvgSensitivity` class, placed in the module `xai_metrics.metrics.sensitivity`. It can be executed directly using this class (Section 4.3.1.2) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.3.1.3).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated using uniform noise with a lower bound of 0.2. The original signs of the attribution values are preserved, and attribution normalisation is disabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/AvgSensitivity_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.3.1.1 LIME explanation function

Some evaluation metrics require an explanation function that can recompute feature attributions for perturbed observations during their execution. In this example, the required function is created by `make_lime_explain_func()` using the `LIMEExplainer` implementation provided by `XAIMetrics`.

The function first constructs the corresponding `ExplainerContext` from the example configuration file. It then defines the parameters used by LIME and instantiates a `LIMEExplainer`. Finally, the `as_explain_func()` method converts the explainer into a callable that follows the interface expected by the evaluation metrics and by `Quantus`.

```
import pandas as pd
from pathlib import Path
from xai_metrics.config import ConfigController
from xai_metrics.explainers.lime import LIMEExplainer

config_path = "examples/specific_examples/config.yaml"

def make_lime_explain_func():
    context, _ = ConfigController(config_path).build_explainers_context()

    params = {
        "mode": "classification",
        "random_state": 42,
        "num_samples": 5000,
        "labels": [1],
        "distance_metric": "euclidean"
    }

    return LIMEExplainer(context=context, params=params).as_explain_func()
```

In this configuration, LIME operates in classification mode, uses a fixed random seed, generates 5000 perturbed samples, computes distances using the Euclidean metric, and produces the explanation associated with class 1.

This implementation avoids duplicating the internal explanation logic in the example. The returned callable delegates input preparation, model prediction, LIME execution, and attribution formatting to the explainer already integrated into `XAIMetrics`.

4.3.1.2 Direct evaluation using the AvgSensitivity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `AvgSensitivity` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one average sensitivity score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.sensitivity import AvgSensitivity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = AvgSensitivity(
```

```
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False,
        "lower_bound": 0.2
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.3.1.3 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"AvgSensitivity"` metric, although this argument would not be necessary if `AvgSensitivity` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["AvgSensitivity"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: AvgSensitivity
```

```yaml
params:
  nr_samples: 20
  abs: false
  normalise: false
  lower_bound: 0.2
```

4.3.2 Model Randomization

The Model Randomization metric evaluates whether an explanation depends on the parameters learned by the model [18]. The underlying idea is that a meaningful explanation method should react when the learned parameters of the model are modified. Therefore, the metric creates a randomized copy of the original model, recomputes the explanations using this modified model, and compares them with the original attribution values.

For each evaluated observation, the original explanation is compared with the explanation obtained from the randomized model using the Spearman rank correlation coefficient. Formally, the metric is computed as:

$$\mathrm{MR}_i = \rho_{\mathrm{Spearman}}\left(\phi(f, x_i, y_i), \phi(\widetilde{f}, x_i, y_i)\right),$$

where f denotes the original model, $\widetilde{f}$ its randomized copy, and ϕ the explanation function.

Unlike distance-based sensitivity metrics, Model Randomization evaluates the similarity between the rankings induced by the attribution values. Therefore, a high positive correlation indicates that the relative importance assigned to the input features remains similar after randomizing part of the model. Conversely, a low correlation indicates that the explanation changes substantially when the learned parameters are modified, which is the expected behaviour for an explanation method that effectively depends on the model.

The metric returns values normally bounded within $[-1, 1]$. Values close to 1 indicate highly similar attribution rankings between the original and randomized models, values close to 0 indicate weak rank association, and negative values indicate that the attribution rankings tend to be reversed. Consequently, lower correlations generally provide stronger evidence that the explanations depend on the learned parameters of the model.

The parameters used by this metric can be summarized as follows:

- `batch_size (int, optional)`: maximum number of observations whose explanations are recomputed simultaneously. Processing explanations in batches can reduce memory requirements when a large number of observations is evaluated. If `None` is specified, all selected observations are processed together. (*Default:* `64`)
- `fraction (float)`: fraction of parameterised model layers whose parameters are randomized. The value must belong to the interval $[0, 1]$. The number of layers selected for randomization is calculated as

  $$\lfloor N_{\mathrm{layers}} \times \texttt{fraction} \rfloor,$$

  where N_{layers} is the number of parameterised layers detected in the model. (*Default:* `0.25`)
- `reverse (bool)`: whether to reverse the order of the parameterised layers before selecting those that will be randomized. When enabled, the randomization starts from the last parameterised layers encountered while traversing the model, approximating a randomization process that begins from the output side in sequential architectures. (*Default:* `True`)

- `random_state (int, optional)`: random seed used when generating the new parameter values. Providing a fixed value makes the model randomization reproducible. If `None` is specified, the generated parameter values are not associated with a fixed seed. (*Default:* `42`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is required because the explanations must be recomputed after modifying the parameters of the model. It must accept the randomized model, the input observations, and optionally their target values, following the interface:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument contains the randomized copy of the model, `inputs` contains the observations whose explanations must be generated, and `targets` optionally contains their corresponding target outputs. The function must return a `NumPy array` containing one attribution vector for each supplied observation and with the same shape as the original attribution values stored in the `MetricContext`. If no explanation function is provided, the constructor raises a `ValueError`.

The current implementation supports PyTorch models derived from `torch.nn.Module`. Before performing the randomization, a deep copy of the model stored in the `MetricContext` is created. Consequently, the original model is never modified during the evaluation.

The parameterised layers of the copied model are identified by traversing its modules and selecting those containing parameters directly associated with the corresponding module. If `reverse=True`, this collection is reversed before selecting the layers to randomize. The number of randomized layers is then determined by the `fraction` parameter.

For every selected layer, all its parameters are independently replaced with values sampled from a uniform distribution over the interval $[0, 1]$. The randomization is performed without gradient tracking, and the resulting model is subsequently placed in evaluation mode using `eval()` before recomputing the explanations.

It should be noted that the number of randomized layers is obtained using integer truncation. Therefore, for models containing only a small number of parameterised layers, a small positive value of `fraction` may result in zero layers being randomized. In this situation, the copied model retains the same parameters as the original model, and the resulting correlation may therefore remain high, subject to any stochastic behaviour of the explanation method.

After model randomization, the selected observations are processed in batches according to `batch_size`. The attribution values already stored in the `MetricContext` are used as the explanations of the original model, whereas `explain_func` is used exclusively to generate explanations for the randomized model.

For each observation, the metric computes the Spearman correlation between the original and randomized explanations. Before calculating this correlation, both explanations are flattened into one-dimensional vectors. When an explanation is represented by a three-dimensional array, such as an image attribution map with dimensions (H, W, C), the last dimension is first averaged before flattening the attribution map.

If the Spearman correlation is undefined, for example when one of the attribution vectors is constant, the implementation assigns a score of 0.0. The signed correlation value is preserved, so negative values represent an inversion of the attribution ranking rather than simply an absence of association.

The metric returns one Model Randomization score for each evaluated observation. A low score indicates that modifying the learned model parameters produces substantial changes in the

attribution ranking, whereas a high positive score indicates that the explanation remains similar despite the randomization. This makes the metric useful as a sanity check for determining whether the explanation method is actually dependent on the internal parameters learned by the predictive model.

The execution is skipped through a `MetricSkipped` exception when the model is not a PyTorch `torch.nn.Module`, when no observations have been selected, or when the model contains no parameterised layers. A `ValueError` is raised when the number of stored explanations does not match the number of selected inputs, when `batch_size` is not positive, when `fraction` does not belong to $[0, 1]$, or when the explanations returned by `explain_func` do not have the same shape as the original attribution values.

The metric is implemented through the `ModelRandomization` class, placed in the module `xai_metrics.metrics.sensitivity`. It can be executed directly using this class (Section 4.3.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.3.2.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for the configured PyTorch model. In this example, 25% of the parameterised layers are selected for randomization starting from the final layers of the model. Randomized parameter values are generated using a fixed random seed of 42, while explanations are recomputed in batches containing at most 64 observations.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/ModelRandomization_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.3.2.1 Direct evaluation using the ModelRandomization class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `ModelRandomization` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one model randomization score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.sensitivity import ModelRandomization

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = ModelRandomization(
    context=context,
    params={
        "batch_size": 64,
        "fraction": 0.25,
        "reverse": True,
        "random_state": 42
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.3.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"ModelRandomization"` metric, although this argument would not be necessary if `ModelRandomization` is the only metric defined in the configuration.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["ModelRandomization"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: ModelRandomization
    params:
      batch_size: 64
      fraction: 0.25
      reverse: true
      random_state: 42
```

4.3.3 Random Logit

The Random Logit metric evaluates whether an explanation depends on the target class for which it is generated [18]. The underlying idea is that explanations produced for different output classes should generally differ when the explanatory method is sensitive to the class being explained.

Therefore, for each evaluated observation, the metric compares its original explanation with another explanation generated for a randomly selected alternative class.

For an observation x_i with target class y_i, an alternative target class $\widetilde{y}_i$ is sampled uniformly from all classes except the original one:

$$\widetilde{y}_i \sim \text{Uniform}\left(\{0, \ldots, C-1\} \setminus \{y_i\}\right),$$

where C represents the total number of output classes.

The explanation is then recomputed using the same model and input observation but replacing the original target with the sampled alternative class. The Random Logit score is calculated as the Structural Similarity Index (SSIM) between both explanations:

$$\text{RL}_i = \text{SSIM}\left(\phi(f, x_i, y_i), \phi(f, x_i, \widetilde{y}_i)\right),$$

where f denotes the predictive model and ϕ the explanation function.

Unlike metrics that perturb the input observations or modify the parameters of the predictive model, Random Logit keeps both the input and model unchanged. Only the class whose behaviour is being explained is modified. Consequently, the metric evaluates whether the attribution values are specific to the selected output class.

A low Random Logit score indicates that the explanation changes substantially when another target class is selected, suggesting greater target sensitivity or class specificity. Conversely, a high similarity score indicates that similar attribution patterns are generated for different target classes, which may suggest that the explanation method does not sufficiently distinguish between the model outputs.

The parameters used by this metric can be summarized as follows:

- `batch_size (int, optional)`: maximum number of observations whose alternative-target explanations are recomputed simultaneously. Processing observations in batches can reduce memory requirements when a large number of explanations must be generated. If `None` is specified, all selected observations are processed together. (*Default:* `64`)
- `num_classes (int, optional)`: total number of output classes of the predictive model. The configured value must be at least 2. If no value is provided, the metric attempts to infer the number of classes from the model output. For PyTorch models, the second dimension of the output tensor is used, while for models exposing a `predict_proba()` method, the second dimension of the probability matrix is considered. (*Default:* `None`)
- `random_state (int, np.random.Generator, optional)`: random seed or NumPy random number generator used to select the alternative target class for each evaluated observation. Providing a fixed value makes the selection of alternative classes reproducible across executions. (*Default:* `42`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed using the randomly selected alternative target classes. It must accept the model, the input observations, and the corresponding targets, following the interface:

```
1 def explain_func(model, inputs, targets=None, **kwargs):
2 ...
```

The `model` argument contains the original predictive model, `inputs` contains the observations whose explanations must be generated, and `targets` contains the randomly selected alternative classes. The function must return a `NumPy array` containing one attribution vector for each input observation and with the same shape as the original attribution values stored in the `MetricContext`. If no explanation function is provided, the constructor raises a `ValueError`.

The metric requires classification targets to be available through `y_test`. Both one-dimensional integer class labels and two-dimensional encoded class targets are supported. When targets are represented as a one-dimensional array, their values are interpreted directly as class indices. The randomly selected alternative targets are consequently passed to `explain_func` using integer class indices as well.

When the targets are represented as a two-dimensional array, the original class of each observation is obtained by applying `argmax` along the final dimension. In this case, the randomly selected alternative classes are converted back to one-hot encoded vectors before being passed to the explanation function. This preserves the target representation expected by explanation methods operating with encoded class vectors.

If `num_classes` is not explicitly provided, the metric attempts to determine the number of model outputs automatically. For PyTorch models derived from `torch.nn.Module`, the selected observations are passed through the model in evaluation mode without gradient tracking. The number of classes is obtained from the second dimension of the resulting prediction matrix. If the model returns a tuple or list, its first element is considered as the prediction output.

For non-PyTorch models exposing a `predict_proba()` method, the number of classes is inferred from the second dimension of the probability matrix returned by this method. If the model does not belong to either of these cases and `num_classes` has not been configured explicitly, the evaluation is skipped.

Once the number of classes has been determined, one alternative target is sampled independently for each observation. Given C possible classes, each class different from the original target has probability

$$\frac{1}{C-1}$$

of being selected. The implementation ensures that the sampled alternative class can never coincide with the original target class.

After selecting the alternative targets, the observations are processed in batches according to `batch_size`. The original model and the selected inputs are passed to `explain_func` together with their alternative target classes, producing a new set of attribution values that can be compared with the original explanations.

For each observation, the similarity between the original and alternative-target explanation is calculated using a global formulation of the SSIM. Both attribution arrays are first flattened into one-dimensional vectors. Therefore, unlike the local-window formulation commonly used for images, the implementation evaluates the attribution vectors globally.

Given an original explanation e and an alternative-target explanation $\widetilde{e}$, their similarity is calculated as:

$$\mathrm{SSIM}(e,\widetilde{e}) = \frac{\left(2\mu_e\mu_{\widetilde{e}} + C_1\right)\left(2\,\mathrm{cov}(e,\widetilde{e}) + C_2\right)}{\left(\mu_e^2 + \mu_{\widetilde{e}}^2 + C_1\right)\left(\sigma_e^2 + \sigma_{\widetilde{e}}^2 + C_2\right)},$$

where μ_e and $\mu_{\widetilde{e}}$ represent the mean attribution values, σ_e^2 and $\sigma_{\widetilde{e}}^2$ their population variances, and $\mathrm{cov}(e, \widetilde{e})$ their population covariance.

The stabilisation constants are determined from the joint range of the two explanations:

$$C_1 = (0.01L)^2, \qquad C_2 = (0.03L)^2,$$

where

$$L = \max(e, \widetilde{e}) - \min(e, \widetilde{e}).$$

If the joint data range is effectively zero, meaning that both explanation vectors contain almost no variation according to the numerical tolerance used by the implementation, the similarity is directly assigned a value of 1.0. An epsilon value is also added to the denominator of the SSIM expression to improve numerical stability.

The result is not explicitly clipped to a predefined range. Therefore, although SSIM is conventionally interpreted as a similarity measure in which larger values represent greater similarity, small numerical deviations outside its conventional range may occur.

The metric returns one Random Logit score for each evaluated observation. Lower values indicate that the explanation changes when a different output class is selected and therefore suggest greater sensitivity to the explained class. Higher similarity values indicate that the original and alternative-target explanations remain similar.

Because one alternative class is randomly selected for each observation, the resulting scores may differ between executions when the random generator is not fixed. Configuring `random_state` with a fixed seed makes the alternative-class selection reproducible.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected, when classification targets are unavailable, when the number of classes cannot be determined, or when the model output does not correspond to a classification problem containing at least two classes. A `ValueError` is raised when the number of stored explanations differs from the number of selected observations, when `batch_size` is not positive, when `num_classes` is smaller than 2, when one of the target classes lies outside the valid model output range, or when the explanations returned by `explain_func` do not have the same shape as the original attribution values.

The metric is implemented through the `RandomLogit` class, placed in the module `xai_metrics.metrics.sensitivity`. It can be executed directly using this class (Section 4.3.3.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.3.3.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for a binary classification problem. In this example, the number of output classes is explicitly configured as 2, explanations are recomputed in batches containing at most 64 observations, and a fixed random seed of 42 is used to select the alternative target class.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/RandomLogit_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.3.3.1 Direct evaluation using the RandomLogit class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `RandomLogit` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one random logit score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.sensitivity import RandomLogit

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = RandomLogit(
    context=context,
    params={
        "batch_size": 64,
        "random_state": 42,
        "num_classes": 2
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.3.3.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"RandomLogit"` metric, although this argument would not be necessary if `RandomLogit` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["RandomLogit"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: RandomLogit
    params:
      batch_size: 64
      random_state: 42
      num_classes: 2
```

4.4 Robustness

The robustness property and its interpretation are described in Section 3.1. This section presents the robustness metrics implemented in `XAIMetrics`, together with their computation, configurable parameters, implementation details, and execution examples.

4.4.1 Average Stability

The Average Stability metric evaluates the local robustness of an explanation by measuring how much the attribution values change when small perturbations are applied to the corresponding input [10]. For each evaluated observation, several neighbouring inputs are generated by adding random noise. Their explanations are then recomputed using an explanation function provided by the user and compared with the original explanation.

For an observation $\mathbf{x}_i$, the j-th neighbouring input is generated as:

$$\widetilde{\mathbf{x}}_i^{(j)} = \mathbf{x}_i + \boldsymbol{\varepsilon}^{(j)},$$

where $\boldsymbol{\varepsilon}^{(j)}$ represents the random perturbation associated with neighbour j. The difference between the original and neighbouring explanations is subsequently calculated using a configurable distance function:

$$d_i^{(j)} = d\left(\phi(\widetilde{\mathbf{x}}_i^{(j)}), \phi(\mathbf{x}_i)\right),$$

where ϕ denotes the explanation function and d the selected distance measure.

The final Average Stability score for observation i is obtained by averaging the explanation distances across all generated neighbours:

$$\mathrm{AS}_i = \frac{1}{K} \sum_{j=1}^{K} d_i^{(j)},$$

where K is the number of neighbouring inputs generated for each observation.

A low Average Stability score indicates that small perturbations of the input produce explanations similar to the original one and therefore suggests greater local robustness. Conversely, a high value indicates that neighbouring inputs produce substantially different explanations, suggesting that the explanation method is less stable in the local region surrounding the evaluated observation.

Unlike the Local Lipschitz Estimate, which considers the maximum ratio between explanation variation and input variation, Average Stability directly measures the mean distance between the original explanation and the explanations obtained from the sampled neighbours. Therefore, the metric characterizes the average change in the explanation within the sampled neighbourhood rather than its maximum local variation.

The parameters used by this metric can be summarized as follows:

- `radius (float)`: maximum absolute magnitude used when sampling the perturbations applied to the input observations. When the default non-symmetric perturbation strategy is used, noise values are sampled from $[0, \texttt{radius}]$. If symmetric perturbations are enabled, they are sampled from $[-\texttt{radius}, \texttt{radius}]$. The value must be non-negative. (*Default:* `0.1`)
- `nb_samples (int)`: number of neighbouring inputs generated for each evaluated observation. Increasing this value provides a more extensive sampling of the local neighbourhood but also increases the computational cost because the explanation function must be evaluated for every generated neighbour. (*Default:* `20`)
- `distance (str or callable)`: distance function used to compare the explanation of each neighbouring input with the original explanation. The value can be `"l1"`, `"l2"`, or a custom callable. With `"l1"`, the distance is computed as the sum of the absolute element-wise differences. With `"l2"`, the Euclidean distance is used. A custom callable must receive two attribution arrays and return a scalar value. (*Default:* `"l2"`)
- `random_state (int, np.random.Generator, optional)`: random seed or NumPy random generator used to sample the perturbations. Providing a fixed value makes the generated neighbourhood reproducible across repeated executions. (*Default:* `None`)
- `symmetric_noise (bool)`: determines whether perturbations are sampled symmetrically around zero. If `False`, noise is sampled uniformly from $[0, \texttt{radius}]$. If `True`, perturbations are sampled uniformly from $[-\texttt{radius}, \texttt{radius}]$. The default value reproduces the perturbation strategy used by the corresponding Xplique implementation. (*Default:* `False`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing the input observations. It must accept the predictive model, the perturbed observations, and optionally their target values, following the interface:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument contains the model being evaluated, `inputs` contains the neighbouring observations whose explanations must be generated, and `targets` optionally contains their corresponding target outputs. The function must return one attribution array for each supplied input observation. If no explanation function is provided, the constructor raises a `ValueError`.

The observations specified in the `MetricContext` are selected together with their original attribution values and, when available, their target labels. If target values are present, the target associated with each original observation is repeated for all of its generated neighbours so that every perturbed version is explained with respect to the same target.

The perturbations are sampled from a uniform distribution. When `symmetric_noise=False`, the noise applied to every input feature is sampled as:

$$\varepsilon \sim \mathcal{U}(0, \texttt{radius}).$$

When `symmetric_noise=True`, the sampling interval becomes:

$$\varepsilon \sim \mathcal{U}(-\texttt{radius}, \texttt{radius}).$$

An important characteristic of the implementation is that the random perturbation masks are generated only once and subsequently reused for every evaluated observation. Therefore, all observations are evaluated using the same set of perturbation patterns. This provides a more consistent comparison between observations because their local stability is assessed under equivalent random displacements.

For each observation, the K perturbation masks are added to the original input, producing `nb_samples` neighbouring observations. The explanation function is then called once with the complete set of neighbours associated with that observation.

The explanations returned by `explain_func` must have the same feature shape as the original attribution values. In addition, the function must return exactly one explanation for every generated neighbour. Otherwise, the metric raises a `ValueError`.

When `distance="l1"`, the distance between a neighbouring explanation $\widetilde{e}$ and the original explanation e is calculated as:

$$d_{\mathrm{L1}}(\widetilde{e}, e) = \sum_k \left| \widetilde{e}_k - e_k \right|.$$

When `distance="l2"`, the Euclidean distance is used:

$$d_{\mathrm{L2}}(\widetilde{e}, e) = \sqrt{\sum_k \left(\widetilde{e}_k - e_k \right)^2}.$$

Alternatively, a custom distance function can be supplied directly through the `distance` parameter. This callable receives the neighbouring explanation and the corresponding original explanation and must return a finite scalar value. Non-finite distance values cause the evaluation to raise a `ValueError`.

Once the distances between the original explanation and all neighbouring explanations have been calculated, their arithmetic mean is returned as the Average Stability score for that observation.

The metric returns one Average Stability score for each evaluated observation. Lower values indicate that the explanation changes only slightly within the sampled neighbourhood and therefore suggest greater local stability. A score close to zero indicates that the neighbouring observations produce explanations very similar to the original one. Higher values indicate greater variation in the generated explanations.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected. A `ValueError` is raised when the number of original explanations does not match the number of selected inputs, when `radius` is negative, when `nb_samples` is not positive, when an unsupported distance is specified, when the explanation function returns an incorrect number or shape of explanations, or when the calculated distance is not finite.

The metric is implemented through the `AverageStability` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.1.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. In this experiment, 20 neighbouring inputs are generated for each evaluated observation using a perturbation radius of 0.1. The explanations are compared using the Euclidean distance, and a fixed random seed of 42 is used to make the generated perturbations reproducible.

Since `symmetric_noise` is not explicitly included in the example configuration, its default value of `False` is used. Consequently, each component of the perturbation is sampled uniformly from $[0, 0.1]$. The same set of 20 random perturbation masks is reused for every evaluated observation.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/AverageStability_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.1.1 Direct evaluation using the AverageStability class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `AverageStability` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one average stability score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import AverageStability

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = AverageStability(
    context=context,
    params={
        "radius": 0.1,
        "nb_samples": 20,
        "distance": 'l2',
```

```
        "random_state": 42
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"AverageStability"` metric, although this argument would not be necessary if `AverageStability` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["AverageStability"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: AverageStability
    params:
      radius: 0.1
      nb_samples: 20
      distance: 12
      random_state: 42
```

4.4.2 **Local Lipschitz Estimate**

The Local Lipschitz Estimate metric evaluates the local stability of an explanation by comparing changes in the explanation with changes in the corresponding input [11]. For each evaluated observation, several neighbour inputs are generated by applying small Gaussian perturbations. Their explanations are then recomputed using an explanation function provided by the user.

For each perturbed input, the metric calculates the ratio between the distance separating the original and perturbed explanations and the distance separating the corresponding inputs. The Local Lipschitz Estimate is the maximum ratio obtained across all sampled neighbours:

$$\max_{x' \in \mathcal{N}(x)} \frac{\|\Phi(x) - \Phi(x')\|}{\|x - x'\|},$$

where (x) is the original observation, (x') is a perturbed neighbour, and (Φ) represents the explanation function.

A low Local Lipschitz Estimate indicates that nearby inputs receive similar explanations and therefore suggests greater local robustness. Conversely, a high score indicates that small changes in the input may produce comparatively large changes in the explanation. Since the metric evaluates only a finite number of sampled neighbours, the resulting value is an approximation of the maximum local variation rather than the exact Lipschitz constant over the complete neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of neighbour inputs generated for each evaluated observation. Increasing this value provides a more extensive exploration of the local neighbourhood but also increases the computational cost, since an explanation must be generated for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before comparing the explanations. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)
- `perturb_mean (float)`: mean of the Gaussian noise used to generate neighbouring inputs. (*Default:* `0.0`)
- `perturb_std (float)`: standard deviation of the Gaussian noise used to generate neighbour inputs. Larger values produce more distant neighbours and therefore define a wider local region around the original observation. (*Default:* `0.1`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
1 def explain_func(model, inputs, targets=None, **kwargs):
2 ...
```

The `model` argument will contain the model being evaluated, `inputs` will contain the observations whose explanations must be generated, and `targets` optionally will identify the target outputs. The function must return a `NumPy array` containing the generated attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default comparison, distance, normalisation, and perturbation functions provided by `Quantus`. The changes in both the inputs and the explanations are measured using the Euclidean distance, while neighbouring inputs are generated by adding Gaussian noise with the configured mean and standard deviation.

Before computing the metric, the model is set to training mode, following the current implementation of the wrapper. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The execution device stored in the context is also forwarded to `Quantus`.

The metric returns one Local Lipschitz Estimate score for each evaluated observation. Lower values indicate that the explanation varies less than the input within the sampled neighbourhood and therefore suggest greater local stability. Higher values indicate that relatively small input changes may produce large variations in the explanation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `LocalLipschitzEstimate` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.2.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 neighbour inputs are generated using Gaussian noise with a mean of (0.0) and a standard deviation of (0.1). The original signs of the attribution values are preserved, and attribution normalisation is enabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/LocalLipschitzEstimate_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.2.1 **Direct evaluation using the LocalLipschitzEstimate class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `LocalLipschitzEstimate` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one local Lipschitz estimate score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import LocalLipschitzEstimate

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = LocalLipschitzEstimate(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": True,
        "perturb_mean": 0.0,
        "perturb_std": 0.1
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"LocalLipschitzEstimate"` metric, although this argument would not be necessary if `LocalLipschitzEstimate` is the only metric defined in the configuration.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["LocalLipschitzEstimate"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
```

```
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: LocalLipschitzEstimate
    params:
      nr_samples: 20
      abs: false
      normalise: true
      perturb_mean: 0.0
      perturb_std: 0.1
```

4.4.3 Max Sensitivity

The Max Sensitivity metric evaluates the robustness of an explanation by measuring the largest relative change produced in the explanation when small random perturbations are applied to the corresponding input [17]. For each evaluated observation, several perturbed inputs are generated, and their explanations are recomputed using an explanation function provided by the user.

For each perturbed input, the metric compares the new attribution vector with the original explanation. The relative change is calculated as the norm of the difference between both explanations divided by the norm of the original explanation. The Max Sensitivity score is the maximum relative change obtained across all sampled perturbations:

$$\max_{x' \in \mathcal{N}(x)} \frac{\|\Phi(x) - \Phi(x')\|_F}{\|\Phi(x)\|_F},$$

where (x) is the original observation, (x') is a randomly perturbed version of that observation, $(\mathcal{N}(x))$ is the set of sampled neighbours, and (Φ) represents the explanation function. The Frobenius norm is used to measure both the explanation difference and the magnitude of the original explanation.

A low Max Sensitivity score indicates that none of the sampled perturbations produces a large relative change in the explanation and therefore suggests greater robustness. Conversely, a high score indicates that at least one small perturbation produces a substantially different explanation. Since only a finite number of perturbed inputs are evaluated, the resulting value is an approximation of the maximum sensitivity within the sampled neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. Increasing this value explores a larger number of possible variations but also increases the computational cost, since a new explanation must be generated for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)

- `normalise (bool)`: whether to normalise the original and perturbed attribution values before comparing them. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `False`)
- `lower_bound (float)`: lower bound used by the uniform-noise perturbation function when generating perturbed inputs. This parameter controls the range from which the random perturbations are sampled. (*Default:* `0.2`)
- `upper_bound (float or None)`: optional upper bound used by the uniform-noise perturbation function. When its value is `None`, the default behaviour defined by `Quantus` is used. (*Default:* `None`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument contains the model being evaluated, `inputs` contains the observations whose explanations must be generated, and `targets` optionally identifies the target outputs. The function must return a `NumPy array` containing the attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default comparison, norm, normalisation, and perturbation functions provided by `Quantus`. Explanation changes are calculated using an element-wise difference, their magnitude is measured using the Frobenius norm, and perturbed observations are generated using uniform noise with the configured bounds.

Before computing the metric, the model is set to training mode, following the current implementation of the wrapper. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The explanation function and the execution device stored in the context are then forwarded to `Quantus`.

The metric returns one max sensitivity score for each evaluated observation. Lower values indicate that the explanations remain comparatively stable across all the sampled perturbations. Higher values indicate that at least one perturbed input produces a large relative change in the corresponding explanation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `MaxSensitivity` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.3.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.3.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated using uniform noise with a lower bound of (0.2). The original signs of the attribution values are preserved, and attribution normalisation is disabled. Since no value is assigned to `upper_bound`, the default behaviour provided by `Quantus` is used.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MaxSensitivity_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.3.1 Direct evaluation using the MaxSensitivity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `MaxSensitivity` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one max sensitivity score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import MaxSensitivity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MaxSensitivity(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False,
        "lower_bound": 0.2
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.3.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MaxSensitivity"` metric, although this argument would not be necessary if `MaxSensitivity` is the only metric defined in the configuration.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MaxSensitivity"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: MaxSensitivity
    params:
      nr_samples: 20
      abs: false
      normalise: false
      lower_bound: 0.2
```

4.4.4 MeGe

The MeGe metric evaluates the representativity and stability of explanations across models trained on different subsets of the available data [19]. The underlying idea is that models with similar predictive behaviour should also produce similar explanations for the same observations. Therefore, the metric repeatedly retrains the predictive model while excluding different subsets of the selected observations and compares the explanations produced by the resulting models.

The selected observations are first divided into k equally sized subsets. For each subset, a new model is trained using all remaining subsets, while the excluded subset is provided to the training function as hold-out or validation data. Consequently, k independently trained models are obtained.

Each trained model is then used to predict and explain every selected observation. The resulting explanations are compared pairwise, but only under specific conditions intended to ensure that the compared models exhibit equivalent predictive behaviour.

For a given observation, a pair of models is considered valid only when the observation belongs to the subset excluded from the training data of at least one of the two models. In addition, at least

one of the models must predict the true class, and both models must produce the same predicted class.

For every valid pair of explanations e_a and e_b, their dissimilarity is measured using a Spearman-based distance:

$$d(e_a, e_b) = \sqrt{1 - |\rho_{\text{Spearman}}(e_a, e_b)|},$$

where ρ_{Spearman} denotes the Spearman rank correlation between the two attribution vectors.

Because the absolute value of the correlation is used, both perfectly positive and perfectly negative rank correlations produce a distance of zero. A Spearman correlation close to zero produces a distance close to one. This behaviour follows the distance definition used by the original MeGe formulation.

Let $\mathcal{D}_i$ denote the set of valid explanation distances obtained for observation i. The final MeGe score is calculated as:

$$\text{MeGe}_i = \frac{1}{1 + \frac{1}{|\mathcal{D}_i|} \sum_{d \in \mathcal{D}_i} d}.$$

Higher values indicate stronger agreement between the explanations generated by models exhibiting equivalent predictive behaviour. A score close to 1 indicates that the valid model pairs produce highly similar explanation rankings, whereas lower values indicate greater disagreement between the explanations.

The parameters used by this metric can be summarized as follows:

- `k_splits (int)`: number of equally sized subsets into which the selected observations are divided. One independent model is trained for each split using all observations that do not belong to that split. The value must be at least 2, and the number of selected observations must be exactly divisible by `k_splits`. Increasing this value increases both the number of models that must be trained and the number of potential pairwise explanation comparisons. (*Default:* `4`)

In addition to this parameter, the metric requires two external functions: an explanation function, provided through the `explain_func` argument, and a model training function, provided through `training_func`.

The explanation function is used to generate attribution values for each of the models trained during the execution of the metric. It must accept the trained model, the input observations, and their targets, following the interface:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument contains one of the models trained for a particular split, `inputs` contains all selected observations, and `targets` contains their corresponding target values. The function must return one explanation for each supplied observation. If no explanation function is provided, the constructor raises a `ValueError`.

The training function is responsible for constructing and fitting one model for each data split. It must follow the interface:

```
def training_func(X_train, y_train, X_validation, y_validation):
...
```

The `X_train` and `y_train` arguments contain all observations and labels belonging to the subsets used for training, whereas `X_validation` and `y_validation` correspond to the subset excluded from that training iteration. The function must return a trained model compatible with the prediction procedure supported by the metric. If no training function is provided, the constructor raises a `ValueError`.

The model originally stored in the `MetricContext` is not directly used to compute the MeGe score. Instead, the metric relies on the models returned by `training_func`. The original model may nevertheless be used by the user as a template when constructing the training function.

Before training the models, the selected observations and their targets are divided into `k_splits` subsets of identical size. Therefore, the number of selected observations must be exactly divisible by `k_splits`. For every split, all remaining subsets are concatenated to form the training data, while the excluded subset is provided separately to `training_func`.

This procedure produces exactly `k_splits` trained models. Each of these models is subsequently evaluated on every selected observation, regardless of whether that observation belonged to the training data of that particular model.

The metric supports several model prediction interfaces. PyTorch models derived from `torch.nn.Module` are evaluated directly. Models exposing a `predict()` or `predict_proba()` method are also supported.

For PyTorch models, the model is placed in evaluation mode and executed without gradient tracking. If the model returns a tuple or list, its first element is interpreted as the prediction output. When the resulting output is two-dimensional, the predicted class is obtained using `argmax`. One-dimensional outputs are interpreted directly as predicted class indices.

For models exposing `predict()`, one-dimensional outputs are treated as class labels directly. When `predict()` returns a multidimensional output, the class associated with the maximum output value is selected. If the model exposes a `classes_` attribute compatible with the output dimension, the resulting indices are mapped back to the original class labels.

Similarly, when `predict_proba()` is used, the predicted class corresponds to the highest probability. If the model exposes `classes_`, the selected probability index is converted to its corresponding class label.

Classification targets are required by the metric. Targets may be provided as one-dimensional class labels or as two-dimensional encoded vectors. In the latter case, the true target class is obtained using `argmax`.

Once the predictions and explanations generated by all trained models have been obtained, the metric performs pairwise comparisons between them. For a pair of models a and b, an observation is considered only when it belongs to the split excluded from the training data of model a, model b, or both. Observations that were included in the training data of both models are ignored for that pair.

Among these eligible observations, a comparison is retained only if at least one of the two models predicts the true class. Furthermore, both models must predict the same class. These conditions ensure that explanation similarity is evaluated only when the two models exhibit compatible predictive behaviour for the corresponding observation.

The explanations associated with each valid pair are compared using the Spearman-based distance. Before computing the correlation, explanations containing more than two dimensions are

averaged over their final dimension. Missing values are replaced by zero and the resulting arrays are flattened into one-dimensional vectors.

The distance between two explanations is calculated as:

$$d(e_a, e_b) = \sqrt{1 - |\rho_{\text{Spearman}}(e_a, e_b)|}.$$

A value of 0 therefore corresponds to a perfect positive or negative rank association, while a value close to 1 corresponds to little or no monotonic association between the attribution rankings.

If the Spearman correlation is undefined, for example because one of the explanations is constant, the resulting distance is considered `NaN` and is excluded from the collection of valid comparisons for that observation.

Once all valid pairwise distances have been collected, their arithmetic mean is calculated independently for each observation. The final MeGe score is then obtained as:

$$\text{MeGe}_i = \frac{1}{1 + \text{mean}(\mathcal{D}_i)}.$$

The metric returns one MeGe score for each evaluated observation. Higher values indicate that models with equivalent predictive behaviour produce more similar explanations. Lower values indicate greater variation among the explanations produced by the retrained models.

If no valid model pair is available for a particular observation, its score is returned as `NaN`. This can occur, for example, when the trained models repeatedly disagree in their predicted class or when none of the eligible model pairs satisfies the validity conditions.

If no valid comparison is produced for any selected observation, the entire metric is skipped through a `MetricSkipped` exception.

The execution is also skipped when classification targets are unavailable, when no observations have been selected, or when the models returned by `training_func` do not provide a supported prediction interface. A `ValueError` is raised when `k_splits` is smaller than 2, when the number of selected observations is not divisible by `k_splits`, or when `explain_func` does not return one explanation for every selected observation.

The metric is implemented through the `MeGe` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.4.2) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.4.3).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment divides the selected observations into three equally sized subsets. Three independent models are therefore trained, each excluding one of these subsets from its training data.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MeGe_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.4.1 **Model training function**

The MeGe metric requires a training function because several independent models must be fitted using different subsets of the selected data. In this example, the function is created through `make_training_func()`, using the model already loaded into the experimental context as a template.

The function first creates a deep copy of the supplied base model so that every training iteration starts from an independent model instance. The wrapped estimator is then retrieved and fitted using the training observations associated with the current split.

```
from copy import deepcopy

def make_training_func(base_model):
    def training_func(X_train, y_train, X_holdout, y_holdout):
        trained_wrapped = deepcopy(base_model)
        estimator = trained_wrapped.model
        estimator.fit(X_train)
        return trained_wrapped
    return training_func
```

In this example, the underlying estimator is trained using only `X_train`. The corresponding `y_train`, `X_holdout`, and `y_holdout` arguments are accepted to comply with the interface expected by the metric but are not used by the Isolation Forest training procedure.

The use of `deepcopy()` is important because the same base model is reused as the template for all splits. Without creating an independent copy, successive training iterations could modify the same model instance and prevent the resulting models from representing independent training configurations.

The function returns the complete wrapped model rather than the underlying estimator. This ensures compatibility with the prediction and explanation procedures already integrated into `XAIMetrics`.

4.4.4.2 **Direct evaluation using the MeGe class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `MeGe` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one MeGe score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import MeGe

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MeGe(
    context=context,
    params={
```

```
        "k_splits": 3
    },
    explain_func = make_lime_explain_func(),
    training_func = make_training_func(context.model)
)
scores = metric.run()
```

4.4.4.3 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MeGe"` metric, although this argument would not be necessary if `MeGe` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MeGe"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: MeGe
    params:
      k_splits: 3
```

4.4.5 **Relative Input Stability**

The Relative Input Stability metric evaluates explanation robustness by comparing the relative change in an explanation with the relative change in its corresponding input [20]. For each evaluated observation, several perturbed inputs are generated, and their explanations are recomputed using an explanation function provided by the user.

For each perturbed input, the metric calculates the ratio between the relative variation in the explanation and the relative variation in the input. The Relative Input Stability score is the maximum ratio obtained across all the sampled perturbations:

$$\max_{x' \in \mathcal{N}(x)} \frac{\left\| \frac{\Phi(x) - \Phi(x')}{\Phi(x)} \right\|_p}{\max\left(\left\| \frac{x - x'}{x} \right\|_p, \epsilon \right)},$$

where (x) is the original observation, (x') is a perturbed observation, $(\mathcal{N}(x))$ is the set of sampled neighbours, and (Φ) represents the explanation function. The divisions in the numerator and denominator are performed element-wise. A small constant (ϵ) is used to avoid divisions by zero.

A low Relative Input Stability score indicates that the relative change in the explanation is small compared with the relative change in the input and therefore suggests greater robustness. Conversely, a high score indicates that comparatively small relative changes in the input may produce large relative changes in the explanation.

Relative Input Stability considers their relative changes. This reduces the influence of differences in the scale of the input features and attribution values. Since the metric evaluates only a finite number of randomly perturbed inputs, the resulting value is an approximation of the maximum relative instability within the sampled neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. Increasing this value provides a more extensive exploration of the local neighbourhood but also increases the computational cost, since an explanation must be generated for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before calculating their relative changes. When enabled, the `normalise_by_average_second_moment_estimate()` function provided by `Quantus` is used. (*Default:* `False`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument contains the model being evaluated, `inputs` contains the observations whose explanations must be generated, and `targets` optionally identifies the target outputs. The function must return a `NumPy array` containing the generated attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default perturbation and stability functions provided by `Quantus`. Neighbouring inputs are generated by adding uniform noise with an upper bound of (0.2). The relative differences in the inputs and explanations are measured using the corresponding vector norms. A constant of (10^{-6}) is used to prevent divisions by zero.

By default, perturbations that change the model prediction are not considered valid stability comparisons. Their individual results are assigned a `NaN` value. Consequently, the final score for an observation may also be `NaN` when one of the sampled perturbations changes its prediction.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The explanation function and the execution device stored in the context are then forwarded to `Quantus`.

The metric returns one relative input stability score for each evaluated observation. Lower values indicate that the explanations vary less than the inputs in relative terms and therefore suggest greater stability. Higher values indicate that small relative input variations may produce comparatively large changes in the explanations.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `RelativeInputStability` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.6.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.5.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated. The original signs of the attribution values are preserved, and attribution normalisation is disabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/RelativeInputStability_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.5.1 Direct evaluation using the RelativeInputStability class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `RelativeInputStability` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one relative input stability score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import RelativeInputStability

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = RelativeInputStability(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.5.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"RelativeInputStability"` metric, although this argument would not be necessary if `RelativeInputStability` is the only metric defined in the configuration.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["RelativeInputStability"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
```

```yaml
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: RelativeInputStability
    params:
      nr_samples: 20
      abs: false
      normalise: false
```

4.4.6 **Relative Output Stability**

The Relative Output Stability metric evaluates explanation robustness by comparing the relative change in an explanation with the corresponding change in the output of the predictive model [20]. For each evaluated observation, several perturbed inputs are generated, and their explanations and model outputs are recomputed.

For each perturbed input, the metric calculates the ratio between the relative change in the explanation and the distance between the original and perturbed model-output vectors. The Relative Output Stability score is the maximum ratio obtained across all sampled perturbations that preserve the original prediction:

$$\max_{x' \in \mathcal{N}(x)\ \hat{y}x' = \hat{y}x} \frac{\left\| \frac{\Phi(x) - \Phi(x')}{\Phi(x)} \right\|_p}{\max\left(\left\| h(x) - h(x') \right\|_p, \epsilon_{\min} \right)},$$

where (x) is the original observation, (x') is a perturbed neighbour, $(\mathcal{N}(x))$ is the set of sampled neighbours, (Φ) represents the explanation function, and $(h(x))$ represents the model-output vector for (x). The division in the numerator is performed element-wise, and $(\epsilon_{\min})$ is a small constant used to avoid division by zero.

A low Relative Output Stability score indicates that the explanation changes only slightly in relation to the corresponding variation in the model output and therefore suggests greater robustness. Conversely, a high score indicates that a comparatively small change in the model output produces a large relative change in the explanation.

Relative Output Stability uses the behaviour of the predictive model as its reference. Consequently, changes in an explanation are not necessarily considered undesirable when they are accompanied by substantial changes in the model output. Since only a finite number of perturbed inputs are evaluated, the resulting value is an approximation of the maximum relative instability within the sampled neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. Increasing this value provides a more extensive exploration of the local neighbourhood but also increases the computational cost, since the model output and explanation must be recomputed for every perturbation. (*Default:* 200)

- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before calculating their relative changes. When enabled, the `normalise_by_average_second_moment_estimate` function provided by `Quantus` is used. (*Default:* `False`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The implementation uses the default perturbation, normalisation, and stability functions provided by `Quantus`. Neighbouring inputs are generated by adding uniform noise with an upper bound of (0.2). The denominator measures the distance between the original and perturbed model-output vectors, and a constant of (10^{-6}) is used to avoid division by zero.

The metric is intended to compare explanations only when the original and perturbed observations receive the same predicted class. By default, a perturbation that changes the predicted class produces a `NaN` result. Therefore, the score returned for an observation may be `NaN` when its prediction is not preserved across the sampled perturbations.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The explanation function and the execution device stored in the context are then forwarded to `Quantus`.

The metric returns one relative output stability score for each evaluated observation. Lower values indicate that the explanations remain stable relative to changes in the model output. Higher values indicate that small variations in the model output may be accompanied by comparatively large changes in the explanation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `RelativeOutputStability` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section **??**) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.6.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated. The original signs of the attribution values are preserved, and attribution normalisation is disabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/RelativeOutputStability_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.6.1 Direct evaluation using the RelativeOutputStability class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `RelativeOutputStability` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one relative output stability score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import RelativeOutputStability

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = RelativeOutputStability(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.6.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"RelativeOutputStability"` metric, although this argument would not be necessary if `RelativeOutputStability` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["RelativeOutputStability"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: RelativeOutputStability
    params:
      nr_samples: 20
      abs: false
      normalise: false
```

4.5 Fidelity

The fidelity property and the distinction between completeness and soundness are described in Section 3.1. This section presents the fidelity metrics implemented in `XAIMetrics`, organized according to these two subcategories.

4.5.1 Completeness

The following metrics evaluate different aspects of completeness as defined in the taxonomy presented in Section 3.1.

4.5.1.1 Average Drop

The Average Drop metric evaluates whether the features identified as important by an explanation are sufficient to preserve the model score when the remaining information is attenuated [21]. The underlying idea is that, if an explanation correctly identifies the features that are most relevant to the model prediction, retaining those features to a greater extent should preserve a substantial part of the original model score.

For each evaluated observation, the absolute attribution values are converted into a normalised importance mask. Features with larger attribution magnitudes receive values closer to 1, whereas features with smaller attribution magnitudes receive values closer to 0. This mask is multiplied element-wise by the original input to obtain a perturbed version in which the most relevant features are preserved to a greater extent.

For an observation x_i, the original and masked model scores are defined as:

$$\text{base}_i = g(f, x_i, y_i),$$

and

$$\text{after}_i = g(f, x_i \odot M_i, y_i),$$

where f denotes the predictive model, g is the scoring operator, y_i is the evaluated target, M_i is the normalised attribution mask, and $\odot$ represents element-wise multiplication.

The Average Drop score is then computed as:

$$\text{AD}_i = \frac{\max(\text{base}_i - \text{after}_i, 0)}{\text{base}_i + \varepsilon},$$

where ε is a small numerical constant used to avoid division by zero.

The maximum operation in the numerator prevents increases in the model score after masking from producing negative values. Therefore, if the explanation-based mask preserves or increases the original model score, the resulting Average Drop score is 0.

Lower Average Drop values are better. A low score indicates that the features receiving high attribution values are sufficient to preserve the model output after the remaining information is attenuated. Conversely, a high value indicates that applying the explanation-based mask causes a substantial decrease in the selected model score, suggesting that the attribution values may not adequately identify the information required by the model.

The parameters used by this metric can be summarized as follows:

- `batch_size (int, optional)`: maximum number of observations processed simultaneously. If `None` is specified, all selected observations are processed together. Dividing the evaluation into batches can reduce memory requirements for large datasets or models. (*Default:* `64`)
- `operator (callable, optional)`: custom scoring function used to obtain one scalar model score for each observation. The callable must accept the model, input observations, and target values:

```
def operator(model, inputs, targets):
...
```

  When no custom operator is provided, the metric performs model inference directly and selects the corresponding model output. (*Default:* `None`)
- `activation (str, optional)`: optional activation function applied to the model predictions before selecting the score used by the metric. Supported values are `None`, `'sigmoid'`, and `'softmax'`. When `None` is used, the model outputs are left unchanged. (*Default:* `None`)

The attribution mask is constructed independently for each evaluated observation. First, all attribution values are converted to their absolute values. Therefore, Average Drop evaluates the magnitude of feature importance regardless of the original attribution sign.

The resulting values are min-max normalised to the interval $[0, 1]$:

$$M_i = \frac{|\Phi(x_i)| - \min|\Phi(x_i)|}{\max|\Phi(x_i)| - \min|\Phi(x_i)| + \varepsilon},$$

where $\Phi(x_i)$ represents the attribution values associated with observation x_i.

As a consequence, the feature with the highest attribution magnitude receives a mask value close to 1, while the feature with the lowest attribution magnitude receives a value close to 0. Intermediate features are scaled proportionally between these limits.

The mask is subsequently multiplied by the original input:

$$\widetilde{x}_i = x_i \odot M_i.$$

Unlike perturbation strategies based on removing a fixed percentage of features, this implementation applies a continuous attenuation according to the relative attribution magnitude. Features considered more important by the explanation are therefore preserved more strongly, whereas less important features are progressively reduced.

The implementation also supports different attribution and input dimensionalities. For image explanations represented with shape (N, H, W, C), the attribution values are averaged over the final dimension before constructing the mask. This produces a single importance value for each spatial position.

When the corresponding input has shape (N, H, W, C), the resulting mask with shape (N, H, W) is expanded over the channel dimension and broadcast to the complete input shape.

Similarly, for time-series inputs represented with shape (N, T, F), attribution values represented as (N, T) are expanded along the feature dimension. Consequently, the same temporal importance value is applied to all features associated with the corresponding time step.

If the attribution mask cannot be broadcast to the input shape, the metric raises a `ValueError`.

The model score can be obtained in several ways. If a custom `operator` is supplied, this function is used directly. Otherwise, the metric attempts to perform inference using the model contained in the `MetricContext`.

PyTorch models derived from `torch.nn.Module` are evaluated directly. The model is placed in evaluation mode and executed without gradient tracking on the device defined by the metric context. If no device is explicitly configured, the device associated with the model parameters is used whenever possible.

For non-PyTorch models, the implementation first attempts to use `predict_proba()`. If this method is unavailable, `predict()` is used. If neither method is available, the model itself is invoked as a callable.

If `activation='sigmoid'`, the sigmoid function is applied to the model predictions:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

If `activation='softmax'`, the model outputs are converted into normalised class probabilities using the softmax function:

$$\mathrm{softmax}(z_c) = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

When `activation=None`, as in the example configuration, the model outputs are used without applying an additional activation function.

For one-dimensional model outputs, the resulting value is directly used as the score for each observation. For two-dimensional outputs with shape (N, C), where C represents the number of output classes, the score associated with the corresponding target class is selected.

If no targets are available and the model returns a two-dimensional output, the maximum output value is used for each observation.

The metric first computes the model score for the original observations and then generates the explanation-based masked inputs. The same scoring procedure is subsequently applied to these perturbed observations. Therefore, both the original and masked scores are obtained using the same model, targets, activation function, and scoring operator.

The metric returns one Average Drop score for each evaluated observation. Lower values indicate that the model score is preserved when the input is filtered according to the attribution values. A value of 0 means that the score obtained from the explanation-based masked input is equal to or greater than the original model score.

The standard interpretation of Average Drop assumes that the evaluated model score is positive, as commonly occurs when probabilities or non-negative confidence values are used. If raw logits, decision scores, or arbitrary custom operator outputs are employed, this assumption may not hold. In such situations, the conventional interpretation and expected numerical behaviour of the metric should be considered carefully.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected. A `ValueError` is raised when `batch_size` is not positive, when `activation` contains a value different from `None`, `'sigmoid'`, or `'softmax'`, when the number of explanations differs from the number of selected observations, when the explanation mask cannot be broadcast to the input shape, or when the default scoring mechanism receives model predictions with an unsupported dimensionality.

The metric is implemented through the `AverageDrop` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.1.1.2).

Both usage modes rely on the same `YAML` configuration file. In this experiment, observations are processed in batches containing at most 64 samples. No additional activation function is applied to the model outputs, so the scores returned by the model wrapper are used directly when calculating Average Drop.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/AverageDrop_examples.py`

4.5.1.1.1 Direct evaluation using the AverageDrop class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `AverageDrop` class is instantiated directly using the desired parameters. Calling the `run()` method returns one average drop score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import AverageDrop

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = AverageDrop(
    context=context,
    params={
        "batch_size": 64,
        "activation": None
    }
)
scores = metric.run()
```

4.5.1.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"AverageDrop"` metric, although this argument would not be necessary if `AverageDrop` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["AverageDrop"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: AverageDrop
    params:
      batch_size: 64
      activation: null
```

4.5.1.2 **Average Gain**

The Average Gain metric evaluates whether retaining the features identified as important by an explanation increases the model score with respect to the original observation [22]. The metric is complementary to Average Drop: while Average Drop measures the decrease in the model score after explanation-based masking, Average Gain measures the positive increase produced by the same type of masking.

For each evaluated observation, the absolute attribution values are transformed into a normalised importance mask. Features with larger attribution magnitudes are preserved to a greater extent, whereas features with smaller attribution magnitudes are attenuated. The resulting mask is multiplied element-wise by the original input before the model score is recomputed.

For each observation i, the original and masked model scores are defined as:

$$\text{base}_i = g(f, x_i, y_i),$$

and

$$\text{after}_i = g(f, x_i \odot M_i, y_i),$$

where f denotes the predictive model, g is the scoring operator, y_i is the evaluated target, M_i is the attribution-derived mask, and $\odot$ represents element-wise multiplication.

The Average Gain score is then calculated as:

$$\text{AG}_i = \frac{\max(\text{after}_i - \text{base}_i, 0)}{1 - \text{base}_i + \varepsilon},$$

where ε is a small numerical constant used to improve numerical stability.

The numerator represents the positive increase in the model score produced by the explanation-based mask. The maximum operation prevents decreases in the score from producing negative Average Gain values. The denominator corresponds to the remaining possible increase between the original score and the maximum score of 1.

The metric returns one Average Gain value for each evaluated observation. A value of 0 indicates that masking the input according to the explanation does not increase the original model score. Higher values indicate that retaining the features identified as important produces a larger increase in the evaluated score. Therefore, higher values are better.

Average Gain is specifically designed for model scores in the interval $[0, 1]$. Consequently, both the original and masked model scores must remain within this interval. If the model produces raw logits or other unbounded scores, an appropriate activation function or custom scoring operator must be used before calculating the metric.

The parameters used by this metric can be summarized as follows:

- `batch_size (int, optional)`: maximum number of observations processed simultaneously. Processing the observations in batches can reduce memory requirements when evaluating large datasets or computationally expensive models. If `None` is specified, all selected observations are processed together. (*Default:* `64`)

- `operator (callable, optional)`: custom scoring function used to obtain one scalar model score for each observation. The function must accept the model, the input observations, and their target values:

```
def operator(model, inputs, targets):
...
```

  The custom operator should return scores in the interval $[0, 1]$. If no operator is provided, model inference is performed directly using the model stored in the `MetricContext`. (*Default:* `None`)

- `perturb_baseline (str)`: strategy used to construct the reference input by replacing all its feature values. Common supported values include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The attribution mask is constructed independently for each observation. First, the absolute values of the attributions are obtained. Therefore, the sign of an attribution does not affect the mask, and only its magnitude is considered when determining the relative importance of each feature.

The attribution values are subsequently min-max normalised to the interval $[0, 1]$:

$$M_i = \frac{|\Phi(x_i)| - \min |\Phi(x_i)|}{\max |\Phi(x_i)| - \min |\Phi(x_i)| + \varepsilon},$$

where $\Phi(x_i)$ represents the attribution values associated with observation x_i.

As a result, features with the largest attribution magnitudes receive values close to 1, whereas features with the smallest attribution magnitudes receive values close to 0. Intermediate attribution values are scaled proportionally between both limits.

The perturbed observation is obtained by multiplying the original input by this normalised mask:

$$\widetilde{x}_i = x_i \odot M_i.$$

Therefore, the metric does not completely remove a fixed number of features. Instead, it applies a continuous attenuation according to the relative magnitude of their attribution values. Highly relevant features are preserved more strongly, whereas features considered less important are increasingly suppressed.

The implementation also supports attribution maps whose dimensionality differs from the corresponding input representation. For image explanations represented with shape (N, H, W, C), the attribution values are first averaged over the final dimension, producing one importance value for each spatial position.

If the corresponding image inputs have shape (N, H, W, C), the resulting attribution mask with shape (N, H, W) is expanded over the channel dimension and broadcast to match the input shape.

For time-series inputs represented with shape (N, T, F), attribution maps represented as (N, T) are expanded over the feature dimension. Consequently, the same temporal importance value is applied to all features associated with a particular time step.

If the resulting attribution mask cannot be broadcast to the input shape, the metric raises a `ValueError`.

When no custom `operator` is supplied, the implementation determines the model score directly from the predictive model. PyTorch models derived from `torch.nn.Module` are executed directly in evaluation mode and without gradient tracking. The device defined in the `MetricContext` is used for inference. If no device is explicitly configured, the implementation attempts to obtain the device from the model parameters.

For non-PyTorch models, the implementation first attempts to use `predict_proba()`. If this method is unavailable, `predict()` is used. If neither method exists, the model itself is invoked directly as a callable.

When `activation='sigmoid'`, the sigmoid function is applied to the model predictions:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

When `activation='softmax'`, the outputs are converted into normalised class probabilities:

$$\mathrm{softmax}(z_c) = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

When `activation=None`, as in the example configuration, no additional transformation is applied and the model outputs are used directly.

For one-dimensional prediction arrays, each prediction is directly interpreted as the score associated with the corresponding observation. For two-dimensional outputs with shape (N, C), the score corresponding to the target class is selected.

If target labels are not available and the model produces a two-dimensional output, the maximum model output is used as the score for each observation.

The model score is first calculated for the original observations. The explanation-based masks are subsequently constructed and multiplied by the original inputs, after which the model scores are recomputed using exactly the same scoring procedure.

Before calculating Average Gain, the implementation verifies that both the original and masked model scores belong to the interval $[0, 1]$. If any of these values falls outside this interval, the metric raises a `MetricSkipped` exception. In this situation, an activation function such as `'sigmoid'` or `'softmax'`, or a custom operator returning probability-like values, should be used.

The metric returns one Average Gain score for each evaluated observation. Higher values indicate that retaining the information considered important by the explanation increases the model score relative to the original input. A score of 0 indicates that the explanation-based mask does not increase the model score.

Unlike Average Drop, whose decrease is normalised by the original score, Average Gain normalises the positive increase by $1 - \mathrm{base}_i$. This quantity represents the remaining score range between the original prediction and the maximum possible score of 1. Consequently, the interpretation of the metric is inherently associated with scores bounded within $[0, 1]$.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected or when any original or masked model score lies outside the interval $[0, 1]$. A `ValueError` is raised when `batch_size` is not positive, when `activation` contains a value different from `None`, `'sigmoid'`, or `'softmax'`, when the number of explanations differs from the number of selected observations, when the explanation mask cannot be broadcast to the input shape, or when the default scoring procedure receives model predictions with an unsupported dimensionality.

The metric is implemented through the `AverageGain` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.1.2.2).

Both usage modes rely on the same `YAML` configuration file. In this experiment, observations are processed in batches containing at most 64 samples. No additional activation function is applied to the model outputs, so the scores returned by the model are used directly. Consequently, these scores must already belong to the interval $[0, 1]$ for the metric to be evaluated.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/AverageGain_examples.py`

4.5.1.2.1 Direct evaluation using the AverageGain class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `AverageGain` class is instantiated directly using the desired parameters. Calling the `run()` method returns one average gain score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import AverageGain

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = AverageGain(
    context=context,
    params={
        "batch_size": 64,
        "activation": None
    }
)
scores = metric.run()
```

4.5.1.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"AverageGain"` metric, although this argument would not be necessary if `AverageGain` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["AverageGain"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: AverageGain
    params:
      batch_size: 64
      activation: null
```

4.5.1.3 Average Increase

The Average Increase metric evaluates whether retaining the features identified as important by an explanation increases the model score with respect to the original observation [21]. The metric is closely related to Average Drop and Average Gain, but instead of measuring the magnitude of the score variation, it only determines whether an increase occurs.

For each evaluated observation, the absolute attribution values are transformed into a normalised importance mask. Features with larger attribution magnitudes are preserved to a greater extent, whereas features with smaller attribution magnitudes are attenuated. The resulting mask is then multiplied element-wise by the original input before recomputing the model score.

For each observation i, the original and masked model scores are defined as:

$$\text{base}_i = g(f, x_i, y_i),$$

and

$$\text{after}_i = g(f, x_i \odot M_i, y_i),$$

where f denotes the predictive model, g is the scoring operator, y_i is the evaluated target, M_i is the attribution-derived mask, and $\odot$ represents element-wise multiplication.

The Average Increase score is then calculated using an indicator function:

$$\text{AIC}_i = \mathbb{1}\left[\text{after}_i > \text{base}_i\right].$$

Consequently, the metric returns a binary value for each evaluated observation. A value of 1 indicates that applying the explanation-based mask increases the selected model score, whereas a value of 0 indicates that the score remains unchanged or decreases.

When the results are aggregated, their arithmetic mean represents the proportion of evaluated observations for which explanation-based masking increases the model score. Therefore, higher values are better.

Unlike Average Gain, Average Increase does not consider the magnitude of the improvement. Any positive increase produces the same value of 1, regardless of whether the resulting score changes only slightly or increases substantially. Similarly, all unchanged or decreased scores produce a value of 0.

The parameters used by this metric can be summarized as follows:

- `batch_size (int, optional)`: maximum number of observations processed simultaneously. Processing observations in batches can reduce memory requirements when evaluating large datasets or computationally expensive models. If `None` is specified, all selected observations are processed together. (*Default:* `64`)
- `operator (callable, optional)`: custom scoring function used to obtain one scalar model score for each observation. The function must accept the model, the input observations, and their target values:

```
def operator(model, inputs, targets):
...
```

  If no custom operator is provided, the metric performs inference directly using the model stored in the `MetricContext` and selects the corresponding output score. (*Default:* `None`)
- `activation (str, optional)`: optional activation function applied to model predictions before selecting the score associated with each observation. Supported values are `None`, `'sigmoid'`, and `'softmax'`. When `None` is used, the model outputs are left unchanged. Although the comparison can be applied to arbitrary scalar outputs, probability-like or confidence-like scores are generally preferred for interpreting Average Increase. (*Default:* `None`)

The attribution mask is constructed independently for each evaluated observation. First, the attribution values are converted to their absolute values. Therefore, the sign of an attribution does not affect the mask, and only its magnitude is considered when determining the relative importance of each feature.

The resulting attribution magnitudes are independently min-max normalised to the interval $[0, 1]$:

$$M_i = \frac{|\Phi(x_i)| - \min |\Phi(x_i)|}{\max |\Phi(x_i)| - \min |\Phi(x_i)| + \varepsilon},$$

where $\Phi(x_i)$ represents the attribution values associated with observation x_i, and ε is a small numerical constant used to avoid division by zero.

As a result, the feature with the largest attribution magnitude receives a mask value close to 1, whereas the feature with the smallest attribution magnitude receives a value close to 0. Intermediate values are scaled proportionally between these limits.

The perturbed observation is obtained by multiplying the original input by the normalised attribution mask:

$$\widetilde{x}_i = x_i \odot M_i.$$

Therefore, the metric applies a continuous attenuation rather than completely removing a fixed subset of features. Features considered highly important by the explanation are retained more strongly, whereas those receiving lower attribution magnitudes are progressively suppressed.

The implementation also supports attribution maps whose dimensionality differs from that of the corresponding input representation. For image explanations represented with shape (N, H, W, C), the attribution values are averaged over the final dimension before constructing the mask. The resulting mask therefore assigns one importance value to each spatial position.

When the corresponding image inputs have shape (N, H, W, C), the resulting mask with shape (N, H, W) is expanded over the channel dimension and broadcast to the complete input shape.

Similarly, for time-series inputs represented with shape (N, T, F), attribution maps represented as (N, T) are expanded over the feature dimension. Consequently, the same temporal importance value is applied to all features corresponding to a particular time step.

If the attribution mask cannot be broadcast to the input shape, the metric raises a `ValueError`.

When no custom `operator` is supplied, the implementation determines the model score directly from the predictive model. PyTorch models derived from `torch.nn.Module` are executed in evaluation mode and without gradient tracking. The execution device defined in the `MetricContext` is used. If no device is explicitly configured, the implementation attempts to infer it from the model parameters.

For non-PyTorch models, the implementation first attempts to use `predict_proba()`. If this method is unavailable, `predict()` is used. If neither interface exists, the model itself is invoked directly as a callable.

If `activation='sigmoid'`, the sigmoid transformation is applied to the model predictions:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

If `activation='softmax'`, the predictions are transformed into normalised class scores:

$$\mathrm{softmax}(z_c) = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

When `activation=None`, as in the example configuration, no additional activation function is applied and the outputs returned by the model are used directly.

For one-dimensional prediction arrays, each output is directly interpreted as the score associated with the corresponding observation. For two-dimensional model outputs with shape (N, C), the score associated with the target class is selected.

If no targets are available and the model returns a two-dimensional output, the maximum model output for each observation is used.

The metric first calculates the model score for the original observations. The explanation-based masks are subsequently constructed and multiplied by the corresponding inputs. The model scores are then recomputed using these masked observations and exactly the same scoring procedure.

For each observation, the metric compares the masked score with the original score. If

$$\mathrm{after}_i > \mathrm{base}_i,$$

the observation receives a score of 1.0. Otherwise, including the case in which both scores are equal, it receives a score of 0.0.

Unlike Average Gain, Average Increase does not require the model scores to belong to the interval $[0, 1]$. The comparison can therefore be performed directly on arbitrary scalar model outputs. Nevertheless, because the metric was originally introduced as an increase-in-confidence measure, probability-like or confidence-like outputs generally provide the most natural interpretation.

The metric returns one Average Increase score for each evaluated observation. When the individual binary values are averaged, the resulting value corresponds to the proportion of observations for which retaining the features identified as important by the explanation increases the model score.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected. A `ValueError` is raised when `batch_size` is not positive, when `activation` contains a value different from `None`, `'sigmoid'`, or `'softmax'`, when the number of explanations differs from the number of selected observations, when the explanation mask cannot be broadcast to the input shape, or when the default scoring procedure receives model predictions with an unsupported dimensionality.

The metric is implemented through the `Completeness` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.3.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.1.3.2).

Both usage modes rely on the same `YAML` configuration file. In this experiment, observations are processed in batches containing at most 64 samples. No additional activation function is applied, so the scores returned by the model are used directly when determining whether explanation-based masking increases the prediction score.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/AverageIncrease_examples.py`

4.5.1.3.1 Direct evaluation using the AverageIncrease class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `AverageIncrease` class is instantiated directly using the desired parameters. Calling the `run()` method returns one average increase score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import AverageIncrease

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = AverageIncrease(
    context=context,
    params={
        "batch_size": 64,
        "activation": None
    }
)
scores = metric.run()
```

4.5.1.3.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"AverageIncrease"` metric, although this argument would not be necessary if `AverageIncrease` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["AverageIncrease"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: AverageIncrease
    params:
      batch_size: 64
      activation: null
```

4.5.1.4 Completeness

The Completeness metric evaluates whether the attribution values account for the difference between the model output for an original observation and the output obtained for a baseline input [15]. This property is also known as *Summation to Delta* or *Conservation*.

For each evaluated observation, a reference input is constructed by replacing all its features with values determined by a selected baseline strategy. The metric then compares the sum of the feature attributions with the difference between the model outputs for the original and baseline inputs:

$$\sum_{i=1}^{d} \Phi_i(x) \approx f_y(x) - f_y(x_{\text{base}}),$$

where (x) is the original observation, (x_{base}) is the baseline input, $(\Phi_i(x))$ is the attribution assigned to feature (i), and $(f_y(x))$ is the model output associated with the evaluated target.

The metric returns a boolean value for each observation. A value of `True` indicates that the sum of the attribution values satisfies the completeness condition, whereas `False` indicates that the attribution sum does not match the difference between the original and baseline model outputs. When the results are aggregated, their mean represents the proportion of observations that satisfy this property. Therefore, higher values are better.

The parameters used by this metric can be summarized as follows:

- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the attribution values before comparing their sum with the difference between the model outputs. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)
- `perturb_baseline (str)`: strategy used to construct the reference input by replacing all its feature values. Common supported values include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default perturbation and output-transformation functions provided by `Quantus`. The baseline input is generated by replacing all features according to the configured strategy, while the identity function is used to transform the model outputs. Therefore, the metric uses the outputs returned by the model wrapper without applying a softmax transformation.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and attribution values and are passed to `Quantus`.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Completeness` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.4.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.1.7.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The original signs of the attribution values are preserved, attribution normalisation is enabled, and the default `"black"` baseline is used.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Completeness_examples.py`

4.5.1.4.1 Direct evaluation using the Completeness class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Completeness` class is instantiated directly using the desired parameters. Calling the `run()` method returns one completeness score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import Completeness

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Completeness(
    context=context,
    params={
        "abs": False,
        "normalise": True
    }
)
scores = metric.run()
```

4.5.1.4.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Completeness"` metric, although this argument would not be necessary if `Completeness` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Completeness"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
```

```yaml
- name: Completeness
  params:
    abs: false
    normalise: true
```

4.5.1.5 **Deletion**

The Deletion metric evaluates whether the features identified as important by an explanation are actually relevant to the model prediction [23]. The underlying idea is that, if an explanation correctly identifies the features that contribute most strongly to the model output, progressively removing those features should cause the corresponding model score to decrease rapidly.

For each evaluated observation, the input features are ranked according to the absolute magnitude of their attribution values. Starting from the original observation, increasingly large subsets of the most important features are replaced by a configurable baseline value. The model score is recomputed after each perturbation, producing a deletion curve that describes how the prediction evolves as important information is progressively removed.

For an observation x_i, the procedure can be represented as:

$$x_i^{(0)} = x_i,$$

$$x_i^{(k)} = \text{delete}\left(x_i, M_i, k\right),$$

and

$$s_i^{(k)} = g\left(f, x_i^{(k)}, y_i\right),$$

where f denotes the predictive model, g is the scoring operator, M_i represents the feature ranking induced by the explanation, k is the number of features replaced by the baseline, and y_i is the target associated with the observation.

The deletion curve begins with the score obtained from the unmodified observation and progressively incorporates scores corresponding to increasingly perturbed versions of the input. The final Deletion score is obtained by applying trapezoidal averaging to consecutive points of this curve:

$$\text{Deletion}_i = \frac{1}{T}\sum_{t=0}^{T-1}\frac{s_i^{(t)} + s_i^{(t+1)}}{2},$$

where T denotes the number of intervals represented in the deletion curve.

A faithful explanation is expected to assign high importance to features that strongly influence the model output. Therefore, deleting the highest-ranked features should produce a rapid reduction in the score, resulting in a smaller area under the deletion curve. Consequently, lower Deletion values are better. Conversely, a high score indicates that the model output remains comparatively large even after the supposedly important features have been replaced, suggesting weaker correspondence between the attribution ranking and the predictive behaviour of the model.

The parameters used by this metric can be summarized as follows:

- `baseline_mode (float, array-like or callable)`: value used to replace the deleted features. A scalar baseline is broadcast to the complete input shape. An array-like value may also be supplied if it is broadcastable to the selected observations. Alternatively, a callable can be provided; in this case, it receives the selected input observations and must return a scalar or array compatible with their shape. (*Default:* `0.0`)

- `steps (int)`: number of deletion intervals used between the original observation and the maximum configured perturbation level. The metric therefore evaluates `steps + 1` deletion levels, including the unperturbed input. When `-1` is specified, every possible number of deleted features up to the maximum perturbation level is evaluated. (*Default:* `10`)

- `max_percentage_perturbed (float)`: maximum fraction of the available input features that can be replaced by the baseline. The value must belong to the interval $(0, 1]$. A value of 1.0 allows the deletion procedure to continue until all features have been replaced, whereas a value of 0.5, for example, limits the procedure to half of the features. (*Default:* `1.0`)

- `batch_size (int)`: maximum number of perturbed observations processed simultaneously when obtaining model scores. Processing predictions in batches can reduce memory requirements for large datasets or computationally expensive models. (*Default:* `64`)

- `operator (callable, optional)`: custom scoring function used to obtain one scalar model score for each observation. The function must accept the model, the input observations, and their target values:

```
def operator(model, inputs, targets):
...
```

  If no custom operator is supplied, the metric performs inference directly using the model stored in the `MetricContext`. (*Default:* `None`)

- `activation (str, optional)`: activation function applied to the model predictions before selecting the score used to construct the deletion curve. Supported values are `None`, `'sigmoid'`, and `'softmax'`. When `None` is used, the outputs are left unchanged. Applying an activation is useful when the desired interpretation corresponds to the decrease in class probability rather than a decrease in an arbitrary model score or logit. (*Default:* `None`)

Before constructing the deletion curve, both the selected inputs and their corresponding attribution values are flattened. Each explanation must therefore contain exactly the same number of values as the corresponding input observation. If the flattened input and attribution arrays do not have identical shapes, the metric raises a `ValueError`.

The features are ranked independently for each observation according to the absolute magnitude of their attribution values:

$$M_i = \operatorname{argsort}\left(|\Phi(x_i)|\right),$$

with the ordering reversed so that the features with the largest attribution magnitudes appear first. Consequently, both strongly positive and strongly negative attributions are considered highly important for the deletion process. The attribution sign does not determine the deletion order.

The maximum number of features that may be replaced is calculated as:

$$K_{\max} = \lfloor d \cdot p \rfloor,$$

where d is the number of flattened input features and p is the configured parameter `max_percentage_perturbed`. The implementation ensures that at least one feature can be perturbed.

The deletion levels are subsequently generated between zero and $K_{\max}$. With a configured number of `steps`, the implementation uses evenly distributed integer deletion levels between these limits. At deletion level k, the k most important features are replaced by their corresponding baseline values.

When `steps=-1`, the number of steps is set equal to $K_{\max}$, causing the metric to evaluate every possible deletion count from zero up to the maximum number of perturbed features.

The baseline is constructed before performing the deletion procedure. When `baseline_mode` is a scalar, it is broadcast to the complete shape of the selected inputs. When it is a callable, the function receives the complete set of selected observations and its output is subsequently broadcast to the input shape.

For each deletion level, a copy of the original flattened inputs is created. The selected feature positions are then replaced by the corresponding values from the flattened baseline. The perturbed inputs are subsequently reshaped back to their original dimensions before being passed to the predictive model.

The model score can be obtained using several inference interfaces. If a custom `operator` is configured, that function is used directly. Otherwise, PyTorch models derived from `torch.nn.Module` are executed in evaluation mode and without gradient tracking.

For non-PyTorch models, the implementation first attempts to use `predict_proba()`. If this method is unavailable, `predict()` is used. If neither method exists, the model itself is invoked directly as a callable.

When `activation='sigmoid'`, the model predictions are transformed according to:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

When `activation='softmax'`, as in the example configuration, the model outputs are transformed into normalised class scores:

$$\text{softmax}(z_c) = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

For numerical stability, the maximum prediction value is subtracted from the outputs before computing the exponential terms.

For one-dimensional model outputs, the prediction is directly used as the score associated with each observation. For two-dimensional outputs with shape (N, C), the score corresponding to the target class is selected. If target labels are not available, the maximum output across the class dimension is used.

The same scoring procedure is applied at every deletion level. Therefore, the resulting sequence of values describes how the selected model score changes as increasingly important features are replaced by the baseline.

After all deletion levels have been evaluated, the scores are arranged into a deletion curve for each observation. The implementation averages consecutive points using the trapezoidal rule and then calculates the mean of these interval values. This produces one Deletion score for every evaluated observation.

Lower values indicate that removing the features considered important by the explanation causes the model score to decrease rapidly, which suggests greater fidelity between the attribution ranking and the model behaviour. Higher values indicate that the model score remains relatively large throughout the deletion process.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected. A `ValueError` is raised when the number of explanations differs from the number of selected inputs, when each explanation does not contain the same number of values as its corresponding input, when `batch_size` is not positive, when `max_percentage_perturbed` does not belong to $(0, 1]$, when `steps` is neither positive nor equal to `-1`, when `activation` contains a value different from `None`, `'sigmoid'`, or `'softmax'`, or when the configured baseline cannot be broadcast to the shape of the selected observations.

The metric is implemented through the `Deletion` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.5.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.1.5.2).

Both usage modes rely on the same `YAML` configuration file. In this experiment, deleted features are replaced by 0.0, and the perturbation process can continue until 100% of the input features have been replaced. The deletion curve is evaluated using 10 intervals, predictions are processed in batches containing at most 64 observations, and the softmax transformation is applied before selecting the target score.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Deletion_examples.py`

4.5.1.5.1 Direct evaluation using the Deletion class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Deletion` class is instantiated directly using the desired parameters. Calling the `run()` method returns one deletion score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import Deletion

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Deletion(
    context=context,
    params={
        "baseline_mode": 0.0,
        "steps": 10,
        "max_percentage_perturbed": 1.0,
        "batch_size": 64,
        "activation": "softmax"
    }
)
scores = metric.run()
```

4.5.1.5.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Deletion"` metric, although this argument would not be necessary if `Deletion` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Deletion"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Deletion
    params:
      baseline_mode: 0.0
      steps: 10
      max_percentage_perturbed: 1.0
      batch_size: 64
      activation: softmax
```

4.5.1.6 Insertion

The Insertion metric evaluates whether the features identified as important by an explanation are actually relevant to the model prediction [23]. The underlying idea is that, if an explanation correctly identifies the features that contribute most strongly to the model output, progressively restoring those features from a baseline input should cause the corresponding model score to increase rapidly.

For each evaluated observation, the input features are ranked according to the absolute magnitude of their attribution values. The insertion process starts from a baseline input, and increasingly large subsets of the most important features are progressively restored to their original values. The model score is recomputed after each insertion step, producing an insertion curve that describes how the prediction evolves as relevant information is progressively introduced.

For an observation x_i, the procedure can be represented as:

$$x_i^{(0)} = b_i,$$

$$x_i^{(k)} = \text{insert}\left(b_i, x_i, M_i, k\right),$$

and

$$s_i^{(k)} = g\left(f, x_i^{(k)}, y_i\right),$$

where f denotes the predictive model, g is the scoring operator, b_i is the baseline input, M_i represents the feature ranking induced by the explanation, k is the number of features restored from the original observation, and y_i is the corresponding target.

The insertion curve begins with the score obtained from the baseline input and progressively incorporates scores corresponding to increasingly restored versions of the original observation. The final Insertion score is obtained by applying trapezoidal averaging to consecutive points of this curve:

$$\text{Insertion}_i = \frac{1}{T}\sum_{t=0}^{T-1}\frac{s_i^{(t)} + s_i^{(t+1)}}{2},$$

where T denotes the number of intervals represented in the insertion curve.

A faithful explanation is expected to assign high importance to features that strongly influence the model output. Therefore, restoring the highest-ranked features should produce a rapid increase in the score, resulting in a larger area under the insertion curve. Consequently, higher Insertion values are better. Conversely, a low score indicates that restoring the supposedly important features does not rapidly recover the model output, suggesting weaker correspondence between the attribution ranking and the predictive behaviour of the model.

The parameters used by this metric can be summarized as follows:

- `baseline_mode (float, array-like or callable)`: baseline used as the initial state of the insertion process. A scalar value is broadcast to the complete input shape. An array-like value may also be supplied if it is broadcastable to the selected observations. Alternatively, a callable can be provided; in this case, it receives the selected input observations and must return a scalar or array compatible with their shape. (*Default:* `0.0`)
- `steps (int)`: number of insertion intervals used between the baseline input and the maximum configured restoration level. The metric therefore evaluates `steps + 1` insertion levels, including the original baseline state. When `-1` is specified, every possible number of restored features up to the configured maximum is evaluated. (*Default:* `10`)
- `max_percentage_perturbed (float)`: maximum fraction of the available input features that can be restored from the original observation. The value must belong to the interval $(0, 1]$. A value of 1.0 allows the insertion procedure to continue until all features have been restored, whereas a value of 0.5, for example, limits the process to half of the available features. (*Default:* `1.0`)
- `batch_size (int)`: maximum number of perturbed observations processed simultaneously when obtaining model scores. Processing predictions in batches can reduce memory requirements for large datasets or computationally expensive models. (*Default:* `64`)

- `operator (callable, optional)`: custom scoring function used to obtain one scalar model score for each observation. The function must accept the model, the input observations, and their target values:

```
def operator(model, inputs, targets):
...
```

  If no custom operator is supplied, the metric performs inference directly using the model stored in the `MetricContext`. (*Default:* `None`)

- `activation (str, optional)`: activation function applied to the model predictions before selecting the score used to construct the insertion curve. Supported values are `None`, `'sigmoid'`, and `'softmax'`. When `None` is used, the outputs are left unchanged. Applying an activation is useful when the desired interpretation corresponds to the increase in class probability rather than an increase in an arbitrary model score or logit. (*Default:* `None`)

Before constructing the insertion curve, both the selected inputs and their corresponding attribution values are flattened. Each explanation must therefore contain exactly the same number of values as its corresponding input observation. If the flattened input and attribution arrays do not have identical shapes, the metric raises a `ValueError`.

The features are ranked independently for each observation according to the absolute magnitude of their attribution values:

$$M_i = \operatorname{argsort}\left(|\Phi(x_i)|\right),$$

with the ordering reversed so that the features with the largest attribution magnitudes appear first. Consequently, strongly positive and strongly negative attribution values are both regarded as important. The sign of the attribution does not affect the insertion order.

The maximum number of features that may be restored is calculated as:

$$K_{\max} = \lfloor d \cdot p \rfloor,$$

where d is the number of flattened input features and p corresponds to `max_percentage_perturbed`. The implementation ensures that at least one feature can be restored.

The insertion levels are subsequently generated between zero and $K_{\max}$. For a configured number of `steps`, the implementation creates evenly distributed integer restoration levels between these limits. At insertion level k, the k most important features are restored from the original observation while all remaining features preserve their baseline values.

When `steps=-1`, the number of steps is set equal to $K_{\max}$, causing the metric to evaluate every possible insertion count from zero to the maximum number of features that can be restored.

The baseline is constructed before performing the insertion procedure. When `baseline_mode` is a scalar, it is broadcast to the complete shape of the selected inputs. When it is a callable, the function receives the complete set of selected observations and its output is subsequently broadcast to the input shape.

The baseline constitutes the initial state of the insertion process:

$$x_i^{(0)} = b_i.$$

At each subsequent insertion level, a copy of this baseline state is created and the selected feature positions are replaced by their corresponding values from the original observation. The perturbed inputs are then reshaped back to their original dimensions before being passed to the predictive model.

The model score can be obtained using several inference interfaces. If a custom `operator` is configured, that function is used directly. Otherwise, PyTorch models derived from `torch.nn.Module` are executed in evaluation mode and without gradient tracking.

For non-PyTorch models, the implementation first attempts to use `predict_proba()`. If this method is unavailable, `predict()` is used. If neither method exists, the model itself is invoked directly as a callable.

When `activation='sigmoid'`, the model predictions are transformed according to:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

When `activation='softmax'`, as in the example configuration, the model outputs are transformed into normalised class scores:

$$\text{softmax}(z_c) = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

For numerical stability, the maximum prediction value is subtracted from the outputs before computing the exponential terms.

For one-dimensional model outputs, the prediction is directly used as the score associated with each observation. For two-dimensional outputs with shape (N, C), the score corresponding to the target class is selected. If target labels are unavailable, the maximum output across the class dimension is used.

The same scoring procedure is applied at every insertion level. Therefore, the resulting sequence of values describes how the selected model score changes as increasingly important information from the original observation is restored.

After all insertion levels have been evaluated, the scores are arranged into an insertion curve for each observation. The implementation averages consecutive points using the trapezoidal rule and subsequently calculates the mean of these interval values. This produces one Insertion score for each evaluated observation.

Higher values indicate that restoring the features considered important by the explanation causes the model score to increase rapidly, suggesting greater fidelity between the attribution ranking and the model behaviour. Lower values indicate that the model score remains comparatively small even after features with high attribution magnitudes have been restored.

Insertion can be considered complementary to the Deletion metric. Deletion begins with the original observation and progressively removes important features, with lower areas indicating better explanations. Insertion performs the opposite procedure: it begins with a baseline input and progressively restores important features, with higher areas indicating better explanations. Both metrics use the same attribution-based ordering but evaluate the model response from opposite perturbation directions.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected. A `ValueError` is raised when the number of explanations differs from the number of

selected inputs, when each explanation does not contain the same number of values as its corresponding input, when `batch_size` is not positive, when `max_percentage_perturbed` does not belong to $(0, 1]$, when `steps` is neither positive nor equal to `-1`, when `activation` contains a value different from `None`, `'sigmoid'`, or `'softmax'`, or when the configured baseline cannot be broadcast to the shape of the selected observations.

The metric is implemented through the `Insertion` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.6.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.1.6.2).

Both usage modes rely on the same `YAML` configuration file. In this experiment, the insertion process starts from an input containing only zeros, and features are progressively restored to their original values. The procedure can continue until 100% of the input features have been restored. The insertion curve is evaluated using 10 intervals, predictions are processed in batches containing at most 64 observations, and the softmax transformation is applied before selecting the corresponding target score.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Insertion_examples.py`

4.5.1.6.1 Direct evaluation using the Insertion class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Insertion` class is instantiated directly using the desired parameters. Calling the `run()` method returns one insertion score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import Insertion

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Insertion(
    context=context,
    params={
        "baseline_mode": 0.0,
        "steps": 10,
        "max_percentage_perturbed": 1.0,
        "batch_size": 64,
        "activation": "softmax"
    }
)
scores = metric.run()
```

4.5.1.6.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Insertion"` metric, although this argument would not be necessary if `Insertion` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Insertion"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Insertion
    params:
      baseline_mode: 0.0
      steps: 10
      max_percentage_perturbed: 1.0
      batch_size: 64
      activation: softmax
```

4.5.1.7 MuFidelity

The MuFidelity metric evaluates whether the importance assigned by an explanation to subsets of input features is consistent with the effect that perturbing those features produces on the model output [10]. The underlying idea is that, if an explanation correctly identifies the features that are relevant to a prediction, perturbing subsets containing features with larger attribution values should generally produce larger changes in the corresponding model score.

For each evaluated observation, several random binary masks are generated. A mask value of 1 preserves the corresponding input feature, whereas a value of 0 replaces it with a configurable baseline value. For observation x_i and perturbation r, the perturbed input is constructed as:

$$\widetilde{x}_i^{(r)} = x_i \odot m_i^{(r)} + b_i \odot \left(1 - m_i^{(r)}\right),$$

where $m_i^{(r)}$ is the sampled binary mask, b_i is the baseline input, and $\odot$ denotes element-wise multiplication.

The model score is first calculated for the original observation. The same score is subsequently recomputed after applying each random perturbation. The prediction drop associated with perturbation r is defined as:

$$\Delta_i^{(r)} = g(f, x_i, y_i) - g(f, \widetilde{x}_i^{(r)}, y_i),$$

where f denotes the predictive model, g is the scoring operator, and y_i represents the corresponding target.

For the same perturbation, the metric calculates the sum of the attribution values associated with the features that were replaced by the baseline:

$$A_i^{(r)} = \sum_j \Phi_{i,j} \left(1 - m_{i,j}^{(r)}\right),$$

where $\Phi_{i,j}$ represents the attribution assigned to feature j of observation i.

After generating all perturbations, the MuFidelity score is calculated as the Spearman rank correlation between the sequence of model score drops and the corresponding attribution sums:

$$\text{MuFidelity}_i = \rho_{\text{Spearman}} \left(\{\Delta_i^{(r)}\}_{r=1}^R, \{A_i^{(r)}\}_{r=1}^R\right),$$

where R is the number of random perturbations generated for each observation.

A high positive correlation indicates that perturbing subsets containing features with larger total attribution tends to produce larger decreases in the model score. Therefore, higher MuFidelity values indicate stronger agreement between the explanation and the predictive behaviour of the model.

The returned values are Spearman correlation coefficients and therefore normally belong to the interval $[-1, 1]$. Values close to 1 indicate strong positive correspondence between attribution importance and model sensitivity, values close to 0 indicate weak rank association, and negative values indicate that subsets with larger attribution sums tend to produce smaller prediction drops.

The parameters used by this metric can be summarized as follows:

- `nb_samples (int)`: number of random perturbation masks generated for each evaluated observation. Increasing this value provides a more extensive sampling of feature subsets and may produce a more stable correlation estimate, but also increases the number of model evaluations required. (*Default:* `200`)
- `n_masks (int)`: alternative name for `nb_samples`. This parameter is considered only when `nb_samples` is not explicitly provided. (*Default:* `200`, through `nb_samples`)
- `batch_size (int, optional)`: maximum number of perturbed observations processed simultaneously. The implementation divides this capacity between the number of original observations and the number of perturbations so that several perturbations from several observations can be evaluated together without exceeding the configured batch size. If `None` is specified, the effective batch size is set to the total number of generated perturbations. (*Default:* `64`)

- `grid_size (int, optional)`: resolution used to construct coarse perturbation masks for structured inputs. If no value is provided, the size of the first feature dimension of the selected inputs is used. For time-series inputs, masks are initially generated using a temporal grid of this size and subsequently expanded to the complete sequence length. For images, a two-dimensional `grid_size` × `grid_size` mask is generated and expanded to the original spatial dimensions. (*Default:* size of the first feature dimension)

- `subset_percent (float)`: probability that each independently sampled mask element is replaced by the baseline. The value must belong to the interval $[0, 1]$. For example, a value of 0.5 means that each mask element has approximately a 50% probability of being perturbed. (*Default:* `0.2`)

- `subset_probability (float)`: alternative name for `subset_percent`. It is considered only when `subset_percent` is not explicitly configured. (*Default:* `0.2`, through `subset_percent`)

- `baseline_mode (float, array-like or callable)`: baseline used to replace perturbed input features. A scalar value is automatically combined with the generated masks through NumPy broadcasting. A compatible array-like value may also be used. If a callable is provided, it receives the repeated inputs associated with the current perturbation batch and must return a value or array compatible with their shape. (*Default:* `0.0`)

- `operator (callable, optional)`: custom scoring function used to obtain one scalar model score for each observation. The callable must accept the predictive model, the input observations, and their target values:

```
def operator(model, inputs, targets):
...
```

  If no custom operator is provided, the metric performs inference directly using the model stored in the `MetricContext`. (*Default:* `None`)

- `activation (str, optional)`: activation function applied to the model predictions before selecting the score used to calculate the prediction drops. Supported values are `None`, `'sigmoid'`, and `'softmax'`. When `None` is used, the outputs returned by the model are left unchanged. (*Default:* `None`)

- `random_state (int, np.random.Generator, optional)`: random seed or NumPy random generator used to construct the perturbation masks. Providing a fixed value makes the generated subsets reproducible across repeated executions. (*Default:* `None`)

The procedure used to generate the perturbation masks depends on the dimensionality of the selected inputs.

For tabular inputs represented as (N, F), one independent Bernoulli-like decision is generated for every feature and perturbation. Each feature is preserved when a uniformly sampled random value is greater than `subset_percent` and is replaced by the baseline otherwise.

Therefore, the probability of perturbing an individual feature is approximately:

$$P(m_j = 0) = \texttt{subset_percent}.$$

For time-series inputs represented as (N, T, F), perturbation masks are initially generated over a lower-resolution representation with shape $(\texttt{grid_size}, F)$. These masks are subsequently expanded over the temporal dimension using nearest-neighbour indexing.

When `grid_size` is smaller than the original sequence length, several neighbouring time steps therefore share the same perturbation state. This makes it possible to perturb temporal regions rather than treating every individual time step independently.

For higher-dimensional inputs such as images, coarse masks are generated using a `grid_size` × `grid_size` spatial grid. These masks are expanded over the original height and width using nearest-neighbour indexing and are shared across the image channels.

Once the masks have been generated, each original observation is repeated once for every perturbation and combined with the baseline according to:

$$\widetilde{x} = x \odot m + b \odot (1 - m).$$

The same set of generated masks is applied to all observations processed within the corresponding input batch.

The model score can be obtained using several supported inference interfaces. When a custom `operator` is configured, it is used directly. Otherwise, PyTorch models derived from `torch.nn.Module` are executed in evaluation mode and without gradient tracking.

For non-PyTorch models, the implementation first attempts to use `predict_proba()`. If this method is unavailable, `predict()` is used. If neither method exists, the model itself is invoked directly as a callable.

When `activation='sigmoid'`, the model predictions are transformed according to:

$$\sigma(z) = \frac{1}{1 + \exp(-z)}.$$

When `activation='softmax'`, the model outputs are transformed into normalised class scores:

$$\text{softmax}(z_c) = \frac{\exp(z_c)}{\sum_j \exp(z_j)}.$$

When `activation=None`, the model outputs are used directly.

For one-dimensional model outputs, each prediction is directly interpreted as the score of its corresponding observation. For two-dimensional outputs with shape (N, C), the score corresponding to the provided target is selected. When targets are unavailable, the maximum output across the class dimension is used.

Before generating perturbations, the metric calculates the original model score once for every selected observation. These values are subsequently reused when calculating all prediction drops associated with that observation.

The implementation divides the configured `batch_size` between perturbations and original observations. First, the number of perturbations evaluated simultaneously is defined as:

$$B_p = \min(\texttt{batch_size}, \texttt{nb_samples}).$$

The number of original observations that can be processed together is then calculated as:

$$B_x = \max\left(1, \left\lfloor \frac{\texttt{batch_size}}{B_p} \right\rfloor\right).$$

This strategy limits the number of perturbed observations passed to the model simultaneously while allowing multiple perturbations and observations to be evaluated together.

For each input batch, perturbations are repeatedly generated until the configured number of `nb_samples` has been reached. The prediction drops associated with these perturbations are accumulated together with the corresponding sums of attribution values for the perturbed features.

When the dimensionality of the attribution array is one dimension smaller than the input representation, an additional final dimension is introduced before combining the attribution values with the perturbation masks. This allows attribution values such as temporal importance scores to be broadcast across an additional feature dimension when required.

For each observation, Spearman correlation is finally calculated between all accumulated prediction drops and attribution sums. If the correlation is undefined, for example because one of these sequences is constant, the implementation returns a value of 0.0 for that observation.

The metric therefore returns one MuFidelity score for each evaluated observation. Unlike implementations that automatically combine all samples into a single dataset-level result, the current implementation preserves these individual correlation values. Their arithmetic mean can subsequently be calculated when an aggregated measure is required.

Higher positive values indicate stronger agreement between the importance assigned by the explanation and the changes observed in the model output after perturbation. Values close to zero indicate weak correspondence, while negative correlations indicate that the attribution ordering and prediction sensitivity exhibit opposite tendencies.

The execution is skipped through a `MetricSkipped` exception when no observations have been selected. A `ValueError` is raised when `nb_samples` is not positive, when `batch_size` is not positive, when `grid_size` is not positive, when `subset_percent` does not belong to $[0, 1]$, when `activation` contains a value different from `None`, `'sigmoid'`, or `'softmax'`, or when the number of attribution arrays does not match the number of selected observations.

The metric is implemented through the `MuFidelity` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.7.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section **??**).

Both usage modes rely on the same `YAML` configuration file. In this experiment, 200 random perturbations are generated for each evaluated observation. Each mask element has a probability of 0.5 of being replaced by the baseline, and the random seed is fixed to 42 so that the generated perturbation masks are reproducible.

The configuration does not explicitly provide `batch_size`, `grid_size`, `baseline_mode`, or `activation`. Therefore, their default values are used: perturbations are processed with a maximum batch size of 64, the mask grid defaults to the size of the first feature dimension, perturbed values are replaced by 0.0, and no additional activation function is applied to the model outputs.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MuFidelity_examples.py`

4.5.1.7.1 Direct evaluation using the MuFidelity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `MuFidelity` class is instantiated directly using the desired parameters. Calling the `run()` method returns one MuFidelity score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import MuFidelity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MuFidelity(
    context=context,
    params={
        "one_hot_targets": True,
        "num_classes": 2,
        "nb_samples": 200,
        "subset_percent": 0.5,
        "abs": False,
        "normalise": False,
        "random_state": 42
    }
)
scores = metric.run()
```

4.5.1.7.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MuFidelity"` metric, although this argument would not be necessary if `MuFidelity` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MuFidelity"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
```

```
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: MuFidelity
    params:
      one_hot_targets: true
      num_classes: 2
      nb_samples: 200
      subset_percent: 0.5
      random_state: 42
```

4.5.2 Soundness

The following metrics evaluate different aspects of soundness as defined in the taxonomy presented in Section 3.1.

4.5.2.1 Non-Sensitivity

The Non-Sensitivity metric evaluates whether features with negligible attribution values correspond to features on which the selected model output does not functionally depend [14]. For each evaluated observation, the metric identifies the features whose attribution values are below a threshold and compares them with the features whose perturbation produces only a negligible change in the model output.

When one feature is perturbed at each step, the two sets considered by the metric can be expressed as:

$$\mathcal{I}_{\Phi}(x) = \{i \,|\, |\Phi_i(x)| < \varepsilon\}\,,$$

$$\mathcal{I}_{f}(x) = \left\{i \,\middle|\, \left|f_y(x) - f_y\left(x^{(i)}\right)\right| < \varepsilon\right\}.$$

where (x) is the original observation, $(\Phi_i(x))$ is the attribution assigned to feature (i), $(x^{(i)})$ is the observation obtained after replacing feature (i) with a baseline value, and (f_y) is the model output associated with the evaluated target.

The Non-Sensitivity score corresponds to the number of disagreements between these sets:

$$|\mathcal{I}\Phi(x) \triangle \mathcal{I}f(x)|\,,$$

where $(\triangle)$ denotes the symmetric difference between two sets. A disagreement occurs when a feature receives a negligible attribution even though perturbing it changes the model output, or when a feature receives a non-negligible attribution despite having no significant effect on the selected output.

A low Non-Sensitivity score indicates stronger agreement between the attribution values and the functional behaviour of the model. A score of (0) indicates that every feature classified as irrelevant by the explanation is also identified as irrelevant through perturbation, and vice versa. Higher values indicate a larger number of disagreements and therefore weaker compliance with the non-sensitivity property.

The parameters used by this metric can be summarized as follows:

- `eps (float)`: threshold used both to identify negligible attribution values and to determine whether the change in the model output after perturbation is insignificant. Attribution values below this threshold are considered negligible, while output differences smaller than the same threshold indicate that the model is insensitive to the perturbed feature. (*Default:* `1e-5`)
- `features_in_step (int)`: number of features replaced simultaneously at each perturbation step. A value of 1 evaluates each feature individually, whereas larger values perturb groups of features. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute attribution values before identifying negligible features. When enabled, the direction of an attribution is ignored and only its magnitude is considered. If all attribution values are negative and this parameter is disabled, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before applying the `eps` threshold. When enabled, the default `normalise_by_max` function provided by `Quantus` is used. (*Default:* `True`)
- `perturb_baseline (str)`: baseline used to replace the selected features during the perturbation process. Common supported values include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default perturbation and aggregation functions provided by `Quantus`. At each step, the selected features are replaced with values determined by the configured baseline. The resulting model output is compared with the output obtained for the original observation, and differences smaller than `eps` are treated as negligible.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and attribution values and are then passed to `quantus.NonSensitivity`.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric returns one Non-Sensitivity score for each evaluated observation. Lower values indicate better agreement between features with negligible attributions and features that do not significantly affect the model output. The mean of the returned scores represents the average number of disagreements per observation.

Since the score is a count rather than a normalised proportion, its possible magnitude depends on the number of input features and on the value of `features_in_step`. Therefore, direct comparisons between datasets with different dimensionalities should be made with caution.

The metric is implemented through the `NonSensitivity` class, which can be found in the module `xai_metrics.metrics.fidelity.soudness`. It can be executed directly using this class (Section 4.5.2.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.2.1.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. Each feature is perturbed individually using the `"black"` baseline. Absolute attribution values and attribution normalisation are enabled, and a threshold of (10^{-5}) is used to identify negligible attributions and model-output changes.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/NonSensitivity_examples.py`

4.5.2.1.1 Direct evaluation using the NonSensitivity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `NonSensitivity` class is instantiated directly using the desired parameters. Calling the `run()` method returns one non-sensitivity score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.soundness import NonSensitivity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = NonSensitivity(
    context=context,
    params={
        "eps": 1e-5,
        "features_in_step": 1,
        "abs": True,
        "normalise": True,
        "perturb_baseline": "black"
    }
)
scores = metric.run()
```

4.5.2.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"NonSensitivity"` metric, although this argument would not be necessary if `NonSensitivity` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["NonSensitivity"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
```

```yaml
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: NonSensitivity
    params:
      eps: 1e-5
      features_in_step: 1
      abs: true
      normalise: true
      perturb_baseline: black
```

CHAPTER

5

Running Available XAI Methods in the Library

This chapter describes the explanation methods implemented in the **XAIMetrics** library and shows how they can be executed within a common experimental workflow. The current implementation provides four local, post-hoc methods for tabular data: LIME, SHAP, BreakDown, and MAPLE. Although all four methods return feature-level attribution values, they rely on different notions of locality and contribution. LIME fits a surrogate model over randomly perturbed samples, SHAP distributes the difference between a prediction and a reference value using Shapley values, BreakDown constructs an order-dependent sequential decomposition, and MAPLE defines a supervised neighbourhood with a tree ensemble before fitting a local linear model.

Each explainer receives a predictive model and a batch of observations and returns an attribution matrix

$$A = [a_{ij}] \in \mathbb{R}^{n \times d},$$

where n is the number of explained observations, d is the number of input features, and a_{ij} is the attribution assigned to feature j for observation i. The columns of this matrix follow the order defined by `context.X_background`, which provides a common representation for the subsequent evaluation and reporting stages.

For each method, the chapter presents its conceptual basis, the parameters accepted by the corresponding `XAIMetrics` wrapper, and the implementation details that affect the returned attributions. Two execution modes are then illustrated. The first instantiates the explainer class directly, while the second uses `run_explanation()` to construct the experimental context and execute the configured methods automatically from a `YAML` file.

All explainers inherit from `BaseExplainer`, receive an `ExplainerContext`, and implement the common `explain()` method. Depending on the method, the context supplies the predictive model, the background data, optional background responses, the observations to explain, their optional target labels, and the execution device. The explainers are registered by name, which allows the runner to instantiate them from the `explainers` section of the configuration file.

The background data play a different role in each method. LIME uses them to characterize the feature distributions used for perturbation; SHAP uses them as the reference data for the selected SHAP algorithm; BreakDown passes them to the DALEX model explainer; and MAPLE divides them into training and validation subsets used to fit its internal tree ensemble and select the retained features. Consequently, using the same background file improves experimental consistency, but it does not imply that all methods use that information in an equivalent way.

In addition, `BaseExplainer.as_explain_func()` converts an explainer into a callable compatible with evaluation metrics that must recompute explanations for perturbed inputs. The resulting callable follows the interface expected by `Quantus`:

```
def explain_func(model, inputs, targets=None, **kwargs):
    ...
```

The `model` argument identifies the model being evaluated, `inputs` contains the observations whose explanations must be recomputed, and `targets` optionally identifies the outputs to explain. The callable returns a `NumPy` attribution matrix with the same row and feature structure as the direct `explain()` method.

5.1 Local Interpretable Model-agnostic Explanations (LIME)

Local Interpretable Model-agnostic Explanations (LIME) generates a local explanation for an individual observation by approximating the behaviour of the original predictive model in a neighbourhood around that observation [5]. Instead of attempting to explain the complete model globally, LIME generates perturbed samples close to the observation and fits a simpler interpretable model using the predictions produced for those samples.

The local surrogate model is obtained by minimizing an objective that combines its approximation error around the observation with a complexity penalty:

$$\xi(x) = \arg\min_{g \in \mathcal{G}} \left[\mathcal{L}\left(f, g, \pi_x\right) + \Omega(g) \right].$$

where x is the observation being explained, f is the original predictive model, g is an interpretable local surrogate model belonging to the family $\mathcal{G}$, π_x assigns greater importance to perturbed samples that are closer to x, $\mathcal{L}$ measures how well g approximates f in the local neighbourhood, and $\Omega(g)$ penalizes the complexity of the surrogate model.

For tabular data, LIME generates a set of perturbed observations around the input and obtains their predictions from the original model. Each perturbed sample receives a weight according to its distance from the original observation. A weighted local model is then fitted, and its coefficients are used as feature-attribution values.

A positive attribution indicates that the corresponding feature contributes towards the selected target output in the local surrogate model, whereas a negative attribution indicates a contribution in the opposite direction. The magnitude of the attribution represents the local importance assigned to the feature. Since LIME approximates the model only within a sampled neighbourhood, its results may vary depending on the perturbation process, the random seed, the kernel, and the number of generated samples.

The implementation provided by `XAIMetrics` uses the `LimeTabularExplainer` class from the `lime` package. The background data stored in the `ExplainerContext` are used to initialize this object and characterize the feature distributions from which the perturbed observations are generated.

The columns of `context.X_background` determine the order of the returned attribution values. For every explained observation, the implementation creates a vector with one position per input feature. Features omitted from the local explanation, for example because `num_features` is smaller than the total number of features, receive an attribution value of 0.

The parameters supported by the explainer can be summarized as follows:

- `mode (str)`: type of predictive task explained by LIME. The usual supported values are `"classification"` and `"regression"`. In classification mode, the explainer requires a probability-prediction function. In regression mode, it uses the standard prediction function. (*Default:* `"classification"`)
- `categorical_features (list or None)`: indices of the features that should be treated as categorical. These indices refer to the column order of `X_background`. (*Default:* `None`)
- `categorical_names (dict or None)`: mapping from categorical feature indices to the names of their possible categories. (*Default:* `None`)
- `kernel_width (float or None)`: width of the proximity kernel used to weight the perturbed observations. Smaller values give more importance to samples very close to the explained observation, whereas larger values define a wider local neighbourhood. If no value is provided, LIME determines its default width from the number of features. (*Default:* `None`)
- `kernel (callable or None)`: optional custom kernel function used to transform distances into proximity weights. Since Python callables cannot be defined directly in a standard `YAML` file, this option is mainly intended for direct class instantiation. (*Default:* `None`)
- `verbose (bool)`: whether the underlying LIME implementation should print additional information during the construction of the local explanation. (*Default:* `False`)
- `class_names (list or None)`: names assigned to the model classes in classification mode. This parameter affects the representation of the explanation but not the order of the returned attribution matrix. (*Default:* `None`)
- `feature_selection (str)`: strategy used by LIME to select the features included in the local surrogate model. Supported strategies depend on the underlying LIME implementation and include options such as `"auto"`, `"forward_selection"`, `"highest_weights"`, `"lasso_path"`, and `"none"`. (*Default:* `"auto"`)
- `discretize_continuous (bool)`: whether continuous variables should be discretized before generating the interpretable representation used by LIME. (*Default:* `True`)
- `discretizer (str)`: strategy used to discretize continuous features. Common options supported by LIME include `"quartile"`, `"decile"`, and `"entropy"`. This parameter is used only when continuous-feature discretization is enabled. (*Default:* `"quartile"`)
- `sample_around_instance (bool)`: whether continuous perturbations should be centred around the observation being explained instead of around the mean of the background data. (*Default:* `False`)
- `random_state (int)`: random seed used during the perturbation and local-model construction processes. Fixing this value improves the reproducibility of the generated explanations. (*Default:* `42`)
- `labels (tuple, list or None)`: target labels for which explanations should be generated. When this parameter is not provided and `top_labels` is also disabled, the implementation uses the target associated with each observation. If no target values are supplied, class 1 is explained by default. (*Default:* `None`)

- `top_labels (int or None)`: number of classes with the highest predicted values for which LIME should generate explanations. When this option is enabled, the `XAIMetrics` wrapper uses the first class returned in `explanation.top_labels` to construct the attribution vector. (*Default:* `None`)
- `num_features (int)`: maximum number of features included in each local explanation. Features not selected by LIME are represented with zero values in the returned attribution matrix. (*Default:* number of input features)
- `num_samples (int)`: number of perturbed observations generated to fit each local surrogate model. Larger values may provide a more stable approximation of the local behaviour but increase the computational cost. A separate perturbation dataset and local model are generated for every explained observation. (*Default:* `500`)
- `distance_metric (str)`: distance metric used to calculate the proximity between the original observation and its perturbed samples. (*Default:* `"euclidean"`)
- `model_regressor (Any or None)`: optional regressor used to fit the local surrogate model. The regressor must support sample weights and expose its fitted coefficients. Since this object cannot normally be represented directly in `YAML`, it is mainly intended for direct class usage. (*Default:* `None`)

The explainer accepts input observations represented as a `pandas.DataFrame`, a PyTorch `Tensor`, a `NumPy` array, or another array-like object. Data frames are reordered according to the columns of `X_background`, while tensors are detached and transferred to the CPU before being converted into `NumPy` arrays. A one-dimensional input is reshaped automatically into a batch containing one observation.

The prediction function used by LIME depends on the configured mode. In classification mode, the implementation searches for a `predict_proba()` method. In any other mode, it searches for `predict()`. The required method is first searched directly on the supplied model and then on its internal `model` attribute. If the required method is unavailable in both locations, the explainer raises an `AttributeError`.

For each input observation, the selected label is determined according to the configured parameters. If `top_labels` is provided, the first class returned by LIME is used. Otherwise, the first value specified in `labels` is selected. When neither parameter is configured, the target passed to `explain()` is used for each observation. If no target is available, label 1 is selected by default.

The local feature weights returned by `LimeTabularExplainer` are converted into a fixed-size attribution vector. Each weight is placed in the position corresponding to its original feature index, ensuring that every returned row follows the column order of the background dataset.

The explainer returns a two-dimensional `NumPy` array with one row for each explained observation and one column for each input feature:

$$A \in \mathbb{R}^{n \times d},$$

where n is the number of observations in the explained batch and d is the number of features in `X_background`.

The explainer is implemented through the `LIMEExplainer` class, located in the module `xai_metrics.explainers.lime`. It is registered under the name `"LIME"` and can be executed directly using this class (Section 5.1.1) or as part of the automated explanation pipeline provided by `run_explanation()` (Section 5.1.2).

Both usage modes rely on the same `YAML` configuration file. The example generates LIME explanations for an Isolation Forest model on the MetroPT3 dataset. The explainer operates in classification mode, generates 5000 perturbed samples for each observation, explains class 1, calculates distances using the Euclidean metric, and uses a fixed random seed of 42.

Examples of both usage modes are available in the library under: `examples/specific_examples/explainers/LIME_examples.py`

5.1.1 Direct explanation using the LIMEExplainer class

In the direct usage mode, the explanation context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, background data, optional background labels, observations to explain, target values, and execution device required by the explanation process.

After constructing the context, the `LIMEExplainer` class is instantiated directly using the desired parameters. Calling the `explain()` method with the model, input batch, and target values returns the complete attribution matrix.

```
from xai_metrics.config import ConfigController
from xai_metrics.explainers.lime import LIMEExplainer

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()
explainer = LIMEExplainer(
    context=context,
    params={
        "mode": "classification",
        "random_state": 42,
        "num_samples": 5000,
        "labels": [1],
        "distance_metric": "euclidean",
    },
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch,
)
```

The resulting array contains one row for every observation in `context.X_batch` and one column for every feature in `context.X_background`. The mean absolute attribution calculated in the example provides a compact summary of the magnitude of the generated values, but it should not be interpreted as a measure of explanation quality.

5.1.2 Automated explanation using run_explanation()

The automated usage mode uses the `run_explanation()` function. The explanation context, the explainers to be executed, and their parameters are obtained from the `YAML` configuration file.

The optional `selected_explainers` argument can be used to restrict the execution to a subset of the configured explanation methods. In this example, it selects only the registered `"LIME"` explainer, although this argument would not be necessary if LIME were the only explainer defined in the configuration.

```python
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"
results = run_explanation(
    selected_explainers=["LIME"],
    config=config_path,
    attribution_output_dir=None,
)
```

The returned structure contains the experimental metadata and the paths of any attribution files generated during the execution. When no output directory is provided, the attributions remain available in the returned result without being explicitly exported through the argument shown in the example.

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

explainers:
  - name: LIME
    params:
      mode: classification
      random_state: 42
      num_samples: 5000
      labels: [1]
      distance_metric: euclidean
```

The paths to `X_background` and `y_background` provide the reference data used to initialize the LIME tabular explainer. The paths to `X_batch` and `y_batch` identify the observations and targets for which the explanations are generated. The test-data and attribution paths belong to the shared experimental context and are primarily used by the metric-evaluation workflow.

5.2 SHapley Additive exPlanations (SHAP)

SHAP! (**SHAP!**) generates feature attributions by applying concepts from cooperative game theory to the explanation of predictive models [4]. Each input feature is interpreted as a player in a cooperative game, while the model prediction represents the value obtained by a coalition of features. The attribution assigned to each feature measures its average marginal contribution across the possible coalitions in which it may participate.

For an observation x, let F denote the complete set of input features and let $S \subseteq F$ be a coalition of features. The value of a coalition can be represented by a function $v_x(S)$. A common conditional formulation is

$$v_x(S) = \mathbb{E}\left[f(X) \mid X_S = x_S\right],$$

where the features in S are fixed to their values in the explained observation and the remaining features are integrated with respect to a reference distribution. In practice, the exact interpretation of $v_x(S)$ depends on the masker and algorithm selected by `shap.Explainer`.

The SHAP value assigned to feature i is its weighted average marginal contribution across all coalitions that do not contain that feature:

$$\phi_i(x) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[v_x\left(S \cup \{i\}\right) - v_x(S) \right].$$

The difference $v_x(S \cup \{i\}) - v_x(S)$ is the marginal contribution of feature i when it is added to coalition S. The factorial term weights each coalition so that the resulting attribution is equivalent to averaging this marginal contribution over all possible feature orderings.

For the selected scalar model output, the SHAP values satisfy the additive decomposition

$$v_x(F) = v_x(\varnothing) + \sum_{i=1}^{d} \phi_i(x),$$

where $v_x(\varnothing)$ is the reference or base value and $v_x(F)$ is the model output associated with the complete observation. When the value function reproduces the selected prediction exactly, $v_x(F) = f(x)$.

A positive SHAP value indicates that the corresponding feature increases the selected model output relative to the reference value, whereas a negative value indicates that it decreases that output. The magnitude of the value represents the strength of the contribution. However, the precise interpretation depends on the output explained by SHAP, such as a class probability, a regression prediction, or another model score.

The exact computation of Shapley values requires evaluating all possible feature subsets and is generally computationally expensive. Therefore, SHAP provides several exact and approximate algorithms adapted to different model types and computational constraints. The specific algorithm used by the library can be selected explicitly or determined automatically from the prediction function, background data, and model characteristics.

The implementation provided by `XAIMetrics` uses the generic `shap.Explainer` interface. The background data stored in the `ExplainerContext` are used to initialize the SHAP explainer and define the reference distribution against which feature contributions are calculated.

The columns of `context.X_background` determine the names and order of the features in the returned attribution matrix. The background data can optionally be subsampled before initializing SHAP, reducing the computational and memory requirements associated with large reference datasets.

The parameters supported by the explainer can be summarized as follows:

- `mode (str)`: type of predictive task explained by SHAP. If its value is `"classification"`, the wrapper uses the model's `predict_proba()` method. For any other value, it uses `predict()`. (*Default:* `"classification"`)
- `max_background_samples (int or None)`: maximum number of observations retained from the background dataset. When this parameter is provided, the background data are subsampled using `shap.sample()`. Reducing the background size can lower the computational cost, although it may also produce a less representative reference distribution. (*Default:* `None`)

- `random_state (int)`: random seed used when subsampling the background data and when initializing `shap.Explainer`. Fixing this value improves the reproducibility of algorithms that involve random sampling. (*Default:* `42`)
- `algorithm (str)`: algorithm passed to `shap.Explainer`. The value `"auto"` allows SHAP to select an appropriate algorithm automatically. Other supported values depend on the installed SHAP version and may include strategies such as `"permutation"`, `"partition"`, `"tree"`, or `"linear"`. (*Default:* `"auto"`)
- `output_names (list or None)`: optional names assigned to the model outputs. This information is passed to `shap.Explainer` and can be useful when the model produces multiple outputs. (*Default:* `None`)
- `output_index (int or None)`: output selected when SHAP returns a separate attribution value for each model output. When this parameter is provided, it takes precedence over the targets supplied to `explain()` and over the target values stored in the context. (*Default:* `None`)
- `default_output (int)`: output index selected when SHAP returns multi-output values and neither `output_index`, the targets passed to `explain()`, nor `context.y_batch` are available. (*Default:* `1`)
- `max_evals (int or None)`: maximum number of model evaluations allowed during the SHAP computation. When no value is provided, the string `"auto"` is passed to SHAP. The interpretation and minimum valid value depend on the algorithm selected by `shap.Explainer`. (*Default:* `None`)
- `batch_size (int or None)`: batch size used by SHAP when evaluating the model. When this parameter is not specified, the value `"auto"` is used. Larger batches may improve execution efficiency but require more memory. (*Default:* `None`)
- `abs (bool)`: whether to convert the generated SHAP values into their absolute values before returning them. When enabled, the direction of each feature contribution is discarded and only its magnitude is preserved. (*Default:* `False`)

The explainer accepts input observations represented as a `pandas.DataFrame`, a PyTorch `Tensor`, a `NumPy` array, or another array-like object. Data frames are reordered according to the columns of `X_background`. PyTorch tensors are detached, transferred to the CPU, and converted into `NumPy` arrays. One-dimensional inputs are reshaped automatically into a batch containing one observation.

The prediction function used by SHAP depends on the configured mode. In classification mode, the implementation searches for a `predict_proba()` method. In any other mode, it searches for `predict()`. The required method is first searched directly on the supplied model and then on its internal `model` attribute. If it is unavailable in both locations, the explainer raises an `AttributeError`.

The underlying `shap.Explainer` is created lazily the first time a particular model is explained. The implementation caches one explainer for each model object using its identity. Subsequent calls with the same model reuse the cached object, avoiding the repeated initialization of the SHAP explainer and its background data.

After the explanation is computed, the wrapper extracts the array stored in `explanation.values`. If this array has two dimensions, it is returned directly because it already contains one attribution vector per observation:

$$A \in \mathbb{R}^{n \times d},$$

where n is the number of explained observations and d is the number of input features.

For models with multiple outputs, SHAP may return a three-dimensional array:

$$A \in \mathbb{R}^{n \times d \times c},$$

where c is the number of model outputs. In this case, the wrapper reduces the result to a two-dimensional attribution matrix by selecting one output for each observation.

The selected output is determined according to the following priority:

1. The value explicitly configured through `output_index`.
2. The target values passed to the `explain()` method.
3. The target values stored in `context.y_batch`.
4. The value configured through `default_output`.

When target values are used, each observation may select a different output index. The number of targets must match the number of explained observations. Otherwise, the wrapper raises a `ValueError`.

If the values returned by SHAP do not have two or three dimensions, the implementation raises a `ValueError`, since their structure cannot be converted into the fixed attribution matrix expected by the library.

When `abs=True`, the absolute-value operation is applied after the output dimension has been selected. Therefore, the returned matrix preserves the feature structure but no longer indicates whether each feature increases or decreases the selected output.

The explainer returns a two-dimensional `NumPy` array with one row for each explained observation and one column for each feature in `X_background`. This uniform representation makes the resulting attribution matrix compatible with the evaluation metrics and reporting components provided by `XAIMetrics`.

The explainer is implemented through the `SHAPExplainer` class, located in the module `xai_metrics.explainers.shap`. It is registered under the name `"SHAP"` and can be executed directly using this class (Section 5.2.1) or as part of the automated explanation pipeline provided by `run_explanation()` (Section 5.2.2).

Both usage modes rely on the same `YAML` configuration file. The example generates SHAP explanations for an Isolation Forest model on the MetroPT3 dataset. The explainer operates in classification mode, allows SHAP to select the algorithm automatically, uses a maximum of 100 background observations, explains output 1, and uses a fixed random seed of 42.

Examples of both usage modes are available in the library under: `examples/specific_examples/explainers/SHAP_examples.py`

5.2.1 Direct explanation using the SHAPExplainer class

In the direct usage mode, the explanation context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, background data, optional background labels, observations to explain, target values, and execution device required by the explanation process.

After constructing the context, the `SHAPExplainer` class is instantiated directly using the desired parameters. Calling the `explain()` method with the model, input batch, and target values returns the complete attribution matrix.

```
from xai_metrics.config import ConfigController
from xai_metrics.explainers.shap import SHAPExplainer

config_path = PROJECT_ROOT / "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()
explainer = SHAPExplainer(
    context=context,
    params={
        "mode": "classification",
        "algorithm": "auto",
        "max_background_samples": 100,
        "output_index": 1,
        "random_state": 42,
    },
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch,
)
```

The resulting array contains one row for every observation in `context.X_batch` and one column for every feature in `context.X_background`. Since `output_index` is explicitly set to 1, the targets passed to `explain()` are not used to select the explained output in this example.

The mean absolute attribution calculated in the complete example summarizes the average magnitude of the generated SHAP values. However, it should not be interpreted as a direct measure of explanation quality, since it depends on the model-output scale, the background distribution, and the number and scale of the input features.

5.2.2 Automated explanation using run_explanation()

The automated usage mode uses the `run_explanation()` function. The explanation context, the explainers to be executed, and their parameters are obtained from the `YAML` configuration file.

The optional `selected_explainers` argument can be used to restrict the execution to a subset of the configured explanation methods. In this example, it selects only the registered `"SHAP"` explainer, although this argument would not be necessary if SHAP were the only explainer defined in the configuration.

```python
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"
results = run_explanation(
    selected_explainers=["SHAP"],
    config=config_path,
    attribution_output_dir=None,
)
```

The returned structure contains the experimental metadata and the paths of any attribution files generated during the execution. When no output directory is provided, the attribution values remain available in the returned result without being exported through the argument shown in the example.

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

explainers:
  - name: SHAP
    params:
      mode: classification
      algorithm: auto
      max_background_samples: 100
      output_index: 1
      random_state: 42
```

The paths to `X_background` and `y_background` provide the reference data used to initialize the SHAP explainer. The paths to `X_batch` and `y_batch` identify the observations and targets for which the explanations are generated.

In this configuration, `output_index` is set explicitly to 1. Consequently, the wrapper always selects the SHAP values corresponding to output 1, even though `y_batch` is available in the context.

5.3 BreakDown

BreakDown generates a local explanation by decomposing the prediction of an individual observation into a sequence of feature contributions [7]. Starting from a reference prediction, the method introduces the feature values of the explained observation one at a time and measures how the expected model output changes at each step.

Let

$$\pi = (\pi_1, \pi_2, \ldots, \pi_d),$$

denote an ordering of the d input features, and let

$$S_j = \{\pi_1, \pi_2, \ldots, \pi_j\}, \qquad S_0 = \varnothing,$$

represent the subset containing the first j features in that ordering. The contribution assigned to feature π_j can be expressed as:

$$\Delta_{\pi_j}(x) = v_x(S_j) - v_x(S_{j-1}),$$

where x is the observation being explained and $v_x(S)$ represents the expected model output when the features in S are fixed to their values in x. A common theoretical definition of this value function is:

$$v_x(S) = \mathbb{E}[f(X) \mid X_S = x_S],$$

where f is the predictive model, X_S denotes the variables contained in S, and x_S contains their values in the explained observation.

The resulting local decomposition can be written as:

$$v_x(F) = v_x(\varnothing) + \sum_{j=1}^{d} \Delta_{\pi_j}(x),$$

where $v_x(\varnothing)$ is the reference prediction and each $\Delta_{\pi_j}(x)$ is the change produced when feature π_j is incorporated after the preceding features in the selected order. For the complete coalition, $v_x(F)$ corresponds to the selected model output for the observation.

A positive contribution indicates that the feature increases the selected model output relative to the value obtained in the preceding step. Conversely, a negative contribution indicates that the feature decreases that output. The magnitude of the contribution represents the size of the change attributed to the feature within the sequential decomposition.

BreakDown does not average the contribution of each feature across all possible feature orderings. Consequently, the attribution assigned to a feature may depend on the order in which the variables are introduced. This effect is particularly relevant when features interact or are statistically dependent. Different orders may distribute the same total prediction difference differently across the input features, although the contributions still form an additive decomposition of the selected prediction.

The implementation provided by `XAIMetrics` uses the `Explainer` and `predict_parts()` functionality from the `DALEX` package. The DALEX explainer is constructed using the model, the background data stored in the `ExplainerContext`, the optional background targets, and a prediction function adapted to the configured task.

For every observation, the wrapper calls `predict_parts()` with `type="break_down"`. The contributions returned by DALEX are then converted into a fixed-size attribution vector whose positions follow the column order of `context.X_background`.

The parameters supported by the explainer can be summarized as follows:

- `mode (str)`: type of predictive task explained by BreakDown. If its value is `"classification"`, the wrapper uses the model's `predict_proba()` method. For any other value, it uses `predict()`. (*Default:* `"classification"`)

- `output_index (int)`: output column selected from the array returned by `predict_proba()` in classification mode. For binary classifiers, index 1 commonly corresponds to the positive class. (*Default:* `1`)
- `label (str)`: label assigned to the DALEX model explainer. If no value is provided, the name of the model class is used. This label identifies the model within the DALEX object but does not modify the generated feature contributions. (*Default:* model class name)
- `verbose (bool)`: whether DALEX should print additional information while constructing the model explainer. (*Default:* `False`)
- `precalculate (bool)`: whether DALEX should precalculate model diagnostics when the internal `dalex.Explainer` object is created. Enabling this option may increase the initialization cost but allows DALEX to prepare information reused by its explanation methods. (*Default:* `True`)
- `order (Any)`: order in which the features are introduced into the sequential decomposition. Since BreakDown contributions depend on this order, changing the parameter may change the individual attribution values. When no order is specified, the default ordering strategy provided by DALEX is used. (*Default:* `None`)
- `n_samples (int or None)`: number of samples passed to the `N` argument of `predict_parts()`. This value controls the sampling used internally by DALEX when estimating the quantities required by the explanation. When it is not provided, the default DALEX behaviour is used. (*Default:* `None`)
- `keep_distributions (bool)`: whether DALEX should retain the distributions of the estimated contributions. These distributions may contain additional diagnostic information, although the `XAIMetrics` wrapper only returns the final attribution matrix. (*Default:* `False`)
- `random_state (int)`: random seed passed to `predict_parts()`. Fixing this value improves the reproducibility of computations involving random sampling. (*Default:* `42`)
- `abs (bool)`: whether to convert the generated contributions into absolute values before returning them. When enabled, the direction of each contribution is discarded and only its magnitude is preserved. (*Default:* `False`)

The explainer accepts input observations represented as a `pandas.DataFrame`, a PyTorch `Tensor`, a `NumPy` array, or another array-like object. Data frames are reordered according to the columns of `X_background`. PyTorch tensors are detached, transferred to the CPU, and converted into `NumPy` arrays. One-dimensional inputs are reshaped automatically into a batch containing one observation.

The prediction function used by BreakDown depends on the configured mode. In classification mode, the implementation searches for a `predict_proba()` method. In any other mode, it searches for `predict()`. The required method is first searched directly on the supplied model and then on its internal `model` attribute. If it is unavailable in both locations, the explainer raises an `AttributeError`.

In classification mode, the prediction array is converted into a `NumPy` array. If it has two dimensions, the column specified through `output_index` is selected:

$$\widehat{f}(x) = [\text{predict_proba}(x)]_{\texttt{output_index}}\,.$$

If the prediction function already returns a one-dimensional array, its values are used directly. In regression mode, the complete output of `predict()` is passed to DALEX.

The `targets` argument accepted by `explain()` is not used by this explainer. The model output to decompose is controlled by the prediction function and, in classification mode, by `output_index`. Therefore, every observation in the same execution is explained with respect to the same selected output column.

The internal `dalex.Explainer` object is created lazily the first time a particular model is explained. The implementation caches one DALEX explainer for each model object using its identity. Subsequent calls with the same model reuse the cached object, avoiding repeated initialization with the background data.

DALEX returns the sequential decomposition as a data frame containing feature names, contributions, and auxiliary rows such as the initial reference value and final prediction. The wrapper ignores rows whose names begin with an underscore and extracts the contribution associated with each input feature.

Some DALEX rows may represent a feature using a string that includes both its name and value, such as `feature_name = feature_value`. When a returned name does not exactly match a column in the background data, the wrapper extracts the text before `" = "` and uses it as the feature name. The corresponding contribution is then placed in the position defined by `context.X_background`.

If the same feature appears in more than one result row, its contributions are accumulated. Features that do not appear in the DALEX result retain an attribution value of 0. This conversion produces a fixed-size vector independently of the row structure returned by `predict_parts()`.

The explainer computes a separate BreakDown explanation for each observation in the input batch. Therefore, the computational cost increases approximately linearly with the number of observations to explain.

The explainer returns a two-dimensional `NumPy` array with one row for each explained observation and one column for each input feature:

$$A \in \mathbb{R}^{n \times d},$$

where n is the number of observations in the explained batch and d is the number of features in `X_background`.

When `abs=False`, the returned values preserve the direction of the contributions. When `abs=True`, the absolute-value operation is applied after all individual explanations have been converted into the attribution matrix.

The explainer is implemented through the `BreakDownExplainer` class, located in the module `xai_metrics.explainers.breakdown`. It is registered under the name `"BreakDown"` and can be executed directly using this class (Section 5.3.1) or as part of the automated explanation pipeline provided by `run_explanation()` (Section 5.3.2).

Both usage modes rely on the same `YAML` configuration file. The example generates BreakDown explanations for an Isolation Forest model on the MetroPT3 dataset. The explainer operates in classification mode, decomposes the probability corresponding to output 1, and uses a fixed random seed of 42. Since no explicit feature order is configured, the default ordering behaviour provided by DALEX is used.

Examples of both usage modes are available in the library under: `examples/specific_examples/explainers/BreakDown_examples.py`

5.3.1 Direct explanation using the BreakDownExplainer class

In the direct usage mode, the explanation context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, background data, optional background labels, observations to explain, target values, and execution device required by the explanation process.

After constructing the context, the `BreakDownExplainer` class is instantiated directly using the desired parameters. Calling the `explain()` method with the model and input batch returns the complete attribution matrix. Although the target values are passed in the example for consistency with the common explainer interface, they are not used by BreakDown.

```
from xai_metrics.config import ConfigController
from xai_metrics.explainers.breakdown import BreakDownExplainer

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()
explainer = BreakDownExplainer(
    context=context,
    params={
        "mode": "classification",
        "output_index": 1,
        "random_state": 42,
    },
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch,
)
```

The resulting array contains one row for every observation in `context.X_batch` and one column for every feature in `context.X_background`. The contributions preserve their signs because the `abs` parameter is not enabled.

The mean absolute attribution calculated in the complete example summarizes the average magnitude of the generated contributions. However, it should not be interpreted as a direct measure of explanation quality, since it depends on the scale of the selected model output and on the distribution of the contributions across the features.

5.3.2 Automated explanation using run_explanation()

The automated usage mode uses the `run_explanation()` function. The explanation context, the explainers to be executed, and their parameters are obtained from the `YAML` configuration file.

The optional `selected_explainers` argument can be used to restrict the execution to a subset of the configured explanation methods. In this example, it selects only the registered `"BreakDown"` explainer, although this argument would not be necessary if BreakDown were the only explainer defined in the configuration.

```python
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"
results = run_explanation(
    selected_explainers=["BreakDown"],
    config=config_path,
    attribution_output_dir=None,
)
```

The returned structure contains the experimental metadata and the paths of any attribution files generated during the execution. When no output directory is provided, the attribution values remain available in the returned result without being exported through the argument shown in the example.

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

explainers:
  - name: BreakDown
    params:
      mode: classification
      output_index: 1
      random_state: 42
```

The paths to `X_background` and `y_background` provide the reference data used to initialize the DALEX model explainer. The paths to `X_batch` and `y_batch` identify the observations and targets associated with the explanation batch. However, `y_batch` does not determine the explained output in this implementation because BreakDown uses the fixed `output_index` selected in the configuration.

In the configuration shown above, `xai_method_name` has been changed to `"BreakDown"` so that the experimental metadata are consistent with the explanation method being executed.

5.4 Model Agnostic Supervised Local Explanations (MAPLE)

Model Agnostic Supervised Local Explanations (MAPLE) generates local explanations by combining a tree-based supervised neighbourhood definition with a local linear model [24]. Unlike methods that define locality only through a geometric distance in the original feature space, MAPLE uses a tree ensemble to identify background observations that follow decision paths similar to those of the observation being explained.

The method first fits a tree ensemble to approximate the output of the original predictive model over a background dataset. The leaves of this ensemble define a supervised representation of locality: two observations are considered more similar when they fall into the same leaves across a larger number of trees.

For an observation x, the local weight assigned to a background observation x_i can be expressed as:

$$w_i(x) = \sum_{t=1}^{T} \frac{\mathbb{I}\left[L_t(x_i) = L_t(x)\right]}{|\{j \mid L_t(x_j) = L_t(x)\}|},$$

where T is the number of trees in the ensemble, $L_t(x)$ is the leaf reached by observation x in tree t, and $\mathbb{I}[\cdot]$ is the indicator function. A background observation receives a contribution from a tree only when it shares the same leaf as the explained observation. This contribution is divided by the number of training observations contained in that leaf.

Consequently, observations that share leaves with x across many trees receive larger local weights. The division by the leaf size prevents observations in large leaves from receiving the same influence as observations belonging to smaller and more specific leaves.

After obtaining these weights, MAPLE fits a local Ridge regression model for each observation. For an observation x, the intercept and coefficient vector are obtained as follows:

$$(\beta_{0,x}, \boldsymbol{\beta}_x) = \underset{\beta_0,\, \boldsymbol{\beta}}{\arg\min} \left[\sum_{i=1}^{n} w_i(x) \left(y_i - \beta_0 - \boldsymbol{\beta}^\top x_i \right)^2 + \lambda \left\| \boldsymbol{\beta} \right\|_2^2 \right],$$

where x_i is a background training observation, y_i is the scalar response produced by the original model for that observation, $w_i(x)$ is its local weight with respect to x, and λ is the Ridge regularisation strength. The intercept $\beta_{0,x}$ is fitted but is not returned as an attribution. The vector $\boldsymbol{\beta}_x$ contains the local feature coefficients returned by the explainer.

The coefficients of this local Ridge model are returned as feature-attribution values. A positive coefficient indicates that increasing the corresponding feature is associated with an increase in the selected model response within the supervised local neighbourhood. Conversely, a negative coefficient indicates a local association with a decrease in that response. The coefficient magnitude represents the strength of this local linear relationship.

Since the attributions are coefficients of a locally fitted linear model, their magnitude also depends on the scale of the input variables. Therefore, coefficients associated with features measured on different scales should be compared with caution unless the input data have been appropriately standardized.

The implementation provided by `XAIMetrics` follows the main MAPLE principle using estimators from `scikit-learn`. It combines a `RandomForestRegressor` or `GradientBoostingRegressor` with locally weighted `Ridge` models.

The background data stored in the `ExplainerContext` are divided into training and validation subsets. The training subset is used to fit the tree ensemble and the local Ridge models. The validation subset is used to determine how many features should be retained in the local explanations.

The model responses used to train the internal MAPLE model are not taken from `y_background`. Instead, the wrapper obtains them directly from the predictive model:

$$y_i = \widehat{f}(x_i),$$

where $\widehat{f}$ denotes the scalar model response selected by the wrapper. In classification mode, the response is the probability associated with the output selected through `output_index`. In regression mode, the standard prediction returned by the model is used.

The parameters supported by the explainer can be summarized as follows:

- `mode (str)`: type of predictive task explained by MAPLE. If its value is `"classification"`, the wrapper uses the model's `predict_proba()` method. For any other value, it uses `predict()`. (*Default:* `"classification"`)
- `output_index (int)`: output column selected when the model prediction is a two-dimensional array. In binary classification, index 1 commonly corresponds to the positive class. (*Default:* `1`)
- `validation_size (float)`: fraction of background observations reserved for validation. The value must be strictly between 0 and 1. The validation subset is used to select the number of features retained by the local linear models. (*Default:* `0.2`)
- `data_type (str)`: strategy used to divide the background data into training and validation subsets. Supported values are `"tabular"` and `"time_series"`. In tabular mode, the observations are divided randomly. In time-series mode, the final observations are reserved for validation to preserve temporal order. (*Default:* `"tabular"`)
- `fe_type (str)`: type of tree ensemble used to define the supervised local neighbourhood. The value `"rf"` selects a random forest, while `"gbrt"` selects gradient boosting regression trees. Any other value raises a `ValueError`. (*Default:* `"rf"`)
- `n_estimators (int)`: number of trees included in the ensemble. Increasing this value may provide a more detailed and stable neighbourhood definition but also increases the fitting and explanation costs. (*Default:* `200`)
- `max_features (float, int, str or None)`: maximum number or fraction of features considered when constructing the tree splits. Its interpretation follows the corresponding `scikit-learn` ensemble parameter. (*Default:* `0.5`)
- `min_samples_leaf (int)`: minimum number of training observations required in each tree leaf. Smaller values produce more specific leaves and potentially more localized neighbourhoods, whereas larger values produce broader neighbourhoods containing more observations. (*Default:* `10`)
- `regularization (float)`: Ridge regularisation strength used by all local linear models. Larger values shrink the local coefficients more strongly and may improve numerical stability, while smaller values allow the local model to fit the weighted responses more closely. (*Default:* `0.001`)
- `random_state (int or None)`: random seed used by the tree ensemble and, in tabular mode, by the background-data split. Fixing this value improves reproducibility. (*Default:* `42`)
- `n_jobs (int or None)`: number of parallel jobs used by `RandomForestRegressor`. A value of `-1` uses all available processors. This parameter does not affect the gradient-boosting implementation. (*Default:* `None`)
- `abs (bool)`: whether to convert the generated local coefficients into absolute values before returning them. When enabled, the direction of each local relationship is discarded and only its magnitude is retained. (*Default:* `False`)

The explainer requires at least three observations in `context.X_background`. If fewer observations are available, the constructor raises a `ValueError`. The background data must also be divisible into non-empty training and validation subsets according to the configured `validation_size`.

For tabular data, the background dataset is divided using `train_test_split()` with random shuffling:

$$X_{\text{background}} = X_{\text{train}} \dot{\cup} X_{\text{val}}.$$

For time-series data, the validation subset is formed from the final observations:

$$X_{\text{train}} = (x_1, \ldots, x_k)\,, \qquad X_{\text{val}} = (x_{k+1}, \ldots, x_n)\,.$$

This chronological division prevents future observations from being included in the training subset when the background data follow a temporal order.

The explainer accepts input observations represented as a `pandas.DataFrame`, a PyTorch `Tensor`, a `NumPy` array, or another array-like object. Data frames are reordered according to the columns of `X_background`. PyTorch tensors are detached, transferred to the CPU, and converted into `NumPy` arrays. One-dimensional inputs are reshaped automatically into a batch containing one observation.

The prediction function used by MAPLE depends on the configured mode. In classification mode, the implementation searches for a `predict_proba()` method. In any other mode, it searches for `predict()`. The required method is first searched directly on the supplied model and then on its internal `model` attribute. If it is unavailable in both locations, the explainer raises an `AttributeError`.

When the model returns a two-dimensional prediction array, the wrapper selects the column specified through `output_index`:

$$\widehat{f}(x) = [\text{predict_proba}(x)]_{\texttt{output_index}}\,.$$

If the configured index is outside the available output dimensions, a `ValueError` is raised. One-dimensional model responses are used directly.

The `targets` argument accepted by `explain()` is not used by this implementation. The explained response is determined by the prediction function and, for multi-output predictions, by the fixed `output_index`. Consequently, all observations within the same execution are explained with respect to the same model output.

After fitting the tree ensemble, the implementation calculates the leaf reached by each training observation in every tree. For an observation to explain, its leaf identifiers are compared with those of the training observations. The local weight of each training observation is then calculated according to the leaves it shares with the explained instance.

The implementation also performs a feature-selection stage. Each feature receives a score based on the trees in which it appears at the root node. Specifically, the impurity value of the root node is accumulated for the feature used in that split. Features are then ordered from the highest to the lowest accumulated score.

For every possible number of retained features, from one feature to the complete feature set, the implementation evaluates the corresponding local models on the validation subset. For each validation observation, it computes the MAPLE weights, fits a weighted Ridge model using the candidate feature subset, and obtains a local prediction.

The validation error for each candidate subset is measured using the Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{n_{\text{val}}} \sum_{i=1}^{n_{\text{val}}} (\widehat{y}_i - y_i)^2}.$$

The feature subset that produces the lowest validation RMSE is retained. Features excluded during this process receive an attribution value of 0 in every local explanation.

Once the internal MAPLE model has been constructed, each new observation is passed through the tree ensemble to obtain its leaf identifiers. These identifiers determine the weights assigned to the training observations. A new weighted Ridge model is then fitted using the selected features, and its coefficients form the local explanation.

A separate local Ridge model is therefore fitted for every explained observation. The tree ensemble and selected feature subset are reused, but the local weights and coefficients change according to the observation.

The internal MAPLE model is created lazily the first time a particular predictive model is explained. The implementation caches one MAPLE object for each model instance using its identity. Subsequent calls with the same model reuse the fitted ensemble, validation-selected feature subset, and background split.

The explainer returns a two-dimensional `NumPy` array with one row for each explained observation and one column for every feature in `X_background`:

$$A \in \mathbb{R}^{n \times d},$$

where n is the number of explained observations and d is the number of input features.

When `abs=False`, the returned coefficients preserve their signs. When `abs=True`, the absolute-value operation is applied after the complete attribution matrix has been generated.

The explainer is implemented through the `MAPLEExplainer` class, located in the module `xai_metrics.explainers.maple`. It is registered under the name `"MAPLE"` and can be executed directly using this class (Section 5.4.1) or as part of the automated explanation pipeline provided by `run_explanation()` (Section 5.4.2).

Both usage modes rely on the same `YAML` configuration file. The example generates MAPLE explanations for an Isolation Forest model on the MetroPT3 dataset. The explainer operates in classification mode and explains output 1. It uses a random forest containing 200 trees, considers 50% of the features during tree construction, requires at least 10 observations per leaf, applies a Ridge regularisation strength of 0.001, and reserves 20% of the background data for validation.

Examples of both usage modes are available in the library under: `examples/specific_examples/explainers/MAPLE_examples.py`

5.4.1 Direct explanation using the MAPLEExplainer class

In the direct usage mode, the explanation context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, background data, observations to explain, target values, and execution device required by the explanation process.

After constructing the context, the `MAPLEExplainer` class is instantiated directly using the desired parameters. Calling the `explain()` method with the model and the input batch returns the complete attribution matrix. Although target values are passed in the example for compatibility with the common explainer interface, they are not used by MAPLE.

```python
from xai_metrics.config import ConfigController
from xai_metrics.explainers.maple import MAPLEExplainer

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()
explainer = MAPLEExplainer(
    context=context,
    params={
        "mode": "classification",
        "output_index": 1,
        "fe_type": "rf",
        "n_estimators": 200,
        "max_features": 0.5,
        "min_samples_leaf": 10,
        "regularization": 0.001,
        "validation_size": 0.2,
        "random_state": 42,
        "n_jobs": -1,
    },
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch,
)
```

The resulting array contains one row for every observation in `context.X_batch` and one column for every feature in `context.X_background`. Features excluded during the validation-based selection stage contain zero attribution values.

The mean absolute attribution calculated in the complete example summarizes the average magnitude of the local Ridge coefficients. However, it should not be interpreted as a direct measure of explanation quality because it depends on the scale of the input variables, the selected model response, and the Ridge regularisation strength.

5.4.2 Automated explanation using run_explanation()

The automated usage mode uses the `run_explanation()` function. The explanation context, the explainers to be executed, and their parameters are obtained from the `YAML` configuration file.

The optional `selected_explainers` argument can be used to restrict the execution to a subset of the configured explanation methods. In this example, it selects only the registered `"MAPLE"` explainer, although this argument would not be necessary if MAPLE were the only explainer defined in the configuration.

```python
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"
results = run_explanation(
    selected_explainers=["MAPLE"],
    config=config_path,
    attribution_output_dir=None,
)
```

The returned structure contains the experimental metadata, the attribution matrices generated for each configured explainer, and the paths of any exported attribution files. When no output directory is provided, the attribution values remain available in the returned result without being explicitly exported through the argument shown in the example.

The corresponding configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

explainers:
  - name: MAPLE
    params:
      mode: classification
      output_index: 1
      fe_type: rf
      n_estimators: 200
      max_features: 0.5
      min_samples_leaf: 10
      regularization: 0.001
      validation_size: 0.2
      random_state: 42
      n_jobs: -1
```

The path to `X_background` provides the observations divided into the training and validation subsets used by MAPLE. The values in `y_background` are not used as the responses of the internal surrogate model because the wrapper obtains these responses directly from the model being explained.

The paths to `X_batch` and `y_batch` identify the observations and associated targets in the explanation batch. However, `y_batch` does not determine the explained output in this implementation because MAPLE uses the fixed `output_index` selected in the configuration.

5.5 Running and comparing several explanation methods

The automated workflow can execute several explanation methods from the same experimental configuration. This is useful when the objective is to generate attribution matrices for the same model and observations and then compare those explanations using the metrics implemented in `XAIMetrics`. However, the methods should not be treated as interchangeable estimators of exactly the same mathematical quantity. LIME fits a perturbation-based local surrogate, SHAP constructs an additive coalition-based explanation, BreakDown performs a sequential and potentially order-dependent decomposition, and MAPLE fits a local linear model over a tree-defined supervised neighbourhood.

The following configuration executes the four available explainers for the datasets and models discovered by the runner. The selected model output is kept consistent across the methods: LIME explains class 1, while SHAP, BreakDown, and MAPLE select output column 1. Signed attributions are retained in all cases.

```yaml
context:
  device: "cpu"
  datasets_dir: "examples/datasets"
  models_dir: "examples/models"
  attributions_dir: "examples/attributions"

explainers:
  - name: LIME
    params:
      mode: classification
      labels: [1]
      num_samples: 5000
      distance_metric: euclidean
      random_state: 42

  - name: SHAP
    params:
      mode: classification
      algorithm: auto
      max_background_samples: 100
      output_index: 1
      random_state: 42
      abs: false

  - name: BreakDown
    params:
      mode: classification
      output_index: 1
      random_state: 42
      abs: false

  - name: MAPLE
    params:
      mode: classification
      output_index: 1
      fe_type: rf
      n_estimators: 200
      max_features: 0.5
      min_samples_leaf: 10
      regularization: 0.001
      validation_size: 0.2
      random_state: 42
      n_jobs: -1
      abs: false
```

The experiment is executed as follows:

```python
from xai_metrics.runner import run_explanation

results = run_explanation(
    config="examples/config_example.yaml",
    attribution_output_dir="examples/attributions"
)
```

For each valid context, the returned object contains the experimental metadata and one attribution matrix for every successfully executed explainer. It also records skipped or failed executions and the paths of the exported attribution files. These files can subsequently be loaded by

`run_evaluation()` so that the explanation methods are assessed with the same metric configuration.

A fair comparison requires consistency across the aspects shared by the different methods. In particular, all explainers should use the same predictive model and the same observations from `X_batch`, explain the same scalar model output, including the same class or output index, and apply identical preprocessing operations while preserving the same feature names and feature order. They should also use the same background observations from `X_background`, follow the same convention regarding signed or absolute attribution values, and use fixed and comparable random seeds whenever sampling or stochastic model fitting is involved.

Even under these controls, some method-specific differences remain unavoidable. The background data are used for different purposes, the locality definitions are not equivalent, and the parameters controlling computational effort are method-specific. For example, `num_samples` controls the perturbation set used by LIME, `max_background_samples` limits the SHAP reference data, `order` can alter the BreakDown decomposition, and the MAPLE ensemble and validation split determine its supervised neighbourhood and selected feature subset.

The returned attribution magnitudes are also not directly comparable in every setting. LIME and MAPLE return coefficients of local linear models and are therefore affected by feature scaling. SHAP and BreakDown return additive contributions on the scale of the selected model output. Consequently, comparisons should primarily rely on evaluation metrics designed for explanation properties rather than on raw attribution magnitudes alone.

Maintaining a common experimental context while documenting these method-specific choices prevents differences in preprocessing, target selection, or reference data from being incorrectly interpreted as differences in explanation quality.

CHAPTER

6

Extending the XAIMetrics Library

This chapter describes how to extend the XAIMetrics library by incorporating new evaluation metrics and explanation methods. The objective is to allow developers and researchers to add new components without modifying the main experiment-execution workflow.

XAIMetrics separates metric evaluation and explanation generation through two common abstractions: `BaseMetric` and `BaseExplainer`. Metrics receive a `MetricContext` and implement their computation through the `run()` method, whereas explainers receive an `ExplainerContext` and implement the `explain()` method.

The library also maintains separate registries for metrics and explainers. New implementations are associated with these registries through the `@register_metric` and `@register_explainer` decorators. Registration takes place when the module containing the decorated class is imported by the corresponding automatic-discovery mechanism.

Consequently, inheriting from a base class is not sufficient by itself. A new component must inherit from the appropriate base class, define a unique value for the `NAME` class attribute, implement the required common interface, use the corresponding registration decorator, and be placed in an importable module within the appropriate XAIMetrics package.

Once registered, a metric can be selected by name in the `metrics` section of a YAML configuration and executed through `run_evaluation()`. Similarly, a registered explainer can be selected in the `explainers` section and executed through `run_explanation()`.

6.1 Adding a New Metric

A metric in XAIMetrics evaluates one or more properties of a feature-attribution explanation. Each metric receives a `MetricContext`, which groups the model, test observations, target values, attribution matrix, selected observations, and execution device required by the evaluation.

The metric parameters are supplied as a mapping and stored by the base class. The implementation retrieves the required values from `self.context` and `self.params`, performs the evaluation, and returns the resulting score or scores from `run()`.

Depending on the metric, the returned value may be a scalar, a Boolean value, or a sequence containing one result for each evaluated observation. Returning one value per observation allows XAIMetrics to generate both aggregated and observation-level reports.

6.1.1 Metric Implementation Process

To add a new metric to XAIMetrics, follow these steps:

1. **Clone and install the repository in editable mode.**

```
git clone https://github.com/kdis-lab/XAIMetrics.git
cd XAIMetrics
pip install -e .
```

   Editable installation allows changes in the source package to be used without reinstalling the project after every modification.

2. **Select the appropriate metric category.**
   Metric implementations are organized under `xai_metrics/metrics/` according to the explainability property they evaluate.
   A new metric should be added to the category that best represents its evaluation objective. For example:

```
xai_metrics/
metrics/
robustness/
new_robustness_metric.py
```

3. **Create a class that inherits from `BaseMetric`.**
   The implementation must import `BaseMetric`, `MetricContext`, and `register_metric`. It must define a unique `NAME` value and implement the `run()` method.

4. **Register the class.**
   The class must be decorated with `@register_metric`. The value of `NAME` becomes the identifier used in YAML configurations and in the `selected_metrics` argument of `run_evaluation()`.
   Metric names are case-sensitive and should not duplicate the identifier of an existing implementation.

5. **Define and validate the configurable parameters.**
   Metric-specific parameters are obtained from `self.params`. Default values should be assigned explicitly through `dict.get()`, and invalid values should raise informative exceptions.
   For example:

```
threshold = float(self.params.get("threshold", 0.1))

if threshold < 0:
raise ValueError("threshold must be non-negative.")
```

   Parameters should use YAML-compatible values whenever they are expected to be configured declaratively. Python callables or other runtime objects that cannot be represented in YAML should be supplied through the dependency-injection mechanisms supported by the runner.

6. **Implement the metric calculation in `run()`.**
   The method should retrieve only the context fields required by the metric. Common fields include:

   - `context.model`: predictive model being evaluated.

- `context.X_test`: complete test dataset.
- `context.y_test`: test targets.
- `context.attributions`: attribution values to evaluate.
- `context.observations`: indices of the evaluated observations.
- `context.device`: execution device.

The implementation should preserve the correspondence between observations, targets, and attribution rows.

7. **Handle incompatible contexts.**
   Invalid parameter values or malformed inputs should normally raise an appropriate exception such as `ValueError`. When the metric is valid but cannot be applied to a particular model, attribution method, or experimental context, it may raise `MetricSkipped`. This allows the remaining metrics and contexts to continue executing.

8. **Document the implementation.**
   The class and its methods should include docstrings describing the property evaluated by the metric, the expected context information, all configurable parameters and their default values, the returned result and its interpretation, whether higher or lower values are preferable, any possible exceptions or conditions under which the metric is skipped, and the corresponding scientific reference when applicable.

9. **Add automated tests.**
   Tests should be added under `tests/`. They should verify the direct execution of the class as well as its integration with the automatic registry and runner.

10. **Document the metric.**
    Provide detailed docstrings in the class and its methods, including parameter descriptions, usage examples, and references if applicable.

11. **Run the complete test suite.**

```
pytest
```

12. **Commit the changes and submit a pull request.**
    The pull request should describe the evaluated property, implementation decisions, parameter defaults, external dependencies, tests, and scientific references associated with the metric.

6.1.2 Metric Interface Requirements

A metric implementation must inherit from `BaseMetric`, use the `@register_metric` decorator, define a unique string identifier through the `NAME` class attribute, receive a `MetricContext`, accept an optional mapping of parameters, implement the `run()` method, and return a result that can be processed by the reporting workflow.

When the metric produces one score for each evaluated observation, the returned sequence should preserve the same order as `context.observations`. When it produces a single aggregated value, this behaviour should be documented clearly.

6.1.3 Example Metric Implementation

The following example illustrates the structure of a simple metric that calculates the mean absolute attribution for each evaluated observation. Its purpose is to demonstrate the extension interface, so the computation should be replaced by the logic of the new evaluation metric.

```python
import numpy as np
from xai_metrics.base import BaseMetric, register_metric

@register_metric
class NewMetric(BaseMetric):
    """
    Example implementation of a metric for XAIMetrics.

    The metric returns the mean absolute attribution value
    for each evaluated observation.
    """

    NAME = "NewMetric"

    def run(self):
        attributions = np.asarray(self.context.attributions)

        normalise = self.params.get("normalise", False)

        if normalise:
            denominator = np.sum(np.abs(attributions), axis=1, keepdims=True)

            denominator[denominator == 0] = 1
            attributions = attributions / denominator

        return np.mean(np.abs(attributions), axis=1)
```

The decorator registers the class using the value `"NewMetric"`. Once the module has been discovered and imported, the metric can be referenced by that identifier.

6.1.4 Metric Configuration and Execution

The new metric can be configured in `YAML` as follows:

```yaml
metrics:
    - name: NewMetric
      params:
        normalise: true
```

It can be executed automatically through the evaluation runner:

```python
from xai_metrics.runner import run_evaluation

results = run_evaluation(
    selected_metrics=["NewMetric"],
    config="examples/specific_examples/config.yaml",
    report_output_dir=None
)
```

The metric can also be instantiated directly:

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.complexity.new_metric import NewMetric

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()

metric = NewMetric(
    context=context,
    params={"normalise": True}
)
scores = metric.run()
```

In both cases, the metric returns one score for each evaluated observation.

6.1.5 Testing a New Metric

The tests for a new metric should verify its correct registration under the identifier defined by `NAME`, the default parameter values and the handling of explicitly configured parameters, the expected numerical results for a small deterministic example, the number and order of the returned observation-level scores, its behaviour with zero, negative, constant, or missing attribution values, the validation of unsupported parameter values, the use of `MetricSkipped` when the experimental context is incompatible, its direct execution through the `run()` method, and its execution through `run_evaluation()` using a `YAML` configuration.

When the implementation wraps an external library, tests should also confirm that the arguments configured in XAIMetrics are forwarded correctly and that the returned external result is converted into the representation expected by the reporting module.

6.2 Adding a New Explainer

An explainer in XAIMetrics generates feature-attribution values for a predictive model and a batch of input observations. Each implementation receives an `ExplainerContext`, which provides the background data, optional background targets, the predictive model, the observations to explain, their optional targets, and the execution device.

All explainers must return a two-dimensional attribution matrix:

$$A \in \mathbb{R}^{n \times d},$$

where n is the number of explained observations and d is the number of input features. The columns of the returned matrix should follow the same order as `context.X_background`.

This common representation allows attribution matrices generated by different explanation methods to be exported, compared, and evaluated using the metrics available in the library.

6.2.1 Explainer Implementation Process

To add a new explanation method to XAIMetrics, follow these steps:

1. **Create a new module under `xai_metrics/explainers/`.**
   The filename should identify the explanation method clearly. For example:

```
xai_metrics/
explainers/
new_explainer.py
```

2. **Create a class that inherits from `BaseExplainer`.**
   The implementation must import `BaseExplainer`, `ExplainerContext`, and `register_explainer`.
3. **Define a unique explainer identifier.**
   The class must define a `NAME` attribute. This identifier is used in the YAML configuration, in `selected_explainers`, and as the key associated with the generated attribution matrix.
4. **Register the class.**
   The class must use the `@register_explainer` decorator. The implementation module must be importable by the explainer-discovery mechanism.
5. **Initialize the method-specific resources.**
   The constructor can use `context.X_background` and `context.y_background` to initialize the underlying explanation algorithm. Expensive resources may be created lazily and cached when appropriate.
6. **Implement input conversion.**
   For consistency with the explainers already included in the library, the implementation should normally accept:
   - `pandas.DataFrame` objects
   - PyTorch `Tensor` objects
   - `NumPy` arrays
   - Other compatible array-like objects.

   Data frames should be reordered according to the columns of `context.X_background`. One-dimensional inputs should be reshaped into a batch containing one observation.
7. **Resolve the model prediction function.**
   The explainer should document the model interface it requires. Classification explainers commonly use `predict_proba()`, while regression explainers generally use `predict()`.
   When the library may receive wrapped models, the prediction method can be searched first on the supplied object and then on its internal `model` attribute.
8. **Implement `explain()`.**
   The method must use the following common signature:

```
def explain(self, model, inputs, targets=None, **kwargs):
...
```

   The method should generate one attribution vector per observation and return a two-dimensional `NumPy` array.
9. **Define the target-output selection strategy.**
   For classification or multi-output models, the explainer must determine which output is being explained. The implementation may use the supplied `targets`, a configured `output_index`, or another explicitly documented strategy.
   The same selected output must be used consistently when generating and evaluating explanations.

10. **Preserve feature alignment.**
    Each returned attribution column must correspond to the same feature position in `context.X_background`. Features not included by a sparse explanation method should normally receive a value of 0.
11. **Handle unsupported contexts.**
    Invalid parameters should raise informative exceptions. If the explanation method is valid but cannot be applied to a particular model or dataset, the implementation may raise `ExplainerSkipped` so that the remaining experiment can continue.
12. **Document, test, and provide an example.**
    The explainer should include detailed docstrings, automated tests, a direct usage example, a runner example, and a YAML configuration.
13. **Run the complete test suite and submit a pull request.**

6.2.2 Explainer Interface Requirements

A new explainer must inherit from `BaseExplainer`, use the `@register_explainer` decorator, define a unique identifier through the `NAME` class attribute, receive an `ExplainerContext`, accept an optional mapping of parameters, implement the `explain(model, inputs, targets=None, **kwargs)` method, return a two-dimensional `NumPy` attribution matrix, and preserve the feature order defined by the background data.

By inheriting from `BaseExplainer`, the new implementation can also be converted into a callable through `as_explain_func()`. This callable can be supplied to evaluation metrics that must recompute explanations for perturbed observations.

6.2.3 Example Explainer Implementation

The following example implements a simple baseline-perturbation explainer. For each feature, it replaces the original value with the feature-wise mean of the background data and measures the resulting change in the selected model output.

The example illustrates the structure required by XAIMetrics:

```
import numpy as np

from xai_metrics.base import BaseExplainer, register_explainer

@register_explainer
class NewExplainer(BaseExplainer):
"""
Simple example of an explainer for XAIMetrics.

The scale parameter multiplies the returned attributions.
"""

NAME = "NewExplainer"

def explain(self, model, inputs, targets=None, **kwargs):
    scale = self.params.get("scale", 1.0)

    attributions = np.asarray(inputs)

    if attributions.ndim == 1:
```

```
        attributions = attributions.reshape(1, -1)

    return attributions * scale
```

6.2.4 Explainer Configuration and Execution

The explainer can be configured in YAML as follows:

```yaml
explainers:
    - name: NewExplainer
      params:
        scale: 0.5
```

It can be executed automatically through the explanation runner:

```
from xai_metrics.runner import run_explanation

results = run_explanation(
    selected_explainers=["NewExplainer"],
    config="examples/specific_examples/config.yaml",
    attribution_output_dir=None
)
```

The explainer can also be instantiated directly:

```
from xai_metrics.config import ConfigController

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()

explainer = BaselineDifferenceExplainer(
    context=context,
    params={"scale": 0.5}
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch
)
```

The inherited `as_explain_func()` method can be used to create a callable compatible with metrics that recompute explanations:

```
explain_func = explainer.as_explain_func()
```

6.2.5 Testing a New Explainer

The automated tests for a new explainer should verify its correct registration under the identifier defined by `NAME`, its construction from an `ExplainerContext`, the handling of default and explicitly configured parameters, support for data frames, NumPy arrays, and tensors when applicable, automatic reshaping of a single observation, preservation of the feature order defined by `X_background`, the expected output shape $n \times d$, correct target or output-index selection for multi-output models,

compatibility with wrapped model objects, deterministic behaviour when a random state is fixed, correct handling of signed and absolute attributions, informative errors for unsupported models or parameters, the use of `ExplainerSkipped` for incompatible contexts, direct execution through `explain()`, conversion through `as_explain_func()`, and execution through `run_explanation()` using a YAML configuration.

When the explainer wraps an external package, tests should isolate that dependency where possible and verify that XAIMetrics forwards the configured parameters correctly. Tests should also confirm that the package-specific output is converted into the fixed-size attribution matrix expected by the library.

6.3 Documentation and Contribution Checklist

Before submitting a new metric or explainer, the following checklist should be completed:

- The implementation inherits from the correct base class.
- The class defines a unique and descriptive `NAME`.
- The corresponding registration decorator is present.
- The module is placed in the correct package and can be imported.
- Parameters have documented types, defaults, and validation rules.
- Returned values follow the representation expected by the runner and reporting modules.
- Incompatible contexts are handled without unnecessarily stopping the complete experiment.
- Unit and integration tests have been added.
- The complete test suite passes.
- A direct usage example has been added.
- A YAML and runner example has been added.
- The API documentation and user manual have been updated.
- External dependencies and scientific references have been documented.

Once these requirements are satisfied, the new component becomes part of the XAI-Metrics extension ecosystem. A registered metric can be executed manually or through $\texttt{run}_e valuation(), while a registered explainer can be executed through its class, through$

CHAPTER

7

Reporting bugs

Feel free to open an issue at Github if anything is not working as expected https://github.com/kdis-lab/XAIMetrics/issues. Merge request are also encouraged, it will be carefully reviewed and merged if everything is all right.

Bibliography

[1] Emrullah ŞAHiN, Naciye Nur Arslan, and Durmuş Özdemir. Unlocking the black box: an in-depth review on interpretability, explainability, and reliability in deep learning, 1 2025.

[2] In-On Wiratsin and Chaiyong Ragkhitwetsagul. Effectiveness of explainable artificial intelligence (xai) techniques for improving human trust in machine learning models: A systematic literature review. *IEEE Access*, 2025.

[3] Logan Cummins, Alexander Sommers, Somayeh Bakhtiari Ramezani, Sudip Mittal, Joseph Jabour, Maria Seale, and Shahram Rahimi. Explainable predictive maintenance: A survey of current methods, challenges and opportunities. *IEEE Access*, 12:57574-57597, 2024.

[4] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 4765-4774. Curran Associates, Inc., 2017.

[5] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, pages 1135-1144, 2016.

[6] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319-3328. PMLR, 2017.

[7] Alicja Gosiewska and Przemyslaw Biecek. Do not trust additive explanations. *arXiv preprint arXiv:1903.11420*, 2019.

[8] Jianlong Zhou, Amir H. Gandomi, Fang Chen, and Andreas Holzinger. Evaluating the quality of machine learning explanations: A survey on methods and metrics, 3 2021.

[9] Pedro Lopes, Eduardo Silva, Cristiana Braga, Tiago Oliveira, and Luís Rosado. Xai systems evaluation: A review of human and computer-centred methods, 10 2022.

[10] Umang Bhatt, Adrian Weller, and José MF Moura. Evaluating and aggregating feature-based model explanations. *arXiv preprint arXiv:2005.00631*, 2020.

[11] David Alvarez Melis and Tommi Jaakkola. Towards robust interpretability with self-explaining neural networks. *Advances in neural information processing systems*, 31, 2018.

[12] Anna Hedström, Leander Weber, Daniel Krakowczyk, Dilyara Bareeva, Franz Motzkus, Wojciech Samek, Sebastian Lapuschkin, and Marina M-C Höhne. Quantus: An explainable ai toolkit for responsible evaluation of neural network explanations and beyond. *Journal of Machine Learning Research*, 24(34):1-11, 2023.

[13] Sanjoy Dasgupta, Nave Frost, and Michal Moshkovitz. Framework for evaluating faithfulness of local explanations. In *International Conference on Machine Learning*, pages 4794-4815. PMLR, 2022.

[14] An-phi Nguyen and María Rodríguez Martínez. On quantitative aspects of model interpretability. *arXiv preprint arXiv:2007.07584*, 2020.

[15] Marek Pawlicki, Aleksandra Pawlicka, Federica Uccello, Sebastian Szelest, Salvatore D'Antonio, Rafał Kozik, and Michał Choraś. Evaluating the necessity of the multiple metrics for assessing explainable ai: A critical examination. *Neurocomputing*, 602, 10 2024.

[16] Prasad Chalasani, Jiefeng Chen, Amrita Roy Chowdhury, Xi Wu, and Somesh Jha. Concise explanations of neural networks using adversarial training. In *International conference on machine learning*, pages 1383-1391. PMLR, 2020.

[17] Chih-Kuan Yeh, Cheng-Yu Hsieh, Arun Suggala, David I Inouye, and Pradeep K Ravikumar. On the (in) fidelity and sensitivity of explanations. *Advances in neural information processing systems*, 32, 2019.

[18] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. *Advances in neural information processing systems*, 31, 2018.

[19] Thomas Fel, David Vigouroux, Rémi Cadène, and Thomas Serre. How good is your explanation? algorithmic stability measures to assess the quality of explanations for deep neural networks. In *2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1565-1575. IEEE, 2022.

[20] Chirag Agarwal, Nari Johnson, Martin Pawelczyk, Satyapriya Krishna, Eshika Saxena, Marinka Zitnik, and Himabindu Lakkaraju. Rethinking stability for attribution-based explanations. *arXiv preprint arXiv:2203.06877*, 2022.

[21] Aditya Chattopadhay, Anirban Sarkar, Prantik Howlader, and Vineeth N Balasubramanian. Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks. In *2018 IEEE winter conference on applications of computer vision (WACV)*, pages 839-847. IEEE, 2018.

[22] Hanwei Zhanga, Felipe Torresa, Ronan Sicrea, Yannis Avrithisb, and Stephane Ayachea. Opti-cam: Optimizing saliency maps for interpretability. 2024.

[23] Vitali Petsiuk, Abir Das, and Kate Saenko. Rise: Randomized input sampling for explanation of black-box models. *arXiv preprint arXiv:1806.07421*, 2018.

[24] Gregory Plumb, Denali Molitor, and Ameet S Talwalkar. Model agnostic supervised local explanations. *Advances in neural information processing systems*, 31, 2018.